

Although Julia is devised for natively supporting mathematical expressions, Python is still performing with the aid of numpy and scipy modules for scientific purposes. Matplotlib is a best plotting modules operable with Python.

In this paper we will explore the methods of Matplotlib with pyplot extension to plotting scientific data.

We will try to devise our plot programs as simplest as possible, almost reminding the Julia plot examples. The presented Python programs will be more relying on system resources, but in present days, recent desktop architectures are allowing the users to exploit more system resources than the earlier built desktop computers.

E:\Dynamic_Content\eclipse_ws_python\Evolution\
page1simplest.py

#Line plots

#pyplot extension

#if nothing is stated pyplot understands as line

a single line graph

Matplotlib Pyplot extension must be imported but there is varieties of forms for organizing the import.

Organization of imports result less or more utilisation of system resources and affect the verbosity of program statements.

page1-simplest.py

from matplotlib.pyplot import xlabel, ylabel, plot, show

Data

X = [0,1,2]

```
# Axis labels

ylabel('y values')
xlabel('x values')

# plots

plot(X)

# display

show()
```

page1-simplest.py is the one giving briefest program statements but his import statement is very heavy and not practical when too many module properties should be imported.

This form of import is not practical and generally never applied.

Therefore, we will not use this form of import.

page1-common.py

```
import matplotlib.pyplot as plt

# Data

X = [0,1,2]

# Axis labels

plt.ylabel('y values')
plt.xlabel('x values')

# plots

plt.plot(X)

# display

plt.show()
```

page1-common.py is the most used form of imports.

This is probably a form utilising acceptable amount of system resources,

but program statements should state the qualified names.

page1.py

```
from matplotlib.pyplot import *
```

```
# Data
```

```
X = [0,1,2]
```

```
# Axis labels
```

```
ylabel('y values')
```

```
xlabel('x values')
```

```
# plots
```

```
plot(X)
```

```
# display
```

```
show()
```

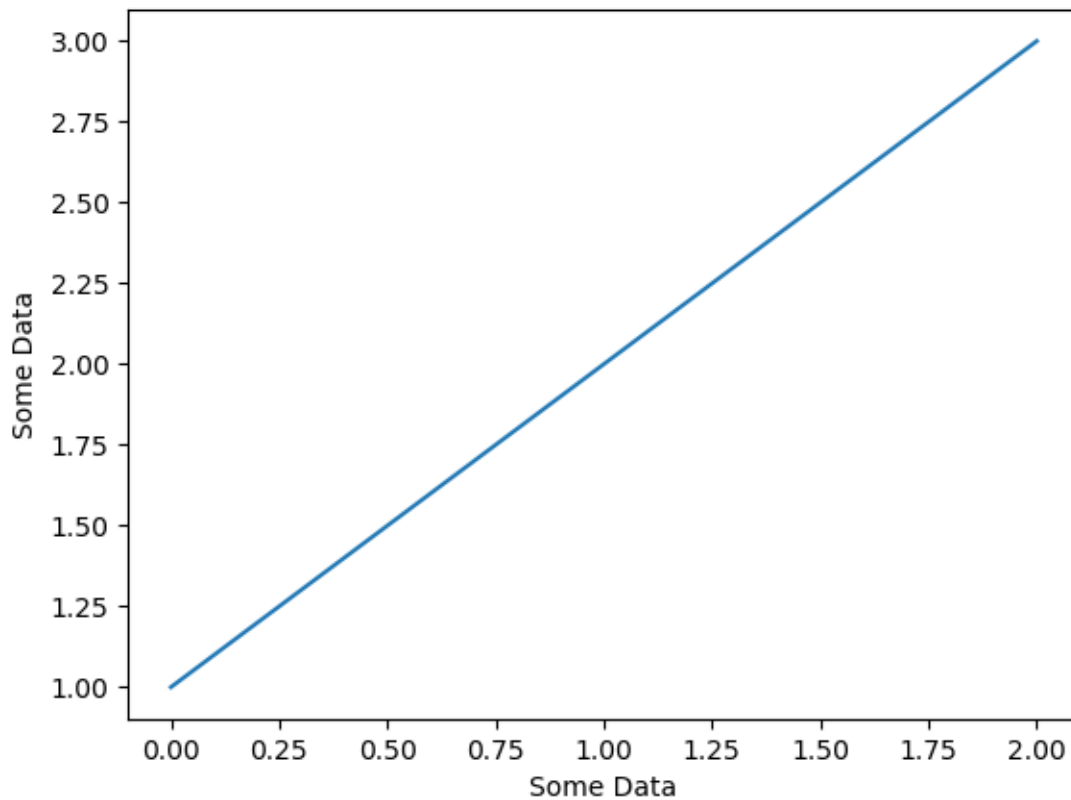
The form of imports of the page1.py program probably is the one which is consuming highest system resources.

It is importing all the properties of the imported modules but some of imported module properties will surely not be utilised.

We'll follow this template because it has the simplest import form and allows the lightest program statements.

Modern desktop computers deliver high system resources to support programs.

#either import forms give the same graph, presented below.



'''

Explanation:

Matplotlib understands [1,2,3] as y data, as it is visible from the graph. Values for x pairs are calculated from a virtual (implicit) list. The y data is [1,2,3] that is 3 points. The (x,y) coordinates are to be found as (x1,y1), (x2,y2), (x3,y3) the knowns are only y coordinates as, (x1,1), (x2,2), (x3,3) the corresponding x coordinates are to be found from a list as [x1,x2,x3] this list is interpreted as list indexes. First index is 0 as it is the initial index of an ordinary Python list. The second is the subsequent of 0, i.e 1, the third is 2 and the list is taught as [0,1,2]. Plotted line is the graph of [0,1,2] versus [1,2,3] and the plotted line is the one passing from the points (0,1), (1,2),(2,3). This interpretation is clearly followed from the graph itself.

Note: Try plotting [3,4,5], this time coordinates will be interpreted as points (0,3), (1,4),(2,5).

```
'''
```

line.py

```
# a single line graph with explicit (x,y) data
# x data are given as an explicit list as X=[x1,x2,x3,...,xn]
# y data are given as an explicit list as Y=[x1,x2,x3,...,xn]
```

```
'''
```

```
from matplotlib.pyplot import *
```

```
# Data
```

```
X = [0,1,2]
```

```
Y = [0,1,2]
```

```
# Axis labels
```

```
ylabel('y values')
```

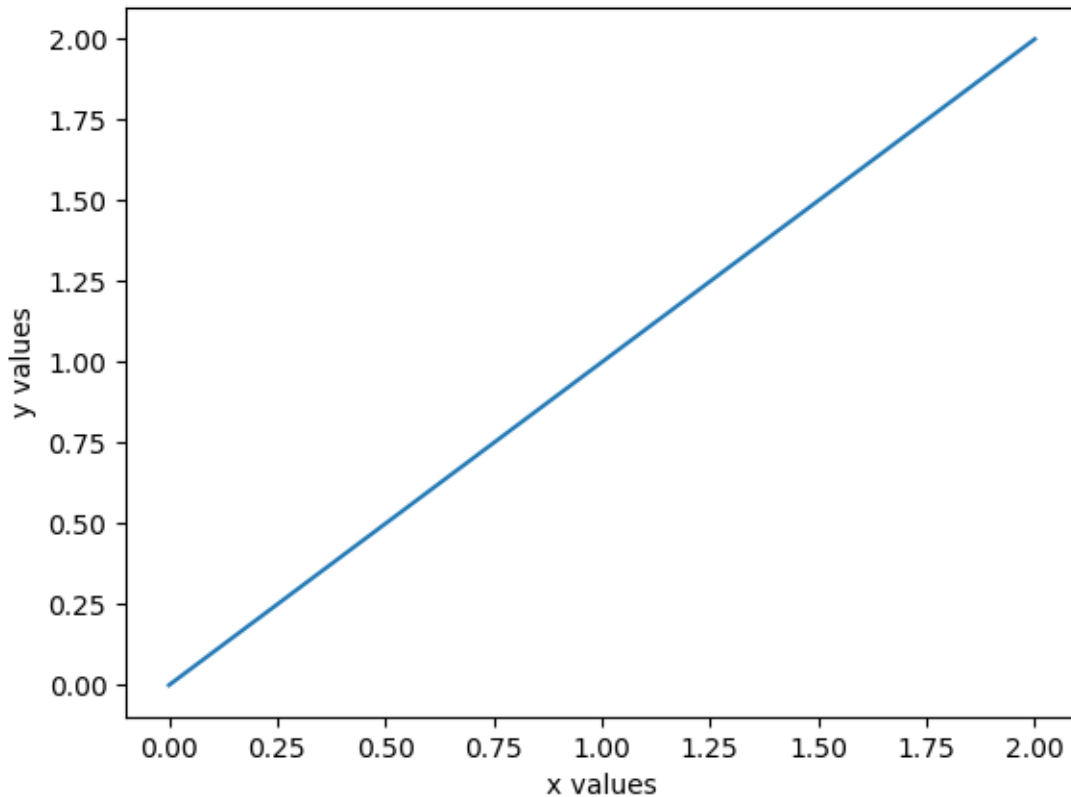
```
xlabel('x values')
```

```
# plots
```

```
plot(X,Y)
```

```
# display
```

```
show()
```



```
# X,Y data may also be given as plot([0,1,2],[1,2,3])
```

```
# Before advancing further, we will learn how to set the  
# dimensions of the plot and the ways of saving the  
# produced plot with line.py program
```

```
# The produced plot may be left to default values  
# or may be handled by using figure property.
```

That is how:

```
matplotlib.pyplot.figure(num=None, figsize=None, dpi=None, facecolor=None, edgecolor=None,
frameon=True, FigureClass=<class 'matplotlib.figure.Figure'>, clear=False, **kwargs)
```

these are default values, below are their meaning and eventually the ways to alter them as user specified values.

num : integer or string, optional, default: none

If not provided, a new figure will be created, and the figure number will be incremented. The figure objects holds this number in a `number` attribute. If num is provided, and a figure with this id already exists, make it active, and returns a reference to it. If this figure does not exists, create it and returns it. If num is a string, the window title will be set to this figure's num.

(My advice: If you are working with a single plot, num is not necessary)

figsize : tuple of integers, optional, default: None

width, height in inches. If not provided, defaults to rc figure.figsize.

dpi : integer, optional, default: None

resolution of the figure. If not provided, defaults to rc figure.dpi.

facecolor :

the background color. If not provided, defaults to rc figure.facecolor.

edgecolor :

the border color. If not provided, defaults to rc figure.edgecolor.

frameon : bool, optional, default: True

If False, suppress drawing the figure frame.

FigureClass : class derived from
matplotlib.figure.Figure

Optionally use a custom Figure instance. (My advice:
Let it as the default)

clear : bool, optional, default: False

If True and the figure already exists, then it is cleared.

figure : Figure

Returns:

The Figure instance returned will also be passed to new_figure_manager in the backends, which allows to hook custom Figure classes into the pylab interface. Additional kwargs will be passed to the figure init function.

Notes

If you are creating many figures, make sure you explicitly call “close” on the figures you are not using, because this will enable pylab to properly clean up the memory. (My advice: Not necessary for a single file)

rcParams defines the default values, which can be modified in the matplotlibrc file (My advice : Do not ever to attempt this!)

Now a normally user defined figure :

figure.py

```
from matplotlib.pyplot import *
```

```
#figure dimensions
```

```
figure(figsize=(9,7), dpi=80)
```

```
# Data
```

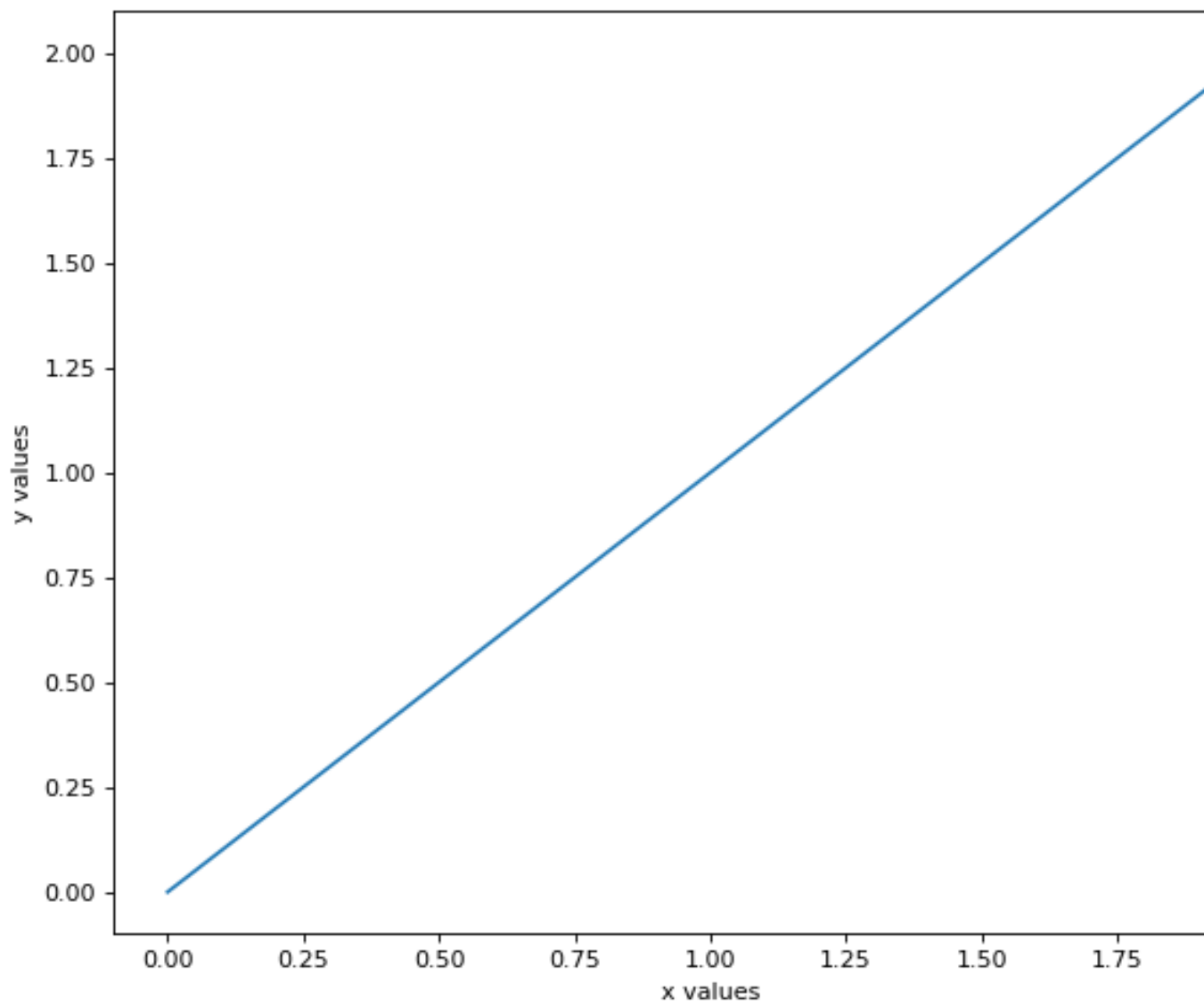
```
X = [0,1,2]
```

```
Y = [0,1,2]
```

```
# Axis labels
```

```
ylabel('y values')
```

```
xlabel('x values')  
# plots  
plot(X,Y)  
# display  
show()
```



The produced plot is bigger than the one produced by the line.py

The reason is : line.py produces a plot with default dimensions,

whereas the figure.py program produce a plot with user defined

dimensions, which are different from default values.

The produced plot may be saved manually by pressing the save tab placed in the bottom of the figure.

Another possibility is to save it automatically by using the savefig("location", "filename.png") statement.

figure1.py

```
from matplotlib.pyplot import *
```

```
#figure dimensions
```

```
figure(figsize=(6,4), dpi=80)
```

```
# Data
```

```
X = [0,1,2]
```

```
Y = [0,1,2]
```

```
# Axis labels
```

```
ylabel('y values')
```

```
xlabel('x values')
```

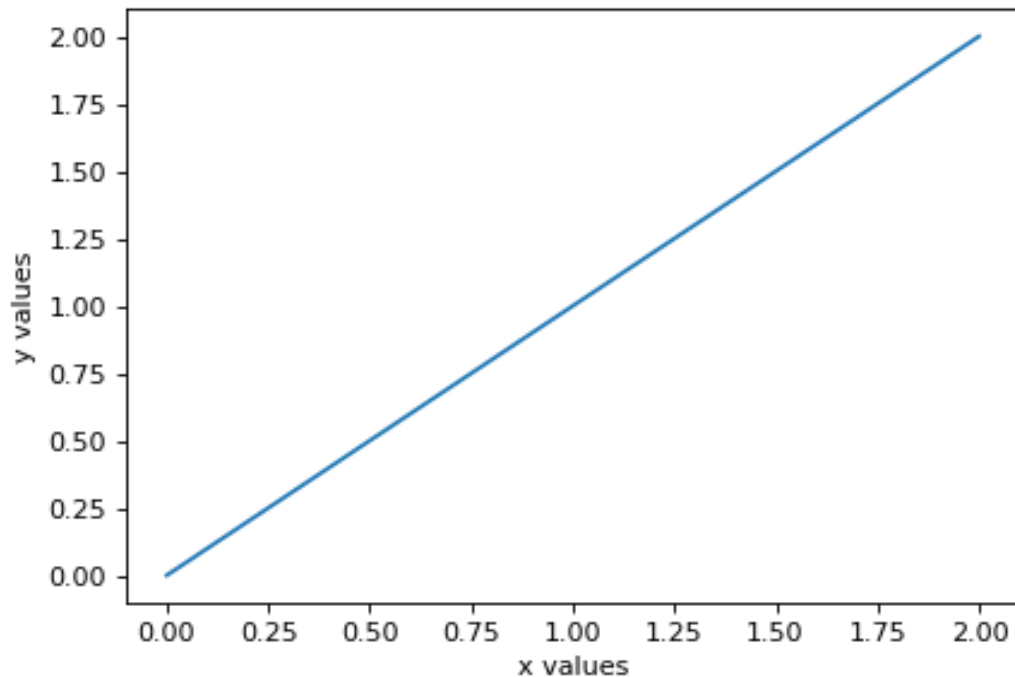
```
# plots
```

```
plot(X,Y)
```

```
savefig("E:\Dynamic_Content\eclipse_ws_python\Evolution\samplefigure1.png")
```

```
# display
```

```
show()
```



```
# We have had a smaller graph which has the dimensions defined in our statement.
```

```
# Subgraphs
```

```
# Subgraps are smaller graphs that fit the portions of the area defined in the figure dimensions.
```

```
# In Matplotlib we can work with several plots in our program and display the in the different subsections of the defined graph area.
```

```
# subplot(1,1,1)
```

Is the default subplot definition, good for one plot per program only and need not to be explicitly stated when working with a single graph per program (as usual!)

```
matplotlib.pyplot.subplot(*args, **kwargs)
```

Return a subplot axes at the given grid position.

Call signature:

```
subplot(nrows, ncols, index, **kwargs)
```

In the current figure, create and return an [Axes](#), at position *index* of a (virtual) grid of *nrows* by *ncols* axes. Indexes go from 1 to *nrows* * *ncols*, incrementing in row-major order.

If *nrows*, *ncols* and *index* are all less than 10, they can also be given as a single, concatenated, three-digit number.

For example, `subplot(2, 3, 3)` and `subplot(233)` both create an [Axes](#) at the top right corner of the current figure, occupying half of the figure height and a third of the figure width.

Note

Creating a subplot will delete any pre-existing subplot that overlaps with it beyond sharing a boundary:

Keyword arguments:

facecolor:

The background color of the subplot, which can be any valid color specifier. See [matplotlib.colors](#) for more information.

polar:

A boolean flag indicating whether the subplot plot should be a polar projection. Defaults to *False*.

projection:

A string giving the name of a custom projection to be used for the subplot. This projection must have been previously registered. See [matplotlib.projections](#).

```
subplot(2,1,1)
```

```
subplot(2,1,2)
```

`subplot(1,2,1)`

`subplot(1,2,2)`

`subplot(2,2,1)`

`subplot(2,2,2)`

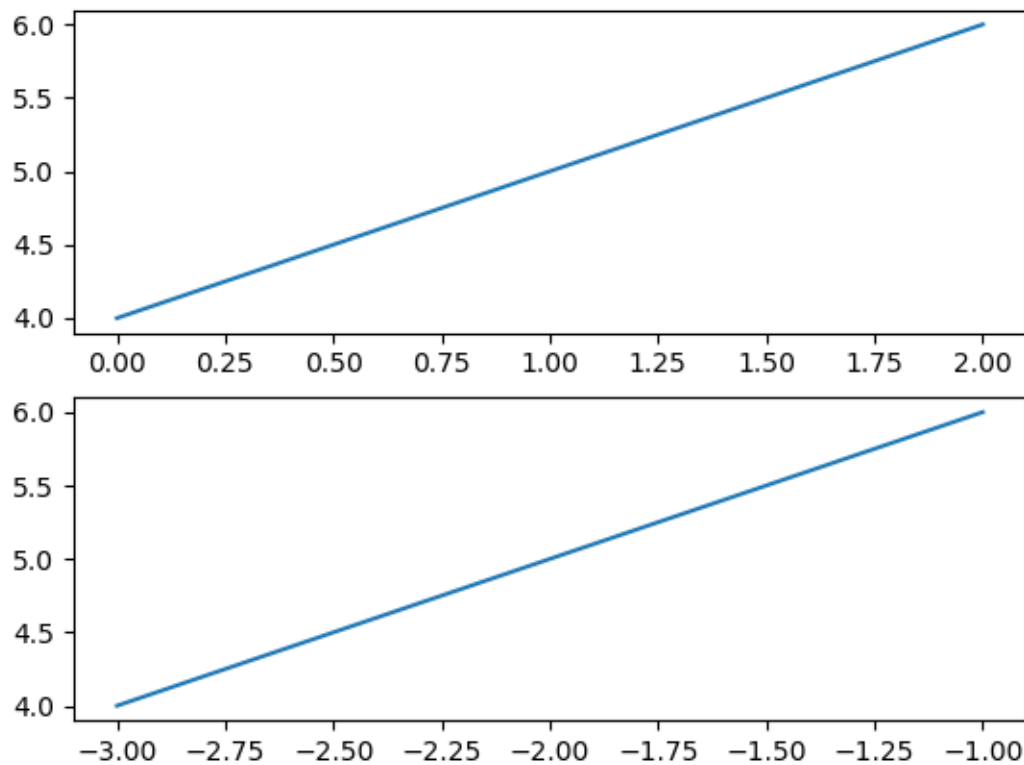


We will make an example by working with two graphs in a single program and display them one below the other in two sections of the predefined figure area.

subplot.py

```
from matplotlib.pyplot import *  
  
# First Graph  
subplot(211)  
  
# Data  
X = [0 , 1 , 2]  
Y = [4 , 5 , 6]  
  
plot(X,Y)
```

```
# Second Graph  
subplot(212)  
X = [-3 , -2 , -1]  
Y = [4 , 5 , 6]  
plot(X,Y)  
show()
```



```
# Line2D properties
```

```
# Matplotlib pyplot extension has several line control  
elements
```

Here are the available [Line2D](#) properties.

Property	Value Type
alpha	float
animated	[True False]
antialiased or aa	[True False]
clip_box	a matplotlib.transform.Bbox instance
clip_on	[True False]
clip_path	a Path instance and a Transform instance, a Patch
color or c	any matplotlib color
contains	the hit testing function
dash_capstyle	['butt' 'round' 'projecting']
dash_joinstyle	['miter' 'round' 'bevel']
dashes	sequence of on/off ink in points
data	(np.array xdata, np.array ydata)
figure	a matplotlib.figure.Figure instance
label	any string
linestyle or ls	['-' '--' '-.' ':' 'steps' ...]
linewidth or lw	float value in points
lod	[True False]
marker	['+' ',' '.' '1' '2' '3' '4']
markeredgecolor or mec	any matplotlib color
markeredgewidth or mew	float value in points
markerfacecolor or mfc	any matplotlib color
markersize or ms	float
markevery	[None integer (startind, stride)]
picker	used in interactive line selection
pickradius	the line pick selection radius
solid_capstyle	['butt' 'round' 'projecting']
solid_joinstyle	['miter' 'round' 'bevel']
transform	a matplotlib.transforms.Transform instance
visible	[True False]
xdata	np.array
ydata	np.array
zorder	any number

To get a list of settable line properties, call the `setp()` function with a line or lines as argument

```
In [69]: lines = plt.plot([1, 2, 3])
```

```
In [70]: plt.setp(lines)
alpha: float
animated: [True | False]
antialiased or aa: [True | False]
...snip
```

```
class matplotlib.lines.Line2D(xdata, ydata, linewidth=None, linestyle=None, color=None,
marker=None, markersize=None, markeredgewidth=None, markeredgewidth=None,
markerfacecolor=None, markerfacecoloralt='none', fillstyle=None, antialiased=None,
dash_capstyle=None, solid_capstyle=None, dash_joinstyle=None, solid_joinstyle=None,
pickradius=5, drawstyle=None, markevery=None, **kwargs)
```

A line - the line can have both a solid linestyle connecting all the vertices, and a marker at each vertex. Additionally, the drawing of the solid line is influenced by the drawstyle, e.g., one can create “stepped” lines in various styles.

Create a [Line2D](#) instance with x and y data in sequences *xdata*, *ydata*.

The kwargs are [Line2D](#) properties:

Property	Description
agg_filter	a filter function, which takes a (m, n, 3) float array and a dpi value, and returns a (m, n, 3) array
alpha	float (0.0 transparent through 1.0 opaque)
animated	bool
antialiased or aa	[True False]
clip_box	a Bbox instance
clip_on	bool
clip_path	[(Path , Transform) Patch None]
color or c	any matplotlib color
contains	a callable function
dash_capstyle	['butt' 'round' 'projecting']
dash_joinstyle	['miter' 'round' 'bevel']
dashes	sequence of on/off ink in points
drawstyle	['default' 'steps' 'steps-pre' 'steps-mid' 'steps-post']
figure	a Figure instance
fillstyle	['full' 'left' 'right' 'bottom' 'top' 'none']

Property	Description
gid	an id string
label	object
linestyle or <code>ls</code>	['solid' 'dashed', 'dashdot', 'dotted' (offset, on-off-dash-seq) '-' '--' '-.' ':' 'None' ' ' '']
linewidth or <code>lw</code>	float value in points
marker	A valid marker style
markeredgecolor or <code>mec</code>	any matplotlib color
markeredgewidth or <code>mew</code>	float value in points
markerfacecolor or <code>mfc</code>	any matplotlib color
markerfacecoloralt or <code>mfcalt</code>	any matplotlib color
markersize or <code>ms</code>	float
markevery	[None int length-2 tuple of int slice list/array of int float length-2 tuple of float]
path_effects	AbstractPathEffect
picker	float distance in points or callable pick function <code>fn(artist, event)</code>
pickradius	float distance in points
rasterized	bool or None
sketch_params	(scale: float, length: float, randomness: float)
snap	bool or None
solid_capstyle	['butt' 'round' 'projecting']
solid_joinstyle	['miter' 'round' 'bevel']
transform	a matplotlib.transforms.Transform instance
url	a url string
visible	bool
xdata	1D array
ydata	1D array
zorder	float

See [set_linestyle\(\)](#) for a decription of the line styles, [set_marker\(\)](#) for a description of the markers, and [set_drawstyle\(\)](#) for a description of the draw styles.

axes

The [Axes](#) instance the artist resides in, or *None*.

`contains(mouseevent)`

Test whether the mouse event occurred on the line. The pick radius determines the precision of the location test (usually within five points of the value). Use [get_pickradius\(\)](#) or [set_pickradius\(\)](#) to view or modify it.

Returns *True* if any values are within the radius along with `{'ind': pointlist}`, where *pointlist* is the set of points within the radius.

TODO: sort returned indices by distance

`draw(renderer)`

draw the Line with `renderer` unless `visibility` is *False*

```
drawStyleKeys = ['default', 'steps-mid', 'steps-pre', 'steps-post', 'steps']
drawStyles = {'default': '_draw_lines', 'steps': '_draw_steps_pre', 'steps-mid':
'_draw_steps_mid', 'steps-post': '_draw_steps_post', 'steps-pre': '_draw_steps_pre'}
fillStyles = ('full', 'left', 'right', 'bottom', 'top', 'none')
filled_markers = ('o', 'v', '^', '<', '>', '8', 's', 'p', '*', 'h', 'H', 'D', 'd', 'P', 'X')
get_aa()
```

alias for `get_antialiased`

```
get_antialiased()
get_c()
```

alias for `get_color`

```
get_color()
get_dash_capstyle()
```

Get the cap style for dashed linestyles

```
get_dash_joinstyle()
```

Get the join style for dashed linestyles

```
get_data(orig=True)
```

Return the `xdata`, `ydata`.

If *orig* is *True*, return the original data.

```
get_drawstyle()  
get_fillstyle()
```

return the marker fillstyle

```
get_linestyle()  
get_linewidth()  
get_ls()
```

alias for get_linestyle

```
get_lw()
```

alias for get_linewidth

```
get_marker()  
get_machedgecolor()  
get_machedgewidth()  
get_markerfacecolor()  
get_markerfacecoloralt()  
get_markersize()  
get_markevery()
```

return the markevery setting

```
get_mec()
```

alias for get_machedgecolor

```
get_mew()
```

alias for get_machedgewidth

```
get_mfc()
```

alias for get_markerfacecolor

```
get_mfcalt(alt=False)
```

alias for get_markerfacecoloralt

```
get_ms()
```

alias for get_markersize

```
get_path()
```

Return the [Path](#) object associated with this line.

```
get_pickradius()
```

return the pick radius used for containment tests

```
get_solid_capstyle()
```

Get the cap style for solid linestyles

```
get_solid_joinstyle()
```

Get the join style for solid linestyles

```
get_window_extent(renderer)
```

```
get_xdata(orig=True)
```

Return the xdata.

If *orig* is *True*, return the original data, else the processed data.

```
get_xydata()
```

Return the *xy* data as a Nx2 numpy array.

```
get_ydata(orig=True)
```

Return the ydata.

If *orig* is *True*, return the original data, else the processed data.

```
is_dashed()
```

return True if line is dashstyle

```
lineStyles = {":": '_draw_nothing', "'": '_draw_nothing', '-': '_draw_solid', '--':
'_draw_dashed', '-.': '_draw_dash_dot', '.': '_draw_dotted', 'None': '_draw_nothing'}
markers = {'.': 'point', ',': 'pixel', 'o': 'circle', 'v': 'triangle_down', '^': 'triangle_up', '<':
'triangle_left', '>': 'triangle_right', '1': 'tri_down', '2': 'tri_up', '3': 'tri_left', '4': 'tri_right',
'8': 'octagon', 's': 'square', 'p': 'pentagon', '*': 'star', 'h': 'hexagon1', 'H': 'hexagon2', '+':
'plus', 'x': 'x', 'D': 'diamond', 'd': 'thin_diamond', '|': 'vline', '_': 'hline', 'P': 'plus_filled',
'X': 'x_filled', 0: 'tickleft', 1: 'tickright', 2: 'tickup', 3: 'tickdown', 4: 'caretleft', 5:
'caretright', 6: 'caretup', 7: 'caredown', 8: 'caretleftbase', 9: 'caretrightbase', 10:
'caretupbase', 11: 'caredownbase', 'None': 'nothing', None: 'nothing', '': 'nothing', '':
'nothing'}
```

```
recache(always=False)
```

```
recache_always()
set_aa(val)
```

alias for `set_antialiased`

```
set_antialiased(b)
```

True if line should be drawn with antialiased rendering

ACCEPTS: [True | False]

```
set_c(val)
```

alias for `set_color`

```
set_color(color)
```

Set the color of the line

ACCEPTS: any matplotlib color

```
set_dash_capstyle(s)
```

Set the cap style for dashed linestyles

ACCEPTS: ['butt' | 'round' | 'projecting']

```
set_dash_joinstyle(s)
```

Set the join style for dashed linestyles ACCEPTS: ['miter' | 'round' | 'bevel']

```
set_dashes(seq)
```

Set the dash sequence, sequence of dashes with on off ink in points. If *seq* is empty or if *seq* = (None, None), the linestyle will be set to solid.

ACCEPTS: sequence of on/off ink in points

```
set_data(*args)
```

Set the x and y data

ACCEPTS: 2D array (rows are x, y) or two 1D arrays

```
set_drawstyle(drawstyle)
```

Set the drawstyle of the plot

‘default’ connects the points with lines. The steps variants produce step-plots. ‘steps’ is equivalent to ‘steps-pre’ and is maintained for backward-compatibility.

ACCEPTS: [‘default’ | ‘steps’ | ‘steps-pre’ | ‘steps-mid’ | ‘steps-post’]
`set_fillstyle(fs)`

Set the marker fill style; ‘full’ means fill the whole marker. ‘none’ means no filling; other options are for half-filled markers.

ACCEPTS: [‘full’ | ‘left’ | ‘right’ | ‘bottom’ | ‘top’ | ‘none’]

`set_linestyle(ls)`

Set the linestyle of the line (also accepts drawstyles, e.g., ‘steps--’)

linestyle	description
'-' or 'solid'	solid line
'--' or 'dashed'	dashed line
'-.' or 'dashdot'	dash-dotted line
':' or 'dotted'	dotted line
'None'	draw nothing
' '	draw nothing
' '	draw nothing

‘steps’ is equivalent to ‘steps-pre’ and is maintained for backward-compatibility.

Alternatively a dash tuple of the following form can be provided:

`(offset, onoffseq),`

where `onoffseq` is an even length tuple of on and off ink in points.

ACCEPTS: [‘solid’ | ‘dashed’, ‘dashdot’, ‘dotted’ | (offset, on-off-dash-seq) | ‘-’ | ‘--’ | ‘-.’ | ‘:’ | ‘None’ | ‘ ’ | ‘ ’]

See also

[`set\_drawstyle\(\)`](#)

To set the drawing style (stepping) of the plot.

ls : { '-', '--', '-.', ':' } and more see description

Parameters:

The line style.

`set_linewidth(w)`

Set the line width in points

ACCEPTS: float value in points

`set_ls(val)`

alias for `set_linestyle`

`set_lw(val)`

alias for `set_linewidth`

`set_marker(marker)`

Set the line marker

ACCEPTS: [A valid marker style](#)

marker: marker style

Parameters:

See [markers](#) for full description of possible argument

`set_markeredgecolor(ec)`

Set the marker edge color

ACCEPTS: any matplotlib color

`set_markeredgewidth(ew)`

Set the marker edge width in points

ACCEPTS: float value in points

`set_markerfacecolor(fc)`

Set the marker face color.

ACCEPTS: any matplotlib color

`set_markerfacecoloralt(fc)`

Set the alternate marker face color.

ACCEPTS: any matplotlib color

`set_markersize(sz)`

Set the marker size in points

ACCEPTS: float

`set_markevery(every)`

Set the markevery property to subsample the plot when using markers.

e.g., if `every=5`, every 5-th marker will be plotted.

ACCEPTS: [None | int | length-2 tuple of int | slice | list/array of int | float | length-2 tuple of float]

every: None | int | length-2 tuple of int | slice | list/array of int | float | length-2 tuple of float

Which markers to plot.

Parameters:

- `every=None`, every point will be plotted.
- `every=N`, every N-th marker will be plotted starting with marker 0.
- `every=(start, N)`, every N-th marker, starting at point start, will be plotted.
- `every=slice(start, end, N)`, every N-th marker, starting at point start, upto but not including point end, will be plotted.
- `every=[i, j, m, n]`, only markers at points i, j, m, and n will be plotted.
- `every=0.1`, (i.e. a float) then markers will be spaced at approximately equal distances along the line; the distance along the line between markers is determined by multiplying the display-coordinate distance of the axes bounding-box diagonal by the value of `every`.
- `every=(0.5, 0.1)` (i.e. a length-2 tuple of float), the same functionality as `every=0.1` is exhibited but the first marker will be 0.5 multiplied by the display-coordinate-diagonal-distance along the line.

Notes

Setting the `markevery` property will only show markers at actual data points. When using float arguments to set the `markevery` property on irregularly spaced data, the markers will likely not appear evenly spaced because the actual data points do not coincide with the theoretical spacing between markers.

When using a start offset to specify the first marker, the offset will be from the first data point which may be different from the first the visible data point if the plot is zoomed in.

If zooming in on a plot when using float arguments then the actual data points that have markers will change because the distance between markers is always determined from the display-coordinates axes-bounding-box-diagonal regardless of the actual axes data limits.

`set_mec(val)`

alias for `set_markeredgecolor`

`set_mew(val)`

alias for `set_markeredgewidth`

`set_mfc(val)`

alias for `set_markerfacecolor`

`set_mfcalt(val)`

alias for `set_markerfacecoloralt`

`set_ms(val)`

alias for `set_markersize`

`set_picker(p)`

Sets the event picker details for the line.

ACCEPTS: float distance in points or callable pick function `fn(artist, event)`

`set_pickradius(d)`

Set the pick radius used for containment tests.

d : float

Parameters:

Pick radius, in points.

`set_solid_capstyle(s)`

Set the cap style for solid linestyles

ACCEPTS: ['butt' | 'round' | 'projecting']

```
set_solid_joinstyle(s)
```

Set the join style for solid linestyles ACCEPTS: ['miter' | 'round' | 'bevel']

```
set_transform(t)
```

set the Transformation instance used by this artist

ACCEPTS: a [matplotlib.transforms.Transform](#) instance

```
set_xdata(x)
```

Set the data np.array for x

ACCEPTS: 1D array

```
set_ydata(y)
```

Set the data np.array for y

ACCEPTS: 1D array

```
update_from(other)
```

copy properties from other to self

```
validCap = ('butt', 'round', 'projecting')
validJoin = ('miter', 'round', 'bevel')
zorder = 2
```

Normally we will use very restricted ones from these properties in our normal graphs.

We will try some of them in real graphs

line-color.py

```
from matplotlib.pyplot import *

#figure scale
figure(figsize=(8,6), dpi=80)

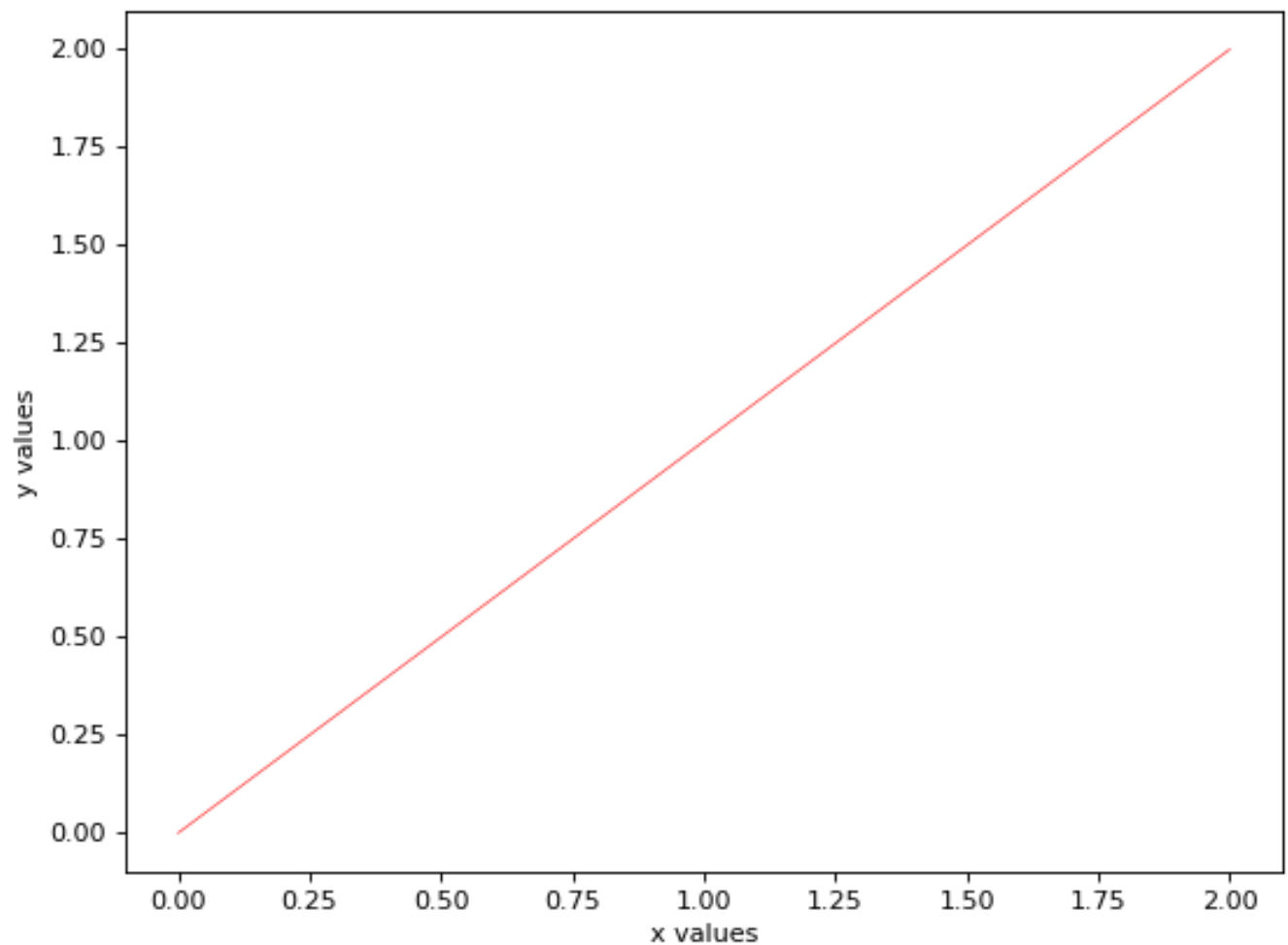
# Data
X = [0,1,2]
Y = [0,1,2]

# Axis labels
ylabel('y values')
xlabel('x values')

# plots
plot(X,Y)

# Save figure using 72 dots per inch
savefig("E:\Dynamic_Content\eclipse_ws_python\evolution\line-color.png", dpi=80)

# display
show()
```



Each line may be labeled

label-legend.py

```
from matplotlib.pyplot import *  
from numpy.core.function_base import linspace  
# Create a new figure of size 8x6 points, using 80 dots per inch  
figure(figsize=(8,6), dpi=80)
```

```
# Data

x= linspace(-3 , 3 , 1000 , endpoint=True)

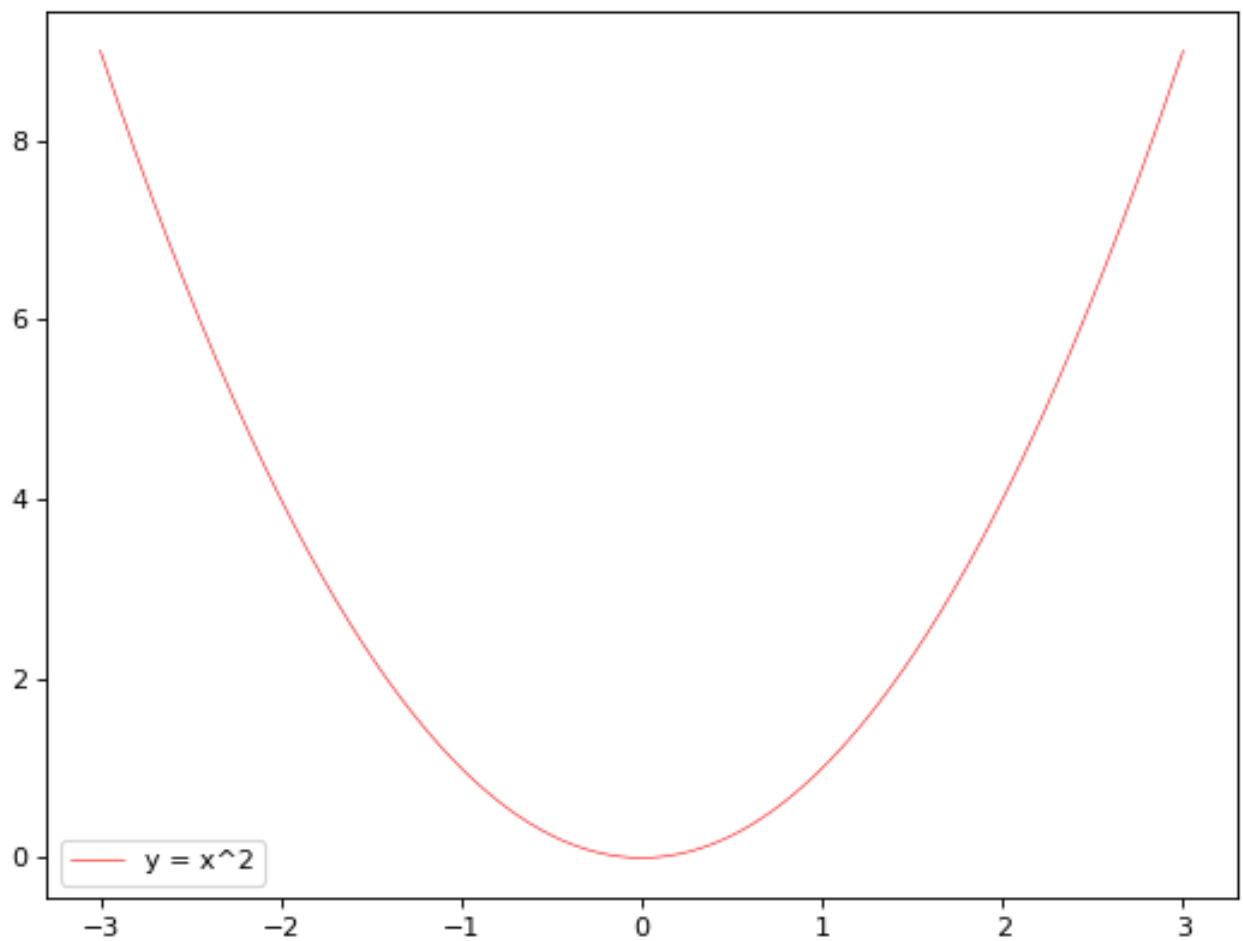
# Plot and Label

plot(x,x**2, color="red", linewidth=0.5 ,
      linestyle="-", label="y = x^2")
# Legend

legend()

# display

show()
```



Line labels may be written with latex.

label-legend-latex.py

```
from matplotlib.pyplot import *
from numpy.core.function_base import linspace

# Create a new figure of size 8x6 points, using 80 dots per inch
figure(figsize=(8,6), dpi=80)

# Data

#
x= linspace(-3 , 3 , 1000 , endpoint=True)

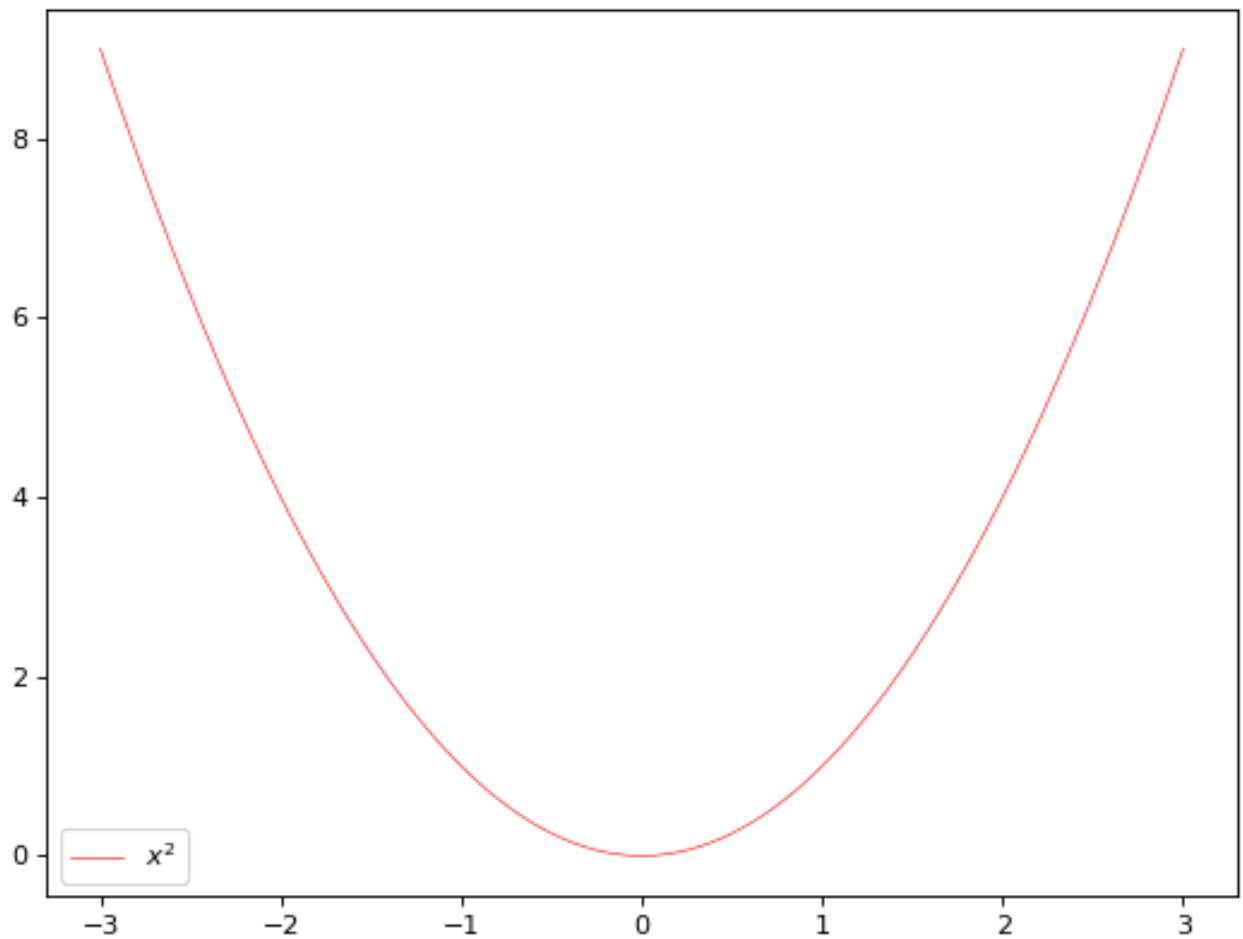
# Plot and Label

plot(x,x**2, color="red", linewidth=0.5 ,
      linestyle="-", label=" ${x^2}$ ")
# Legend

legend()

# display

show()
```



We can place the legend anywhere in the plot and if the will may it put in a box.

legend-placement.py

```
from matplotlib.pyplot import *
from numpy.core.function_base import linspace

# Create a new figure of size 8x6 points, using 80 dots per inch
figure(figsize=(8,6), dpi=80)
```

```
# Data

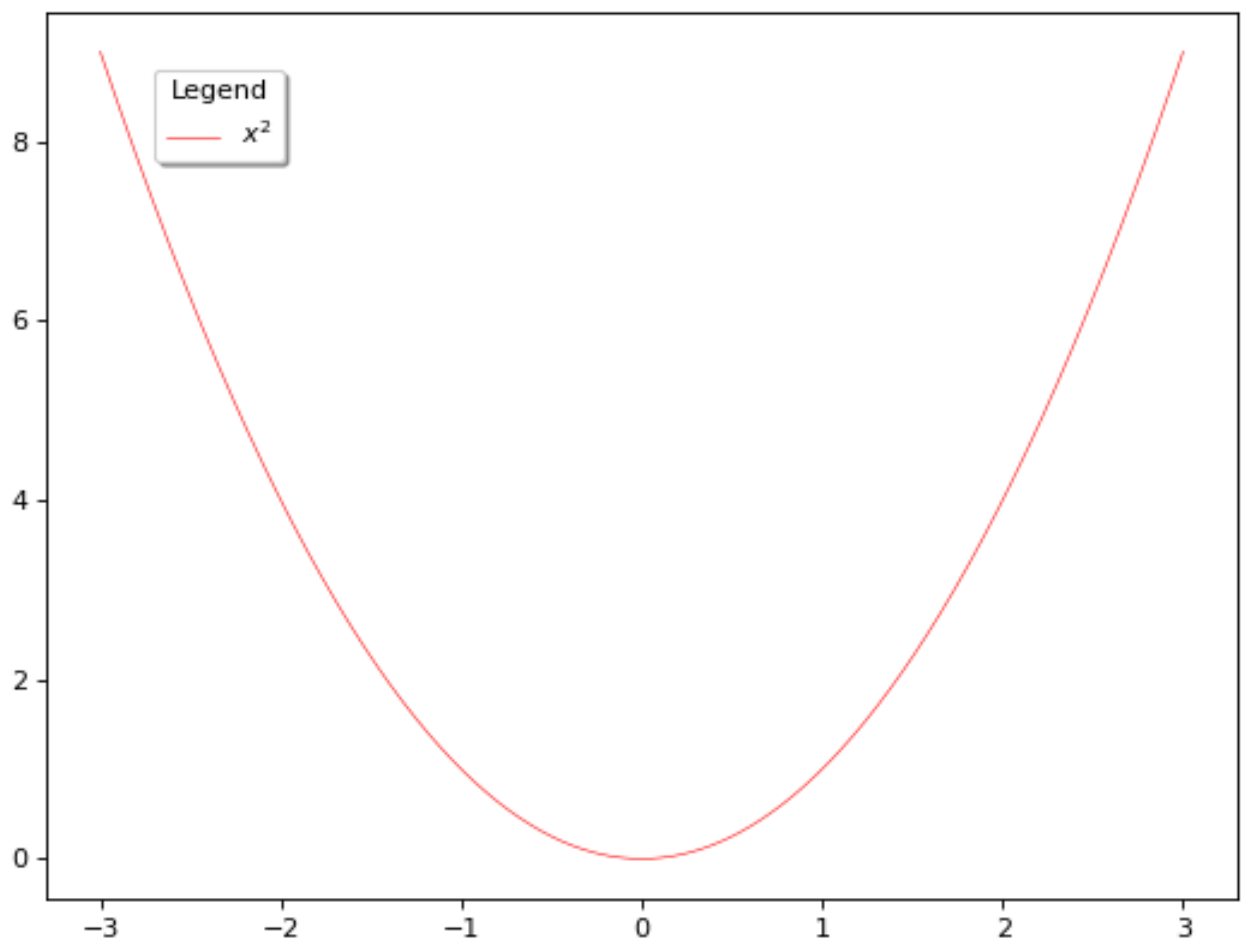
x= linspace(-3 , 3 , 1000 , endpoint=True)

# Plot and Label

plot(x,x**2, color="red", linewidth=0.5 ,
      linestyle="-", label="$x^2$")
# Legend

legend(loc="upper left" , title= "Legend")

show()
```



Note that the legend text is black

The program shown below, is the adoption of this unique demo,

with colored legends.

legend-demo-bde1.py

"""

Unique demo of the legend function with a few features.

Developed by [Bedri Doğan Emir](#)

In addition to the basic legend, this demo shows a few optional features:

- * Custom legend placement.
- * A keyword argument to a drop-shadow.
- * Setting the background color.
- * Setting the font size.
- * Setting the line width.

This demo requires of [customising](#) each plot separately

Use a new label definition line for each plot

"""

import numpy **as** np

from matplotlib.pyplot **import** *

Data

x = np.arange(0,3, .02)

y = np.exp(x)

Create plots with user defined labels.

fig, ax = subplots()

ax.plot(x, y, color='red', linestyle='-',
label="{y = e^x}")

ax.plot(x,x**2, color="blue", linestyle="--",
label="{y = x^2}")

ax.plot(x,x**3, color="green", linestyle="-.",
label="{y = x^3}")

Now add the legend with some customizations.

legend = ax.legend(loc='upper center', shadow=**True**)

The frame is matplotlib.patches,

rectangle instance surrounding the legend.

```

frame = legend.get_frame()
frame.set_facecolor('turquoise')
frame.set_edgecolor('blue')

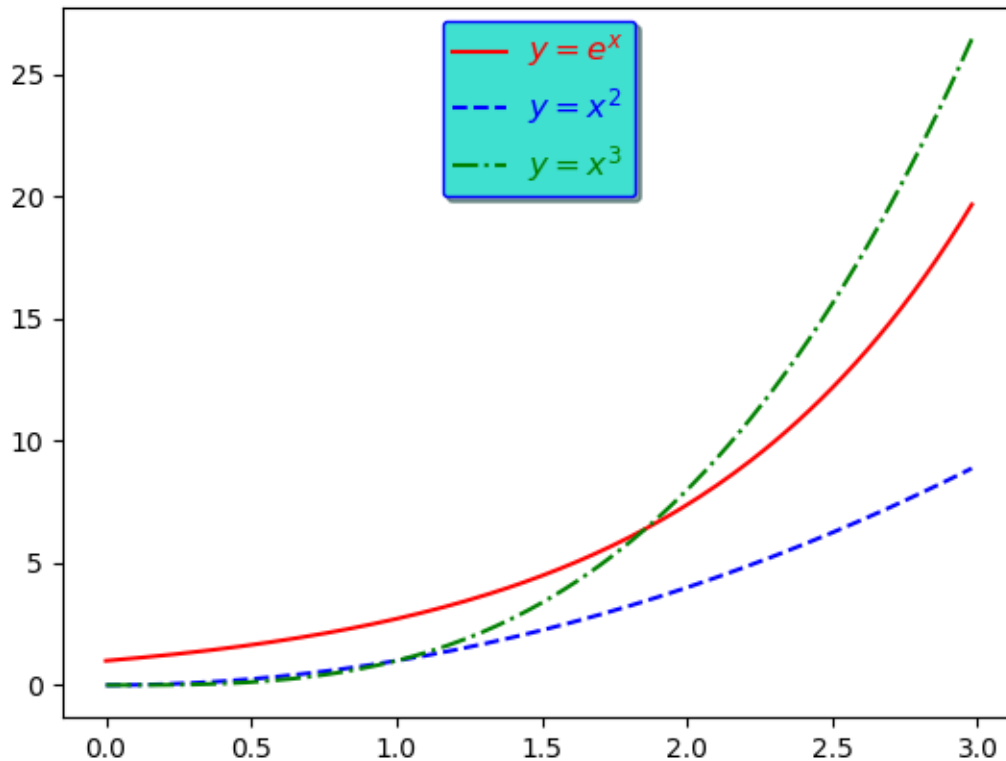
# Set the font size and font color
#Note that : It is very difficult to obtain
#colored legend texts.
#This demo is the arrangement of the unique demo
#found only in Matplotlib user guide legend demo
numline=0
for label in legend.get_texts():
    numline=numline+1
    if numline==1:
        label.set_fontsize(12)
        label.set_color('red')
    if numline==2:
        label.set_fontsize(12)
        label.set_color('blue')
    if numline==3:
        label.set_fontsize(12)
        label.set_color('green')

#Set the legend line width

for label in legend.get_lines():
    label.set_linewidth(1.5)

show()

```



legend-demo-bde2.py

```
# legend-demo-bde1.py
"""
```

Unique demo of the legend function with a few features.
Developed by [Bedri Doğan Emir](#)

In addition to the basic legend, this demo shows a few optional features:

- * Custom legend placement.
- * A keyword argument to a drop-shadow.
- * Setting the background color.
- * Setting the font size.
- * Setting the line width.
- * Colored legends.

This demo has automatic legend coloring feature for every plot
No need of user intervention for label coloring
"""

```
import numpy as np
from matplotlib.pyplot import *
```

```

# Data
x = np.arange(0,3, .02)

y = np.exp(x)

# Create plots with user defined labels.

fig, ax = subplots()

ax.plot(x, y, color = 'red' ,linestyle = '-',
        label="$y = e^x$")
ax.plot(x,x**2, color = "blue" , linestyle="--" ,
        label="$x^2$")
ax.plot(x,x**3, color = "green" , linestyle="-" ,
        label="$x^3$")
ax.plot(x,x**4, color = "maroon" , linestyle="-" ,
        label="$x^4$")
ax.plot(x,1/(x**6+1), color = "indianred" , linestyle="-" ,
        label=r"$\frac{1}{(x^6+1)}$")

# Now add the legend with some customizations.

legend = ax.legend(loc='upper center', shadow=True,
                  bbox_to_anchor=[0.15,0.95],
                  ncol=1, title="Legend",fancybox=True)

legend.get_title().set_color("red")

legend.get_title().set_fontsize(9)
# The frame is matplotlib.patches,
# rectangle instance surrounding the legend.

frame = legend.get_frame()
frame.set_facecolor('white')
frame.set_edgecolor('blue')

# The frame is matplotlib.patches,
# rectangle instance surrounding the legend.

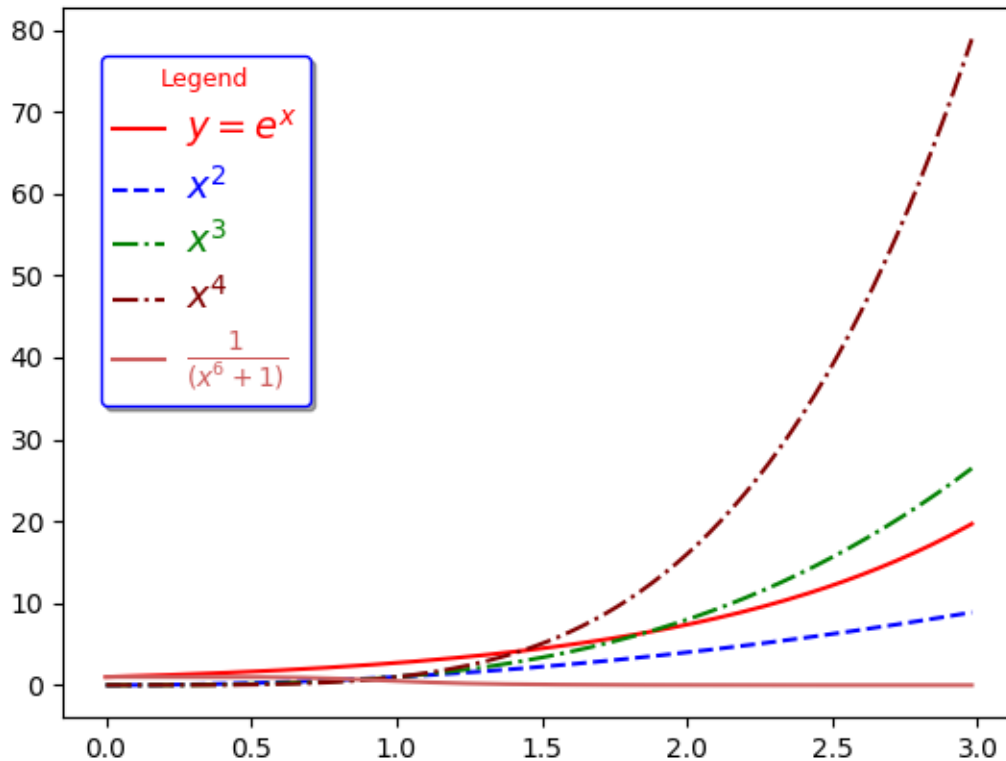
frame = legend.get_frame()
frame.set_facecolor('white')
frame.set_edgecolor('blue')

# Set the font size and font color
#Note that : It is very difficult to obtain
#colored legend texts.
#This demo is the arrangement of the unique demo
#found only in Matplotlib user guide legend demo

numline=0

```

```
colors = ("black", "red" , "blue" , "green" , "maroon" ,  
          "indianred" , "darkturquoise" )  
  
for label in legend.get_texts():  
    label.set_fontsize(14)  
    label.set_alpha(1)  
    numline = numline+1  
    label.set_color(colors[numline])  
    if numline >= 6:  
        numline = 0  
  
# Set the legend line width  
  
for label in legend.get_lines():  
    label.set_linewidth(1.5)  
show()
```



Note: The coordinates of the legend placement is as follows:

By default, the point {0,0} is the lower left position and
 # the point {1,1} is the upper right position of the figure.
 # Then, the point {0.15,0.95} is somewhere at the upper left
 # and the left edge of the legend box is aligned at x= 0.15

'''

It is possible to insert text and annotate the interesting points

of the curve.

'''

```
class matplotlib.text.Annotation(s, xy, xytext=None, xycoords='data', textcoords=None,
arrowprops=None, annotation_clip=None, **kwargs)
```

Bases: [matplotlib.text.Text](#), `matplotlib.text._AnnotationBase`

Annotate the point `xy` with text `s`.

Additional kwargs are passed to [Text](#).

s : str

The text of the annotation

xy : iterable

Length 2 sequence specifying the (x,y) point to annotate

xytext : iterable, optional

Length 2 sequence specifying the (x,y) to place the text at. If None, defaults to `xy`.

xycoords : str, Artist, Transform, callable or tuple, optional

The coordinate system that `xy` is given in.

Parameters: For a [str](#) the allowed values are:

Property	Description
'figure points'	points from the lower left of the figure
'figure pixels'	pixels from the lower left of the figure
'figure fraction'	fraction of figure from lower left
'axes points'	points from lower left corner of axes
'axes pixels'	pixels from lower left corner of axes
'axes fraction'	fraction of axes from lower left
'data'	use the coordinate system of the object being annotated (default)
'polar'	(θ, r) if not native 'data' coordinates

If a [Artist](#) object is passed in the units are fraction if it's bounding box.

If a [Transform](#) object is passed in use that to transform xy to screen coordinates

If a callable it must take a [RendererBase](#) object as input and return a [Transform](#) or [Bbox](#) object

If a [tuple](#) must be length 2 tuple of str, Artist, Transform or callable objects. The first transform is used for the x coordinate and the second for y .

See [Advanced Annotation](#) for more details.

Defaults to 'data'

textcoords : str, Artist, Transform, callable or tuple, optional

The coordinate system that $xytext$ is given, which may be different than the coordinate system used for xy .

All $xycoords$ values are valid as well as the following strings:

Property	Description
'offset points'	offset (in points) from the xy value
'offset pixels'	offset (in pixels) from the xy value

defaults to the input of $xycoords$

arrowprops : dict, optional

If not None, properties used to draw a [FancyArrowPatch](#) arrow between xy and $xytext$.

If **arrowprops** does not contain the key 'arrowstyle' the allowed keys are:

Key	Description
width	the width of the arrow in points
headwidth	the width of the base of the arrow head in points
headlength	the length of the arrow head in points
shrink	fraction of total length to 'shrink' from both ends
?	any key to matplotlib.patches.FancyArrowPatch

If the `arrowprops` contains the key `'arrowstyle'` the above keys are forbidden.
The allowed values of `'arrowstyle'` are:

Name	Attrs
'-'	None
'->'	head_length=0.4,head_width=0.2
'-['	widthB=1.0,lengthB=0.2,angleB=None
' - '	widthA=1.0,widthB=1.0
'-> '	head_length=0.4,head_width=0.2
'<-'	head_length=0.4,head_width=0.2
'<->'	head_length=0.4,head_width=0.2
'< -'	head_length=0.4,head_width=0.2
'< -> '	head_length=0.4,head_width=0.2
'fancy'	head_length=0.4,head_width=0.4,tail_width=0.4
'simple'	head_length=0.5,head_width=0.5,tail_width=0.2
'wedge'	tail_width=0.3,shrink_factor=0.5

Valid keys for [FancyArrowPatch](#) are:

Key	Description
<code>arrowstyle</code>	the arrow style
<code>connectionstyle</code>	the connection style
<code>relpos</code>	default is (0.5, 0.5)
<code>patchA</code>	default is bounding box of the text
<code>patchB</code>	default is None
<code>shrinkA</code>	default is 2 points
<code>shrinkB</code>	default is 2 points
<code>mutation_scale</code>	default is text size (in points)
<code>mutation_aspect</code>	default is 1.
?	any key for matplotlib.patches.PathPatch

Defaults to None

annotation_clip : bool, optional

Controls the visibility of the annotation when it goes outside the axes area.

If [True](#), the annotation will only be drawn when the `xy` is inside the axes. If [False](#), the annotation will always be drawn regardless of its position.

The default is [None](#), which behave as [True](#) only if `xycoords` is “data”.

annotate-bde1.py

In all places which accept text we can also use Latex
 # Use of Latex is encouraged for obtaining good looking and presentation
 # capable of documents.
 # In Matplotlib, Latex facility is supplied as default. For simple Letex
 # usage no previously setting of machine-wide Latex infrastructure is necessary.
 # It is only necessary when the requirements beyond existing abilites
 # would be necessary

```
from numpy.core.function_base import linspace
from numpy import exp
from matplotlib.pyplot import *
from IPython.core.pylabtools import figsize
from scipy.interpolate.interpolate import _dot0

# Data
x = linspace(0,3, 1000,endpoint=True)

y = exp(x)

# Create plots with user defined labels.

fig, ax = subplots(figsize=(8,6),dpi=90)

ax.plot(x, y, color = 'red' ,linestyle = '-',
        label="${y = e^x}$")
ax.plot(x,x**2, color = "blue" , linestyle="--" ,
        label="${x^2}$" )

# Now add the legend with some customizations.

legend = ax.legend(loc='upper center', shadow=True,
                  bbox_to_anchor=[0.15,0.95],
                  ncol=1, title="Legend",fancybox=True)
legend.get_title().set_color("red")
legend.get_title().set_fontsize(9)
```

```

# The frame is matplotlib.patches,
# rectangle instance surrounding the legend.

frame = legend.get_frame()
frame.set_facecolor('white')
frame.set_edgecolor('blue')

# Set the font size and font color
#Note that : It is very difficult to obtain
#colored legend texts.
#This demo is the arrangement of the unique demo
#found only in Matplotlib user guide legend demo
numline=0

colors = ("black", "red" , "blue" , "green" , "maroon" ,
          "indianred" , "darkturquoise" )

for label in legend.get_texts():

    label.set_fontsize(9)

    label.set_alpha(1)

    numline = numline+1

    label.set_color(colors[numline])

    if numline >= 6:

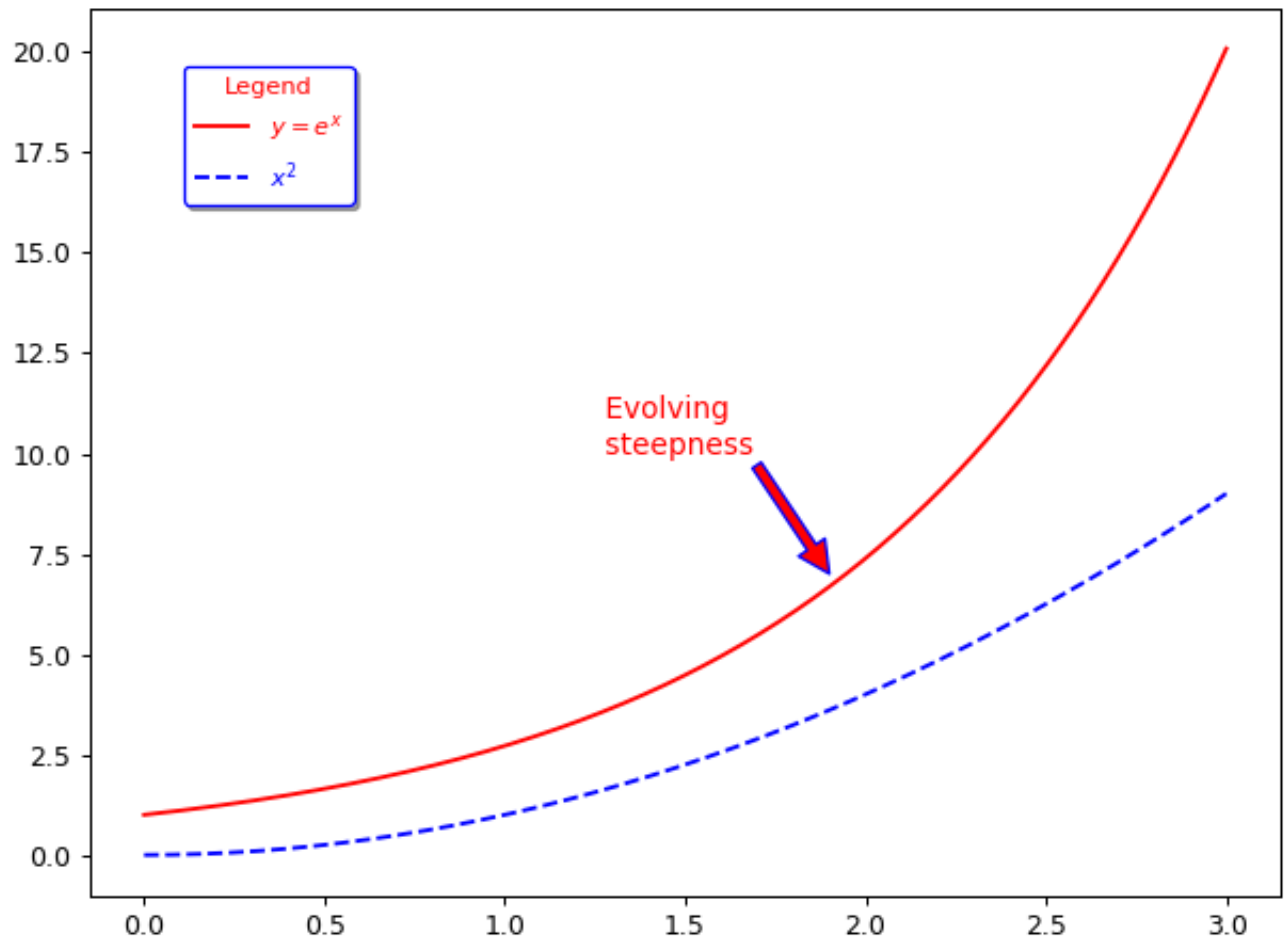
        numline = 0

#Set the legend line width

for label in legend.get_lines():
    label.set_linewidth(1.5)

# Simple annotation
ax.annotate(' Evolving\n steepness', color='red', xy=(1.90, 7), xycoords = "data",
            xytext=(1.25,10), textcoords='data',
            arrowprops=dict(facecolor='red', edgecolor='blue', shrink=2),
            fontsize=11)

```



Note :

Either xytext and the textcoords = 'data', that means that the coordinates of xy(1.90 , 7)

and xytext (1.25 , 10) represent the actual coordinates which are used to position major

ticks of the curve.

Arrow is automatically placed to fill the gap between the coordinates

of the target (xytest) and the placement of the text (textcoords).

annotate-bde2.py

```
from numpy.core.function_base import linspace
from numpy import exp
from matplotlib.pyplot import *
from IPython.core.pylabtools import figsize
from scipy.interpolate.interpolate import _dot0

# Data
x = linspace(0,3, 1000,endpoint=True)

y = exp(x)

# Data
x = np.arange(0,3, .02)

y = np.exp(x)

# Create plots with user defined labels.

fig, ax = subplots()

ax.plot(x, y, color='red', linestyle = '-',
        label="$y = e^x$")
ax.plot(x,x**2, color = "blue", linestyle="--",
        label="$x^2$")

# Now add the legend with some customizations.

legend = ax.legend(loc='upper center', shadow=True,
                  bbox_to_anchor=[0.15,0.95],
                  ncol=1, title="Legend", fancybox=True)
legend.get_title().set_color("red")
legend.get_title().set_fontsize(9)
# The frame is matplotlib.patches,
# rectangle instance surrounding the legend.

frame = legend.get_frame()
frame.set_facecolor('white')
frame.set_edgecolor('blue')

# Set the font size and font color
#Note that : It is very difficult to obtain
#colored legend texts.
#This demo is the arrangement of the unique demo
#found only in Matplotlib user guide legend demo
numline=0
```

```

for label in legend.get_texts():
    numline=numline+1
    if numline==1:
        label.set_fontsize(9)
        label.set_color('red')
    if numline==2:
        label.set_fontsize(9)
        label.set_color('blue')

#Set the legend line width

for label in legend.get_lines():
    label.set_linewidth(1.5)

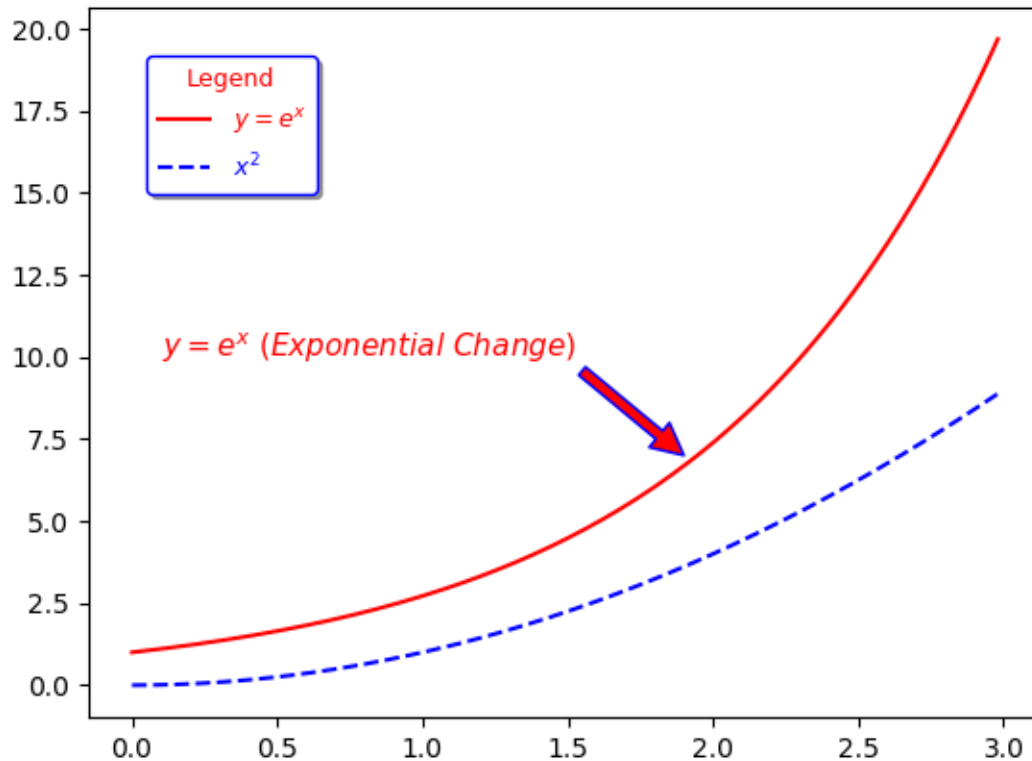
# Simple annotation
ax.annotate('${y = e^x \ (Exponential \ Change)}$',
            color='red', xy=(1.90, 7), xycoords = "data",
            xytext=(0.1,10), textcoords='data',
            arrowprops=dict(facecolor='red', edgecolor='blue',shrink=2),
            fontsize=11)

# Display graph

show()

show()

```



We can decorate axes by defining axes labels and axes ticks on our own needs.

Tick labels may be defined using Latex

axes-bde-1.py

```
import numpy as np
from matplotlib.pyplot import *

# Data
x = np.arange(0,3.02, .02)

y = np.exp(x)

# Create plots with user defined labels.

#figure size and dpi

fig,ax = subplots()
```

```

#figure size and dpi

ax.figure = (8,6);
ax.dpi=80
ax.plot(x, y, color='red', linestyle = '-',
        label="$y = e^x$")
ax.plot(x,x**2, color = "blue", linestyle="--",
        label="$x^2$")

# Now add the legend with some customizations.

legend = ax.legend(loc='upper center', shadow=True,
                  bbox_to_anchor=[0.15,0.95],
                  ncol=1, title="Legend", fancybox=True)
legend.get_title().set_color("red")
legend.get_title().set_fontsize(9)
# The frame is matplotlib.patches,
# rectangle instance surrounding the legend.

frame = legend.get_frame()
frame.set_facecolor('white')
frame.set_edgecolor('blue')

numline=0

colors = ("black", "red", "blue", "green", "maroon",
          "indianred", "darkturquoise")

for label in legend.get_texts():

    label.set_fontsize(9)

    label.set_alpha(1)

    numline = numline+1

    label.set_color(colors[numline])

    if numline >= 6:

        numline = 0

#Set the legend line width

for label in legend.get_lines():
    label.set_linewidth(1.5)

# Simple annotation

ax.annotate('$y = e^x \ (Exponential \ Change)$',

```

```

        color='red', xy=(1.90, 7), xycoords = "data",
        xytext=(0.1,10), textcoords='data',
        arrowprops=dict(facecolor='red', edgecolor='blue',shrink=2),
        fontsize=11)

# Axes limits set after the current data points

if x.min() == 0 :
    xmin = 0.1
else :
    xmin = x.min()

if y.min() == 0 :
    ymin = 0.1
else :
    ymin = y.min()

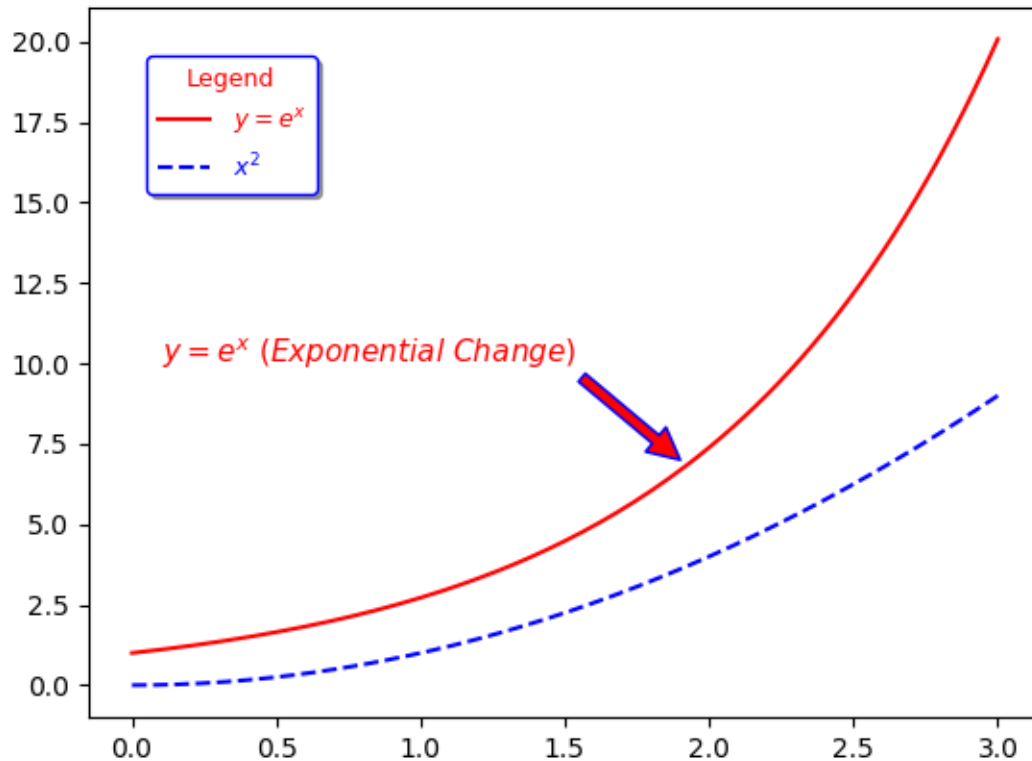
if x.max() == 0 :
    xmax = 0.1
else :
    xmax = x.max()

if y.max() == 0 :
    ymax = 0.1
else :
    ymax = y.max()

xlim=(xmin*1.1,xmax*1.1)
ylim=(ymin*1.1,ymax*1.1)

show()

```



Tick labels may be defined using Latex and may be colored

axes-bde-2.py

```
import numpy as np
from matplotlib.pyplot import *

# Data
x = np.arange(0,3.02, .02)

y = np.exp(x)

# Create plots with user defined labels.

#figure size and dpi
fig,ax = subplots()

#figure size and dpi
ax.figure = (8,6);
```

```

ax.dpi=80
ax.plot(x, y, color='red', linestyle = '-',
        label="$y = e^x$")
ax.plot(x,x**2, color = "blue", linestyle="--",
        label="$x^2$")

# Now add the legend with some customizations.

legend = ax.legend(loc='upper center', shadow=True,
                  bbox_to_anchor=[0.15,0.95],
                  ncol=1, title="Legend", fancybox=True)
legend.get_title().set_color("red")
legend.get_title().set_fontsize(9)

# The frame is matplotlib.patches,
# rectangle instance surrounding the legend.

frame = legend.get_frame()
frame.set_facecolor('white')
frame.set_edgecolor('blue')

numline=0

colors = ("black", "red", "blue", "green", "maroon",
          "indianred", "darkturquoise")

for label in legend.get_texts():

    label.set_fontsize(9)

    label.set_alpha(1)

    numline = numline+1

    label.set_color(colors[numline])

    if numline >= 6:

        numline = 0

#Set the legend line width

for label in legend.get_lines():
    label.set_linewidth(1.5)

# Simple annotation
ax.annotate('$y = e^x \ (Exponential \ Change)$',
            color='red', xy=(1.90, 7), xycoords = "data",
            xytext=(0.1,10), textcoords='data',
            arrowprops=dict(facecolor='red', edgecolor='blue', shrink=2),

```

```

        fontsize=11)

# Axes limits set after the current data points

if x.min() == 0 :
    xmin = 0.1
else :
    xmin = x.min()

if y.min() == 0 :
    ymin = 0.1
else :
    ymin = y.min()

if x.max() == 0 :
    xmax = 0.1
else :
    xmax = x.max()

if y.max() == 0 :
    ymax = 0.1
else :
    ymax = y.max()

# It is also possible to set our own limits

xlim=(xmin*1.1,xmax*1.1)
ylim=(ymin*1.1,ymax*1.1)

# We can set our own tick points as given below

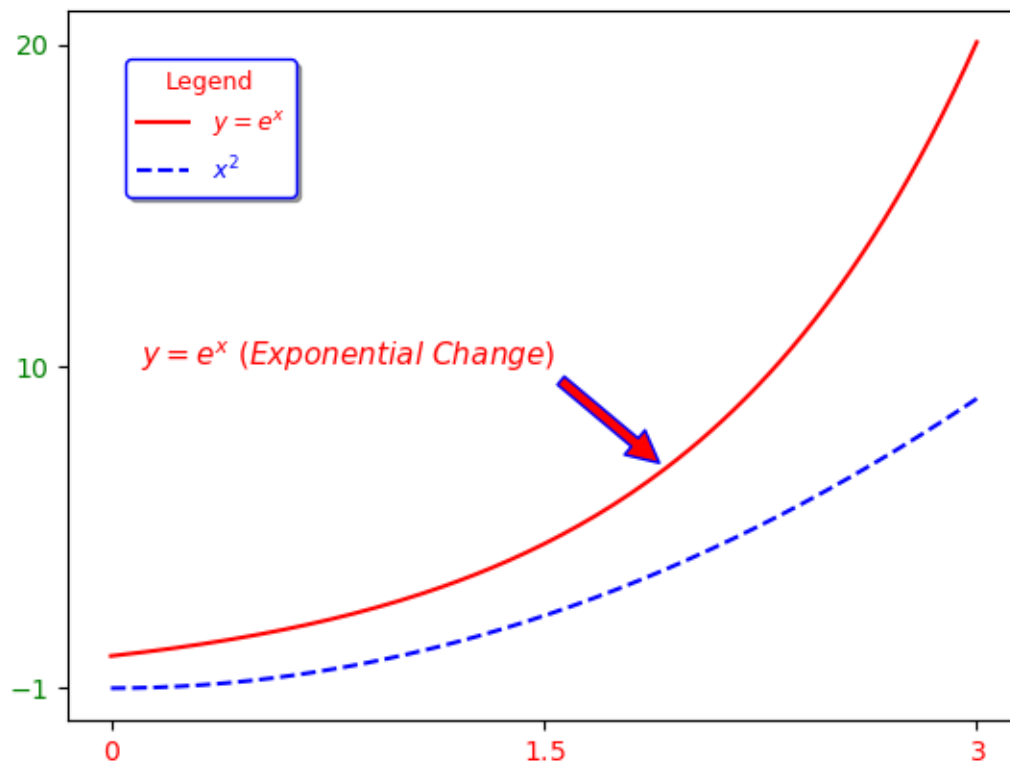
# User defined major ticks using Latex

xticks([0 , 1.5 , 3],
        ['$0$' , '$1.5$' , '$3$'], color='red')
yticks([0, 10 , 20],
        ['$-1$' , '$10$' , '$20$'], color='green')

#Display graph

show()

```



Changing spine positions

spines-bde-1.py

```
import numpy as np
from matplotlib.pyplot import *

# Data
x = np.arange(0,3.02, .02)

y = np.exp(x)

# Create plots with user defined labels.

#figure size and dpi
fig,ax = subplots()

#figure size and dpi
ax.figure = (8,6);
```

```

ax.dpi=80
ax.plot(x, y, color='red', linestyle = '-',
        label="$y = e^x$")
ax.plot(x,x**2, color = "blue", linestyle="--",
        label="$x^2$")

# Now add the legend with some customizations.

legend = ax.legend(loc='upper center', shadow=True,
                  bbox_to_anchor=[0.15,0.95],
                  ncol=1, title="Legend", fancybox=True)
legend.get_title().set_color("red")
legend.get_title().set_fontsize(9)

# The frame is matplotlib.patches,
# rectangle instance surrounding the legend.

frame = legend.get_frame()
frame.set_facecolor('white')
frame.set_edgecolor('blue')

numline=0

colors = ("black", "red", "blue", "green", "maroon",
          "indianred", "darkturquoise")

for label in legend.get_texts():

    label.set_fontsize(9)

    label.set_alpha(1)

    numline = numline+1

    label.set_color(colors[numline])

    if numline >= 6:

        numline = 0

#Set the legend line width

for label in legend.get_lines():
    label.set_linewidth(1.5)

# Simple annotation
ax.annotate('$y = e^x \ (Exponential \ Change)$',
           color='red', xy=(1.90, 7), xycoords = "data",
           xytext=(0.1,10), textcoords='data',
           arrowprops=dict(facecolor='red', edgecolor='blue', shrink=2),

```

```

        fontsize=11)

# Axes limits set after the current data points

if x.min() == 0 :
    xmin = 0.1
else :
    xmin = x.min()

if y.min() == 0 :
    ymin = 0.1
else :
    ymin = y.min()

if x.max() == 0 :
    xmax = 0.1
else :
    xmax = x.max()

if y.max() == 0 :
    ymax = 0.1
else :
    ymax = y.max()

# It is also possible to set our own limits

xlim=(xmin*1.1,xmax*1.1)
ylim=(ymin*1.1,ymax*1.1)

# We can set our own tick points as given below

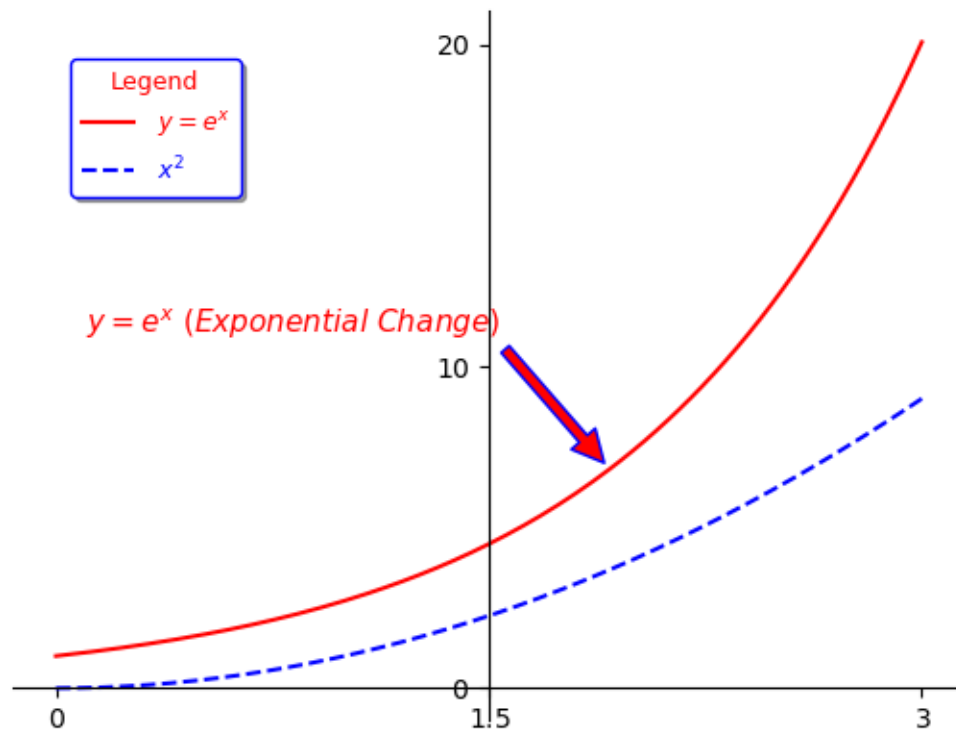
# User defined major ticks using Latex

xticks([0 , 1.5 , 3],
        ['$0$' , '$1.5$' , '$3$'], color='red')
yticks([0, 10 , 20],
        ['$0$' , '$10$' , '$20$'], color='green')

ax.spines['right'].set_color('none')
ax.spines['top'].set_color('none')
ax.xaxis.set_ticks_position('bottom')
ax.spines['bottom'].set_position(('data',0))
ax.yaxis.set_ticks_position('left')
ax.spines['left'].set_position(('data',1.5))
#Display graph

show()

```



Another Example from Matplotlib

sincos.py

```
import numpy as np
import matplotlib.pyplot as plt

#http://matplotlib.sourceforge.net/examples/pylab_examples/spine_placement_demo.html
#http://mple.m-artwork.eu/home/posts/sinsinesinus
fig = plt.figure(figsize=(4, 5))

ax = fig.add_subplot(111)

x = np.arange(-2*np.pi, 2*np.pi, 0.01)

y = np.sin(x)
```

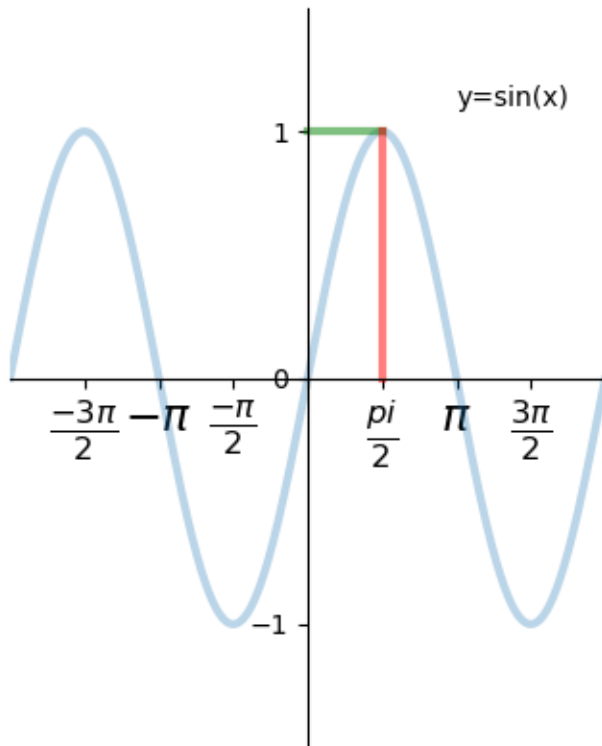
```

ax.spines['top'].set_color('none')
ax.spines['right'].set_color('none')
ax.xaxis.set_ticks_position('bottom')
ax.yaxis.set_ticks_position('left')
ax.plot(x, y, linewidth="3", alpha=0.3)
ax.plot([0, np.pi/2], [1, 1], ls="--", color="green", linewidth="3", alpha=0.5)
ax.plot([np.pi/2, np.pi/2], [1, 0], ls="--", color="red", linewidth="3", alpha=0.5)
ax.spines['left'].set_position('center')
ax.spines['bottom'].set_position('center')

ax.set_xlim(-2*np.pi, 2*np.pi)
xticker = np.arange(-np.pi-np.pi/2, np.pi+np.pi, np.pi/2)

xlabels = [r"$\frac{-3\pi}{2}$", r"$-\pi$", r"$\frac{-\pi}{2}$", "", r"$\frac{\pi}{2}$", r"$\pi$", r"$\frac{3\pi}{2}$"]
ax.set_xticks(xticker)
ax.set_xticklabels(xlabels, size=17)
ax.text(np.pi, 1.1, "y=sin(x)")
ax.set_ylim(-1.5, 1.5)
yticker = np.arange(-1, 2, 1)
ax.set_yticks(yticker)
plt.show()

```



Let's add the title

title-bde-1.py

```
import numpy as np
from matplotlib.pyplot import *

# Data
x = np.arange(0,3.02, .02)

y = np.exp(x)

# Create plots with user defined labels.

#figure size and dpi
fig,ax = subplots()

#figure size and dpi
ax.figure = (8,10);
ax.dpi=80
```

```

title("Exponential \n and \nSquared Change", color='red')
ax.plot(x, y, color = 'red' ,linestyle = '-',
        label="$y = e^x$")
ax.plot(x,x**2, color = "blue" , linestyle="--" ,
        label="$x^2$")

# Now add the legend with some customizations.

legend = ax.legend(loc='upper center', shadow=True,
                  bbox_to_anchor=[0.15,0.95],
                  ncol=1, title="Legend", fancybox=True)
legend.get_title().set_color("red")
legend.get_title().set_fontsize(9)

# The frame is matplotlib.patches,
# rectangle instance surrounding the legend.

frame = legend.get_frame()
frame.set_facecolor('white')
frame.set_edgecolor('blue')

numline=0

colors = ("black", "red" , "blue" , "green" , "maroon" ,
          "indianred" , "darkturquoise" )

for label in legend.get_texts():

    label.set_fontsize(9)

    label.set_alpha(1)

    numline = numline+1

    label.set_color(colors[numline])

    if numline >= 6:

        numline = 0

#Set the legend line width

for label in legend.get_lines():
    label.set_linewidth(1.5)

# Simple annotation
ax.annotate('$y = e^x \ (Exponential \ Change)$',
           color='red', xy=(1.90, 7), xycoords = "data",
           xytext=(0.1,10), textcoords='data',
           arrowprops=dict(facecolor='red', edgecolor='blue',shrink=2),

```

```

        fontsize=11)

# Axes limits set after the current data points

if x.min() == 0 :
    xmin = 0.1
else :
    xmin = x.min()

if y.min() == 0 :
    ymin = 0.1
else :
    ymin = y.min()

if x.max() == 0 :
    xmax = 0.1
else :
    xmax = x.max()

if y.max() == 0 :
    ymax = 0.1
else :
    ymax = y.max()

# It is also possible to set our own limits

xlim=(xmin*1.1,xmax*1.1)
ylim=(ymin*1.1,ymax*1.1)

# We can set our own tick points as given below

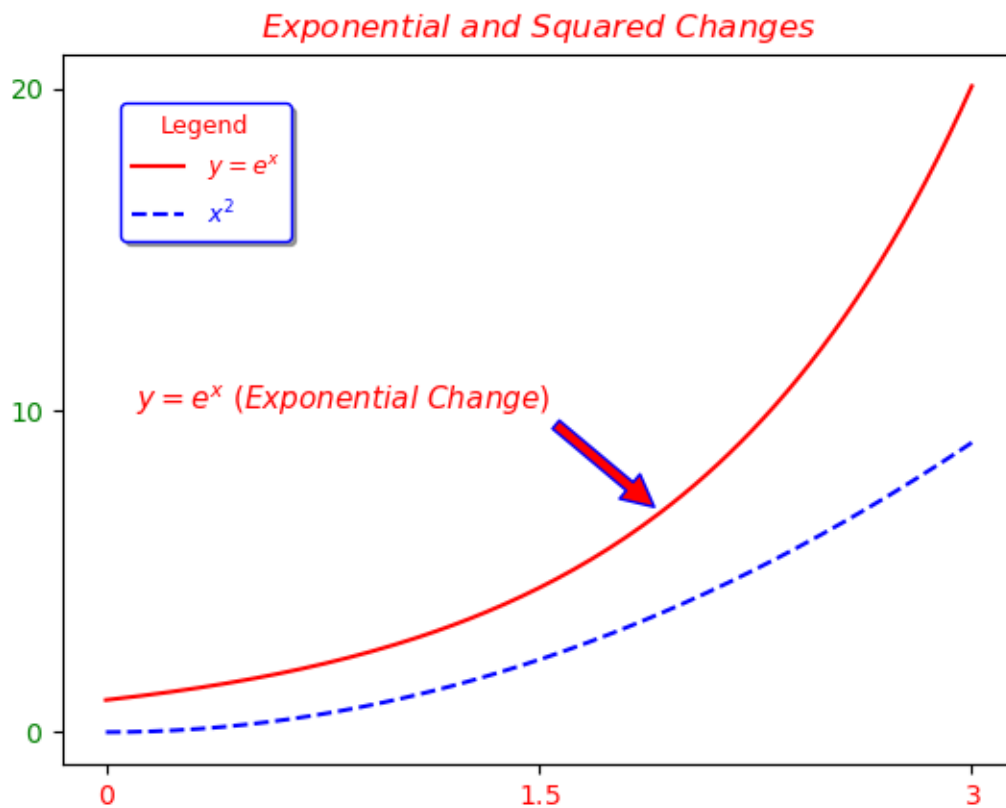
# User defined major ticks using Latex

xticks([0 , 1.5 , 3],
        ['$0$' , '$1.5$' , '$3$'], color='red')
yticks([0, 10 , 20],
        ['$0$' , '$10$' , '$20$'], color='green')

#Display graph

show()

```



Titles may be defined with Latex

title-bde-2.py

```
import numpy as np
from matplotlib.pyplot import *

# Data
x = np.arange(0,3.02, .02)

y = np.exp(x)

# Create plots with user defined labels.

#figure size and dpi
fig,ax = subplots()

#figure size and dpi
ax.figure = (8,10);
```

```

ax.dpi=80

title(r"$Exponential \ and \ Squared \ Changes$", color='red')
ax.plot(x, y, color = 'red' ,linestyle = '-',
        label="$y = e^x$")
ax.plot(x,x**2, color = "blue" , linestyle="--" ,
        label="$x^2$")

# Now add the legend with some customizations.

legend = ax.legend(loc='upper center', shadow=True,
                  bbox_to_anchor=[0.15,0.95],
                  ncol=1, title="Legend", fancybox=True)
legend.get_title().set_color("red")
legend.get_title().set_fontsize(9)

# The frame is matplotlib.patches,
# rectangle instance surrounding the legend.

frame = legend.get_frame()
frame.set_facecolor('white')
frame.set_edgecolor('blue')

numline=0

colors = ("black", "red" , "blue" , "green" , "maroon" ,
          "indianred" , "darkturquoise" )

for label in legend.get_texts():

    label.set_fontsize(9)

    label.set_alpha(1)

    numline = numline+1

    label.set_color(colors[numline])

    if numline >= 6:

        numline = 0

#Set the legend line width

for label in legend.get_lines():
    label.set_linewidth(1.5)

# Simple annotation
ax.annotate('$y = e^x \ (Exponential \ Change)$',
           color='red', xy=(1.90, 7), xycoords = "data",
           xytext=(0.1,10), textcoords='data',

```

```

        arrowprops=dict(facecolor='red', edgecolor='blue',shrink=2),
        fontsize=11)

# Axes limits set after the current data points

if x.min() == 0 :
    xmin = 0.1
else :
    xmin = x.min()

if y.min() == 0 :
    ymin = 0.1
else :
    ymin = y.min()

if x.max() == 0 :
    xmax = 0.1
else :
    xmax = x.max()

if y.max() == 0 :
    ymax = 0.1
else :
    ymax = y.max()

# It is also possible to set our own limits

xlim=(xmin*1.1,xmax*1.1)
ylim=(ymin*1.1,ymax*1.1)

# We can set our own tick points as given below

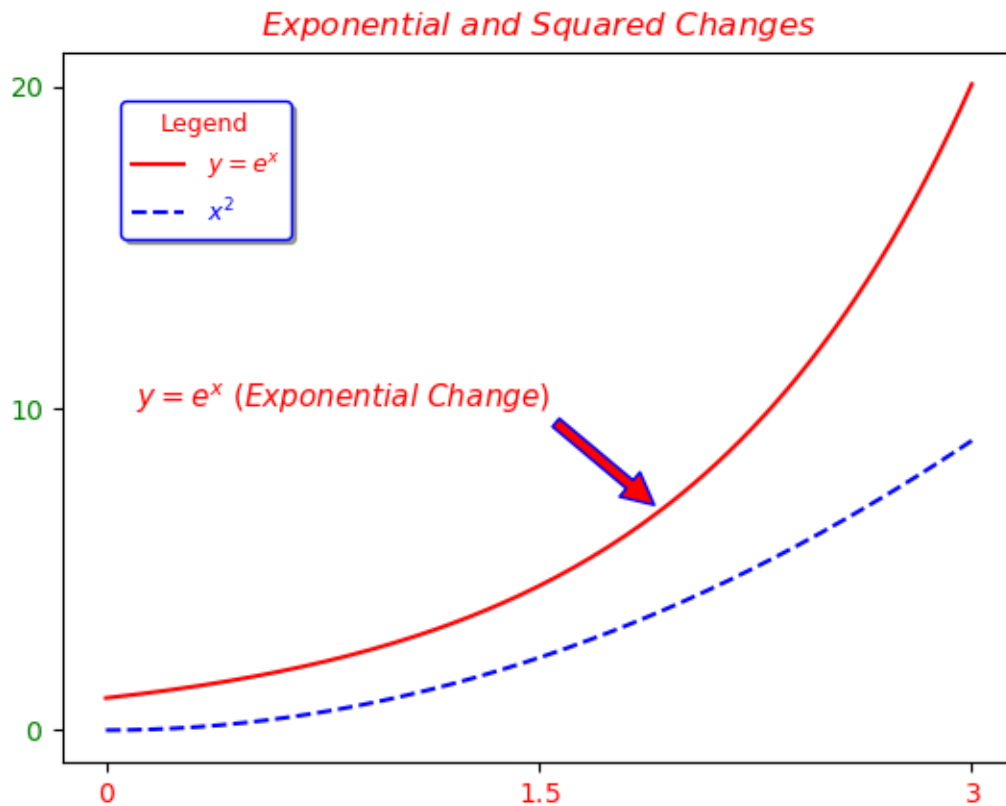
# User defined major ticks using Latex

xticks([0 , 1.5 , 3],
        ['$0$' , '$1.5$' , '$3$'], color='red')
yticks([0, 10 , 20],
        ['$0$' , '$10$' , '$20$'], color='green')

#Display graph

show()

```



About axes labels

Axes may be labelled by text or Latex fashion.

The font dictionary miracle

#

https://matplotlib.org/gallery/text_labels_and_annotations/text_fontdict.html#sphx-glr-gallery-text-labels-and-annotations-text-fontdict-py

fontdict.py

```
import numpy as np
import matplotlib.pyplot as plt
```

```
font = {'family': 'serif',
```

```

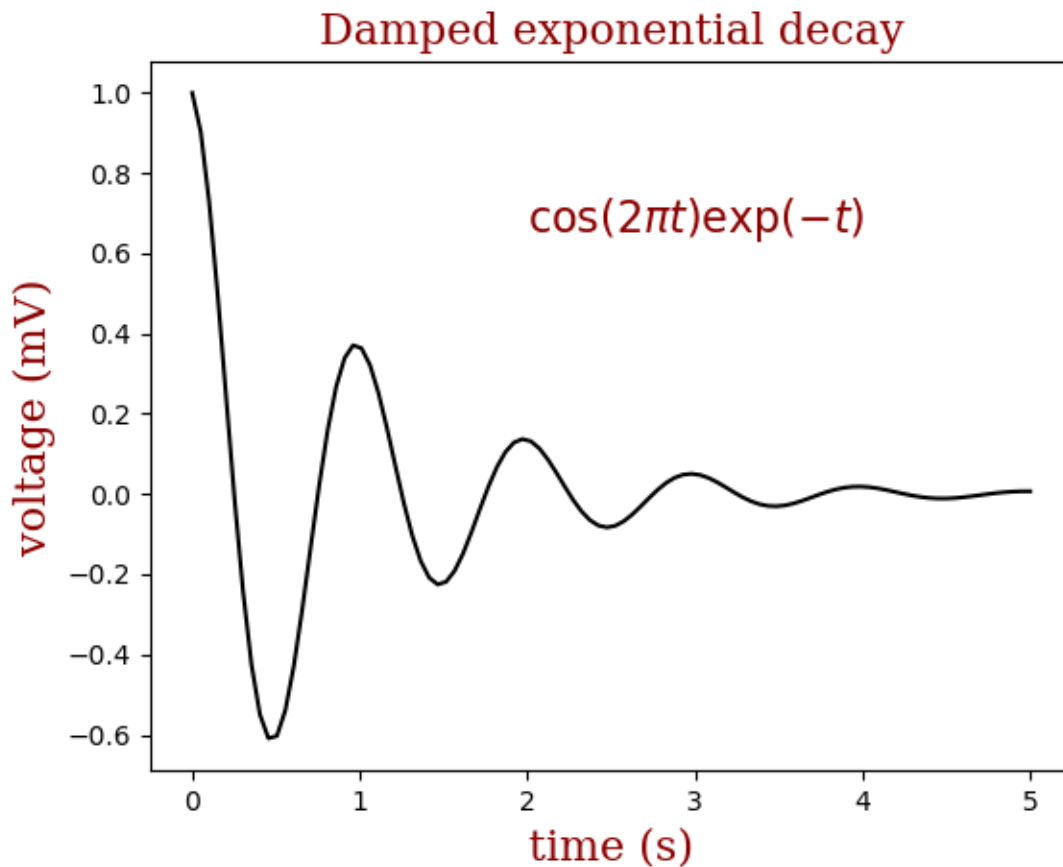
        'color': 'darkred',
        'weight': 'normal',
        'size': 16,
    }

x = np.linspace(0.0, 5.0, 100)
y = np.cos(2*np.pi*x) * np.exp(-x)

plt.plot(x, y, 'k')
plt.title('Damped exponential decay', fontdict=font)
plt.text(2, 0.65, r'$\cos(2 \pi t) \exp(-t)$', fontdict=font)
plt.xlabel('time (s)', fontdict=font)
plt.ylabel('voltage (mV)', fontdict=font)

# Tweak spacing to prevent clipping of ylabel
plt.subplots_adjust(left=0.15)
plt.show()

```



```
# axes_labels_bde-1.py
```

```

import numpy as np
from matplotlib.pyplot import *

# Data
x = np.arange(0,3.02, .02)

y = np.exp(x)

# Create plots with user defined labels.

#figure size and dpi

fig,ax = subplots()

#figure size and dpi

ax.figure = (8,10);
ax.dpi=80

title(r"$Exponential \ and \ Squared \ Changes$", color='red')
ax.plot(x, y, color = 'red' ,linestyle = '-',
        label="$y = e^x$")
ax.plot(x,x**2, color = "blue" , linestyle="--" ,
        label="$x^2$")

# Now add the legend with some customizations.

legend = ax.legend(loc='upper center', shadow=True,
                  bbox_to_anchor=[0.15,0.95],
                  ncol=1, title="Legend", fancybox=True)
legend.get_title().set_color("red")
legend.get_title().set_fontsize(9)

# The frame is matplotlib.patches,
# rectangle instance surrounding the legend.

frame = legend.get_frame()
frame.set_facecolor('white')
frame.set_edgecolor('blue')

numline=0

colors = ("black","red" , "blue" , "green" , "maroon" ,
          "indianred" , "darkturquoise" )

for label in legend.get_texts():

    label.set_fontsize(9)

    label.set_alpha(1)

```

```

numline = numline+1

label.set_color(colors[numline])

if numline >= 6:

    numline = 0

#Set the legend line width

for label in legend.get_lines():
    label.set_linewidth(1.5)

# Simple annotation
ax.annotate('${y = e^x \ (Exponential \ Change)}$',
            color='red', xy=(1.90, 7), xycoords = "data",
            xytext=(0.1,10), textcoords='data',
            arrowprops=dict(facecolor='red', edgecolor='blue',shrink=2),
            fontsize=11)

# Axes limits set after the current data points

if x.min() == 0 :
    xmin = 0.1
else :
    xmin = x.min()

if y.min() == 0 :
    ymin = 0.1
else :
    ymin = y.min()

if x.max() == 0 :
    xmax = 0.1
else :
    xmax = x.max()

if y.max() == 0 :
    ymax = 0.1
else :
    ymax = y.max()

# It is also possible to set our own limits

xlim=(xmin*1.1,xmax*1.1)
ylim=(ymin*1.1,ymax*1.1)

# We can set our own tick points as given below

# User defined major ticks using Latex

xticks([0 , 1.5 , 3],
        ['$0$', '$1.5$', '$3$'], color='red')
yticks([0, 10 , 20],

```

```

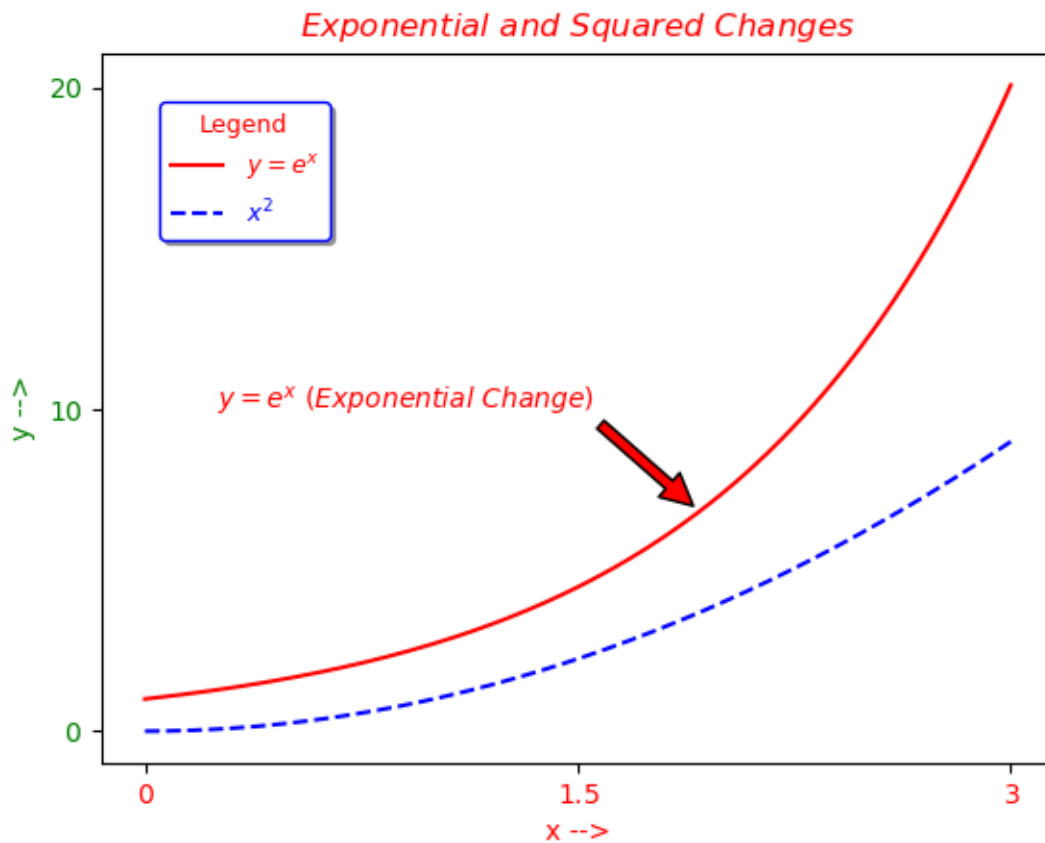
['$0$' , '$10$' , '$20$'], color='green')

xlabel("x -->",color="red")
ylabel("y -->",color="green")

#Display graph

show()

```



Now with Latex

axes_labels_bde-2.py

```

import numpy as np
from matplotlib.pyplot import *

# Data
x = np.arange(0,3.02, .02)

```

```

y = np.exp(x)

# Create plots with user defined labels.

#figure size and dpi

fig,ax = subplots()

#figure size and dpi

ax.figure = (8,12);
ax.dpi=80

title(r"$Exponential \ and \ Squared \ Changes$"+"\n", color='red')
ax.plot(x, y, color = 'red' ,linestyle = '-',
        label="$y = e^x$")
ax.plot(x,x**2, color = "blue" , linestyle="--" ,
        label="$x^2$")

# Now add the legend with some customizations.

legend = ax.legend(loc='upper center', shadow=True,
                  bbox_to_anchor=[0.15,0.95],
                  ncol=1, title="Legend", fancybox=True)
legend.get_title().set_color("red")
legend.get_title().set_fontsize(9)

# The frame is matplotlib.patches,
# rectangle instance surrounding the legend.

frame = legend.get_frame()
frame.set_facecolor('white')
frame.set_edgecolor('blue')

numline=0

colors = ("black","red" , "blue" , "green" , "maroon" ,
         "indianred" , "darkturquoise" )

for label in legend.get_texts():

    label.set_fontsize(9)

    label.set_alpha(1)

    numline = numline+1

    label.set_color(colors[numline])

    if numline >= 6:

```

```

        numline = 0

#Set the legend line width

for label in legend.get_lines():
    label.set_linewidth(1.5)

# Simple annotation

ax.annotate('${y = e^x \ (Exponential \ Change)}$',
            color='red', xy=(1.90, 7), xycoords = "data",
            xytext=(0.1,10), textcoords='data',
            arrowprops=dict(facecolor='red', edgecolor='blue',shrink=2),
            fontsize=11)

# Axes limits set after the current data points

if x.min() == 0 :
    xmin = 0.1
else :
    xmin = x.min()

if y.min() == 0 :
    ymin = 0.1
else :
    ymin = y.min()

if x.max() == 0 :
    xmax = 0.1
else :
    xmax = x.max()

if y.max() == 0 :
    ymax = 0.1
else :
    ymax = y.max()

# It is also possible to set our own limits

xlim=(xmin*1.1,xmax*1.1)
ylim=(ymin*1.1,ymax*1.1)

# We can set our own tick points as given below

# User defined major ticks using Latex

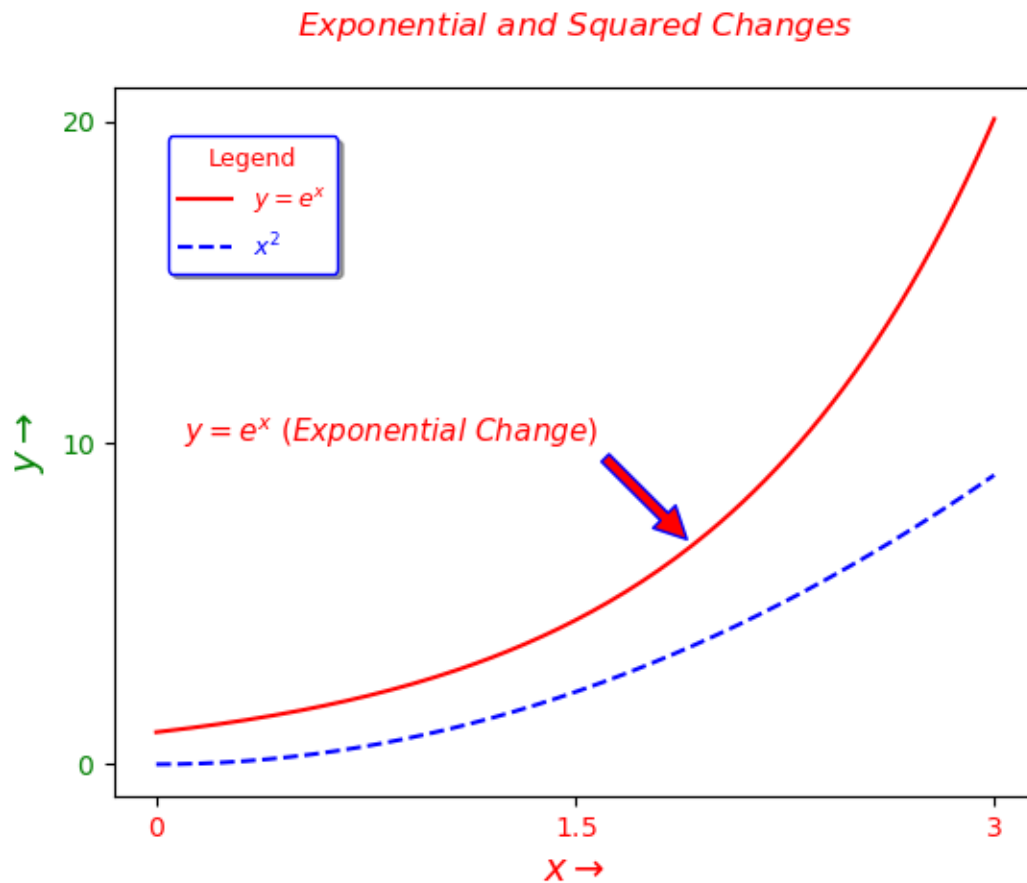
xticks([0 , 1.5 , 3],
        ['$0$' , '$1.5$' , '$3$'], color='red')
yticks([0, 10 , 20],
        ['$0$' , '$10$' , '$20$'], color='green')

xlabel(r"$x \rightarrow$",color="red",fontsize=14)
ylabel(r"$y \rightarrow$",color="green",fontsize=14)

```

```
# Make a room for y axis label
subplots_adjust(left=0.15)

#Display graph
show()
```



Setting graph grids

```
matplotlib.pyplot.grid(b=None, which='major', axis='both', **kwargs)
```

Turn the axes grids on or off.

Set the axes grids on or off; b is a boolean. (For MATLAB compatibility, b may also be a string, 'on' or 'off'.)

If *b* is *None* and `len(kwargs)==0`, toggle the grid state. If *kwargs* are supplied, it is assumed that you want a grid and *b* is thus set to *True*.

which can be ‘major’ (default), ‘minor’, or ‘both’ to control whether major tick grids, minor tick grids, or both are affected.

axis can be ‘both’ (default), ‘x’, or ‘y’ to control which set of gridlines are drawn.

kwargs are used to set the grid line properties, e.g.,:

```
ax.grid(color='r', linestyle='-', linewidth=2)
```

Valid [Line2D](#) kwargs are

Property	Description
agg_filter	a filter function, which takes a (m, n, 3) float array and a dpi value, and returns a (m, n, 3) array
alpha	float (0.0 transparent through 1.0 opaque)
animated	bool
antialiased or aa	[True False]
clip_box	a Bbox instance
clip_on	bool
clip_path	[(Path , Transform) Patch None]
color or c	any matplotlib color
contains	a callable function
dash_capstyle	['butt' 'round' 'projecting']
dash_joinstyle	['miter' 'round' 'bevel']
dashes	sequence of on/off ink in points
drawstyle	['default' 'steps' 'steps-pre' 'steps-mid' 'steps-post']
figure	a Figure instance
fillstyle	['full' 'left' 'right' 'bottom' 'top' 'none']
gid	an id string
label	object
linestyle or ls	['solid' 'dashed', 'dashdot', 'dotted' (offset, on-off-dash-seq) '-' '--' '-.' ':' 'None' ' ' '']
linewidth or lw	float value in points
marker	A valid marker style
markeredgecolor or mec	any matplotlib color

Property	Description
markeredgewidth or mew	float value in points
markerfacecolor or mfc	any matplotlib color
markerfacecoloralt or mfcalt	any matplotlib color
markersize or ms	float
markevery	[None int length-2 tuple of int slice list/array of int float length-2 tuple of float]
path_effects	AbstractPathEffect
picker	float distance in points or callable pick function <code>fn(artist, event)</code>
pickradius	float distance in points
rasterized	bool or None
sketch_params	(scale: float, length: float, randomness: float)
snap	bool or None
solid_capstyle	['butt' 'round' 'projecting']
solid_joinstyle	['miter' 'round' 'bevel']
transform	a matplotlib.transforms.Transform instance
url	a url string
visible	bool
xdata	1D array
ydata	1D array
zorder	float

total-bde-1.py

```

import numpy as np
from matplotlib.pyplot import *

# Data
x = np.arange(0,3.02, .02)

y = np.exp(x)

# Create plots with user defined labels.

#figure size and dpi

```

```

fig,ax = subplots()

#figure size and dpi

ax.figure = (8,12);
ax.dpi=80

title(r"$Exponential \ and \ Squared \ Changes$"+ "\n",
      color='red',fontsize=12)
ax.plot(x, y, color = 'red' ,linestyle = '-',
        label="$y = e^x$")
ax.plot(x,x**2, color = "blue" , linestyle="--" ,
        label="$x^2$")

# Now add the legend with some customizations.

legend = ax.legend(loc='upper center', shadow=True,
                  bbox_to_anchor=[0.15,0.95],
                  ncol=1, title="Legend", fancybox=True)
legend.get_title().set_color("red")
legend.get_title().set_fontsize(9)

# The frame is matplotlib.patches,
# rectangle instance surrounding the legend.

frame = legend.get_frame()
frame.set_facecolor('white')
frame.set_edgecolor('blue')

numline=0

colors = ("black","red" , "blue" , "green" , "maroon" ,
          "indianred" , "darkturquoise" )

for label in legend.get_texts():

    label.set_fontsize(9)

    label.set_alpha(1)

    numline = numline+1

    label.set_color(colors[numline])

    if numline >= 6:

        numline = 0

#Set the legend line width

for label in legend.get_lines():

```

```

label.set_linewidth(1.5)

# Simple annotation
ax.annotate('${y = e^x \ (Exponential \ Change)}$',
            color='red', xy=(1.90, 7), xycoords = "data",
            xytext=(0.1,10), textcoords='data',
            arrowprops=dict(facecolor='red', edgecolor='blue',shrink=2),
            fontsize=11)

# Axes limits set after the current data points

if x.min() == 0 :
    xmin = 0.1
else :
    xmin = x.min()

if y.min() == 0 :
    ymin = 0.1
else :
    ymin = y.min()

if x.max() == 0 :
    xmax = 0.1
else :
    xmax = x.max()

if y.max() == 0 :
    ymax = 0.1
else :
    ymax = y.max()

# It is also possible to set our own limits

xlim=(xmin*1.1,xmax*1.1)
ylim=(ymin*1.1,ymax*1.1)

# We can set our own tick points as given below

# User defined major ticks using Latex

xticks([0 , 1.5 , 3],
        ['$0$', '$1.5$', '$3$'],
        color='red',fontsize=9)
yticks([0, 10 , 20],
        ['$0$', '$10$', '$20$'],
        color='green',fontsize=9)

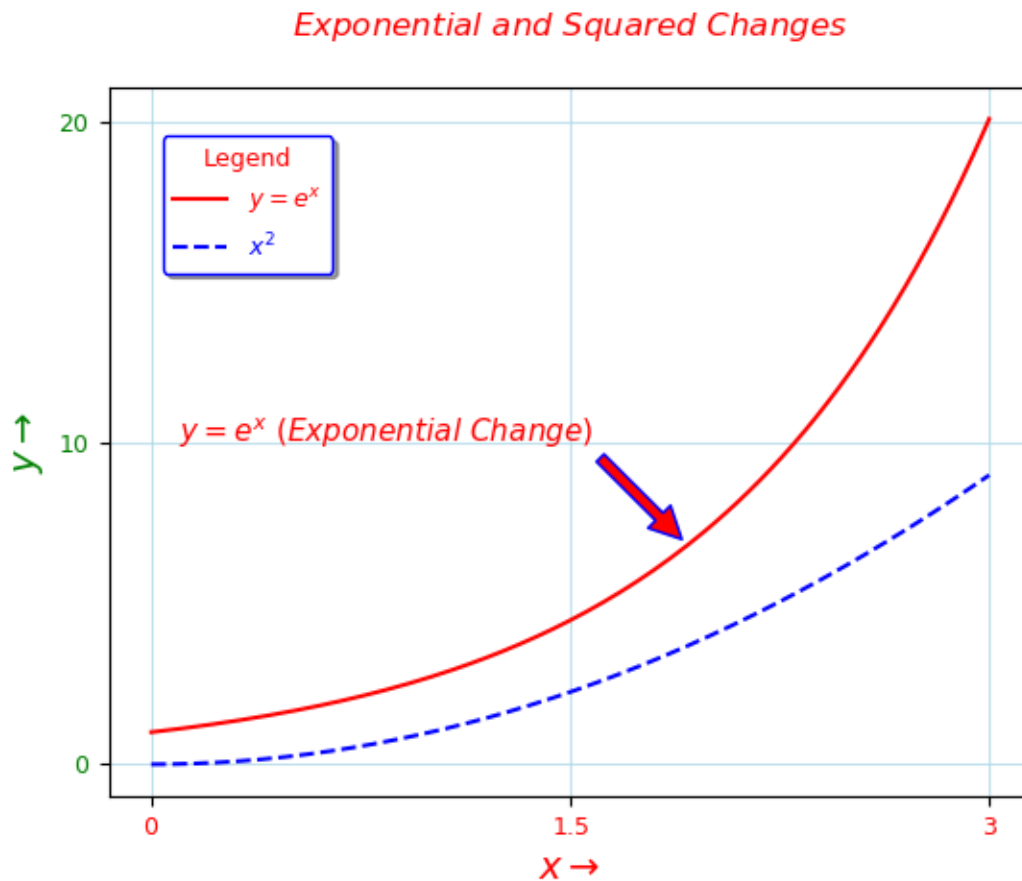
xlabel(r"$x \rightarrow$",color="red",fontsize=14)
ylabel(r"$y \rightarrow$",color="green",fontsize=14)

```

```
# Make a room for y axis label
subplots_adjust(left=0.15)

# Setting graph grids
grid(color="lightblue", alpha=0.75)

#Display graph
show()
```



step-by-step-bde-1.py

```
from numpy import *
from matplotlib.pyplot import *

# Setting the dimensions and position of the curve

# Creating a plot whose dimensions are:

# width = 8 points , height = 6 points
```

```

# resolution = 80 dot per inch

figure(figsize = (8 , 6) , dpi = 80)

# Place of the plot in a subplot area

subplot(111) # This means the graph will be
# generated in the whole plotting area

# Setting the domain of the curve

# Note : The domain consist only from the numbers.

"""
numpy.linspace
This function is similar to arange() function.
In this function, instead of step size,
the number of evenly spaced values between the interval
is specified. The usage of this function is as follows:
numpy.linspace(start, stop, num, endpoint, retstep, dtype)
The constructor takes the following parameters:

start: The starting value of the sequence.

stop: The end value of the sequence, included in the
sequence if endpoint set to true.

num: The number of evenly spaced samples to be generated. Default is 50:

endpoint: True by default, hence the stop value is included in
the sequence. If false, it is not included.

retstep: If true, returns samples and step between
the consecutive numbers:

dtype: Data type of output ndarray.
"""

x = linspace(0 , 2*pi , 1000 , endpoint = True)

# Now the y values (ranges) of two different graphs
# of the two different functions sharing a common domain
# Note the double assignment

sin , cos = sin(x) , cos(x)

# Convert x to radians

x = x / pi

# Plotting characteristics of the functions

# Plot sin(x) with a continuous red line whose width is 1 pixel

plot(x , sin , color = "red" , linestyle = '-' , linewidth = 1.0 ,

```

```

    label = 'sin(x)')

# Plot cos(x) with a continuous green line whose width is 1 pixel

plot(x , cos, color = "green" , linestyle = '-' , linewidth = 1.0 ,
     label = 'cos(x)')

plot(0.75,-0.7071, color = 'red', marker="o" ,
     markersize=5 , markeredgecolor='green')
# set x limits

# x Axes limits set after the current data points

if x.min() == 0 :
    xmin = 0.1
else :
    xmin = x.min()

if x.max() == 0 :
    xmax = 0.1
else :
    xmax = x.max()

xlim=(xmin*1.1,xmax*1.1)

# Set x ticks

xticks([0, 0.25, 0.50 , 0.75 , 1 , 1.25 , 1.50 , 1.75 , 2],
[r"$0$" , r"$\frac{\pi}{4}$" , r"$\frac{\pi}{2}$",
r"$\frac{3\pi}{4}$", r"$\pi$", r"$\frac{5\pi}{4}$",
r"$\frac{3\pi}{2}$", r"$\frac{7\pi}{4}$", r"$2\pi$"],
fontsize=14,color='red')

#Set xlabel

xlabel(r'$\underline{\text{radians}}$ \ \longrightarrow' , color ="red")

# Set y axes limits

ylim(-1-0.15,1*1.1)

# Set y ticks

yticks ([-1,-0.75,-0.50,-0.25,0,0.25,0.50,0.75,1],
[r'$-1$', r'$-0.75$', r'$-0.5$',
r'$-0.25$',r'$0$',r'$0.25$', r'$0.5$',
r'$0.75$', r'$1$'],
fontsize=11,color='green')

```

```

#Set ylabel

ylabel(r'$\sin(x) \ / \ \cos(x) \ \text{longrightarrow}$' ,
      fontsize=11, color = 'green')

# Set Plot Title

title(r'$\text{One} \ \text{Period} \ \text{of} \ \sin(x) \ \text{and} \ \cos(x)$' + '\n',
      fontsize=18,color='red')


legend=legend(fancybox=True, loc='upper center',
              shadow=True, edgecolor='blue',
              framealpha=1, frameon=True,
              bbox_to_anchor=[0.5,0.95],
              title="Legend")

legend.get_title().set_color("red")

legend.get_title().set_fontsize(9)

legend.get_title().set_color("red")

legend.get_title().set_fontsize(9)
# The frame is matplotlib.patches,
# rectangle instance surrounding the legend.

frame = legend.get_frame()
frame.set_facecolor('white')
frame.set_edgecolor('blue')


# The frame is matplotlib.patches,
# rectangle instance surrounding the legend.

frame = legend.get_frame()
frame.set_facecolor('white')
frame.set_edgecolor('blue')

# Set the font size and font color
#Note that : It is very difficult to obtain
#colored legend texts.
#This demo is the arrangement of the unique demo
#found only in Matplotlib user guide legend demo

numline=0

colors = ("black", "red" , "blue" , "green" , "maroon" ,
          "indianred" , "darkturquoise" )

for label in legend.get_texts():

```

```

label.set_fontsize(9)

label.set_alpha(1)

numline = numline+1

label.set_color(colors[numline])

if numline == 2:

    label.set_color("green")

if numline >= 6:

    numline = 0

# Annotation

annotate(r'$\cos(\frac{3\pi}{4})=-0.7071$',
          color='green',xy=(0.75,-0.71) , xycoords='data',xytext=(0,-1),
textcoords='data',
          arrowprops=dict(facecolor='green', arrowstyle="->",
                          connectionstyle="arc3,rad=.2",edgecolor='green'),fontsize=11)

# Setting graph grids

grid(color="turquoise", alpha=0.75)

# Make a room for x and y axes labels

subplots_adjust(left=0.15,bottom=0.15)

# Save figure using 72 dots per inch

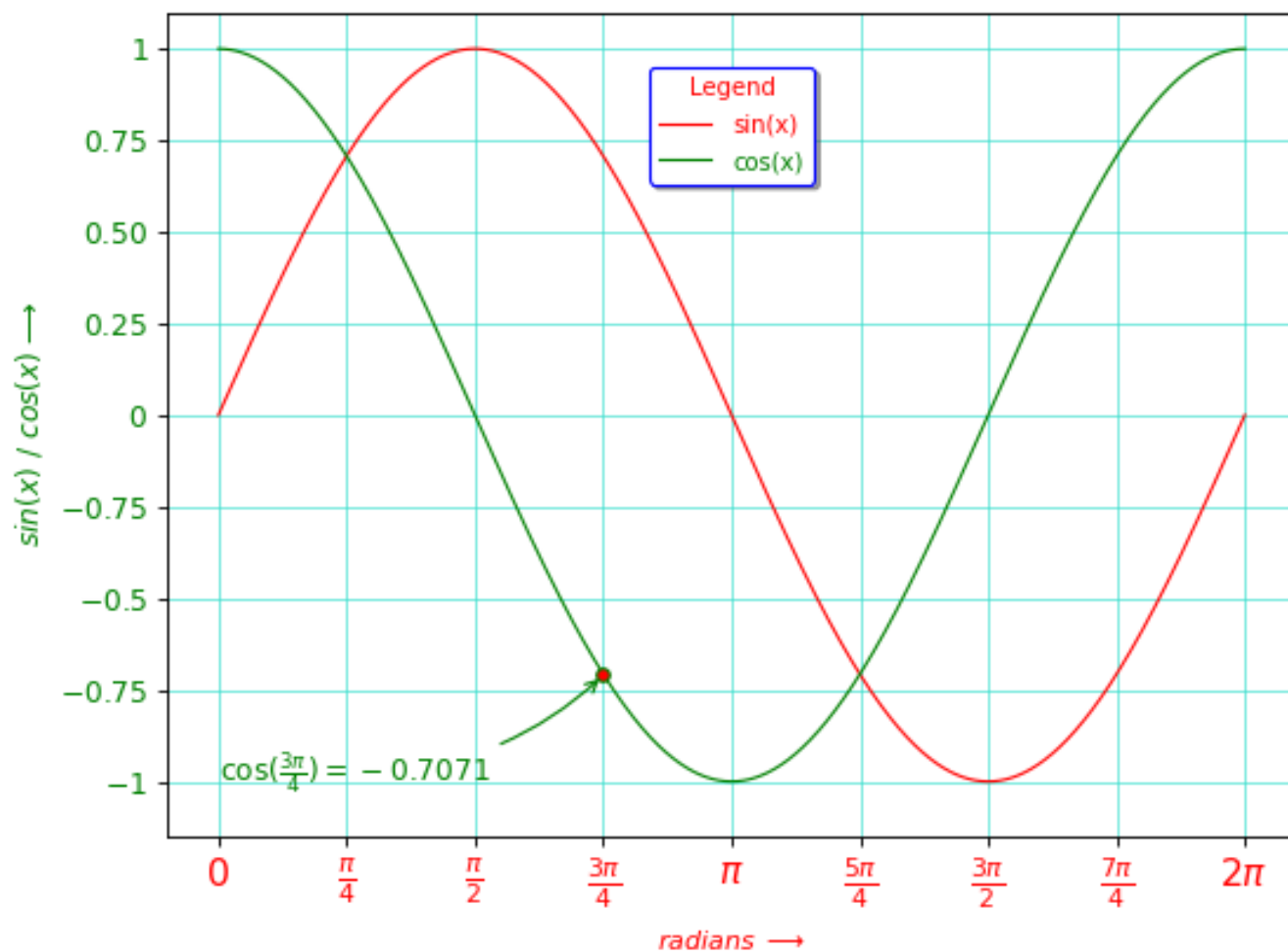
#savefig("step-by-step-bde-1.png", dpi=72)

# Display graph

show()

```

One Period of $\sin(x)$ and $\cos(x)$



tan(pi).py

```
# Imports
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Create a new figure of size 8x6 points, using 80 dots per inch
```

```

plt.figure(figsize=(8,6), dpi=80)
plt.subplot(111)
plt.xlabel(r'$x \ (\text{radians})$' + r'
 $\xrightarrow{\hspace{1cm}}$ ', fontsize=12, color="red", labelpad=10)
plt.ylabel( r'$\tan(x)$' + r'  $\xrightarrow{\hspace{1cm}}$ ', fontsize=12, labelpad=10, color="red")
plt.title("$ Variation \ of \ the \ \tan(x) \ in \ a \ Double \ Period$ \n",
          fontsize=20, color='red')

X = np.linspace(-2*np.pi, 2*np.pi, 500, endpoint=True)
T = np.tan(X)

X = X/np.pi

# Plot tangent using blue color with a continuous line of width 1 (pixels)
plt.plot(X, T, color="red", linewidth=2.0, linestyle="-",
         label="$\tan(x)$")

plt.legend(fontsize=12, loc='upper center', facecolor="white",
          frameon=True, edgecolor='indianred', fancybox=True)

# Set x and y limits
plt.xlim(-2, 2)

plt.xticks([-2, -1.5, -1, -0.5, 0, 0.5, 1, 1.5, 2]
           , [r'$-2\pi$', r'$-1.5\pi$', r'$-\pi$', r'$-0.5\pi$', r'$0$',
             r'$+0.5\pi$', r'$+\pi$', r'$+1.5\pi$',
             r'$+2\pi$'], color="red", fontsize=12)
plt.ylim(-20, 20)
plt.yticks(color="red", fontsize=12)

#plt.yticks( [color="red"])

# Save figure using 72 dots per inch
# savefig("../figures/exercice_2.png", dpi=72)

# Show result on screen

plt.grid(color='turquoise', alpha=0.6)

# Make a room for x and y axes labels

plt.subplots_adjust(left=0.15, bottom=0.15)

# Save figure using 72 dots per inch

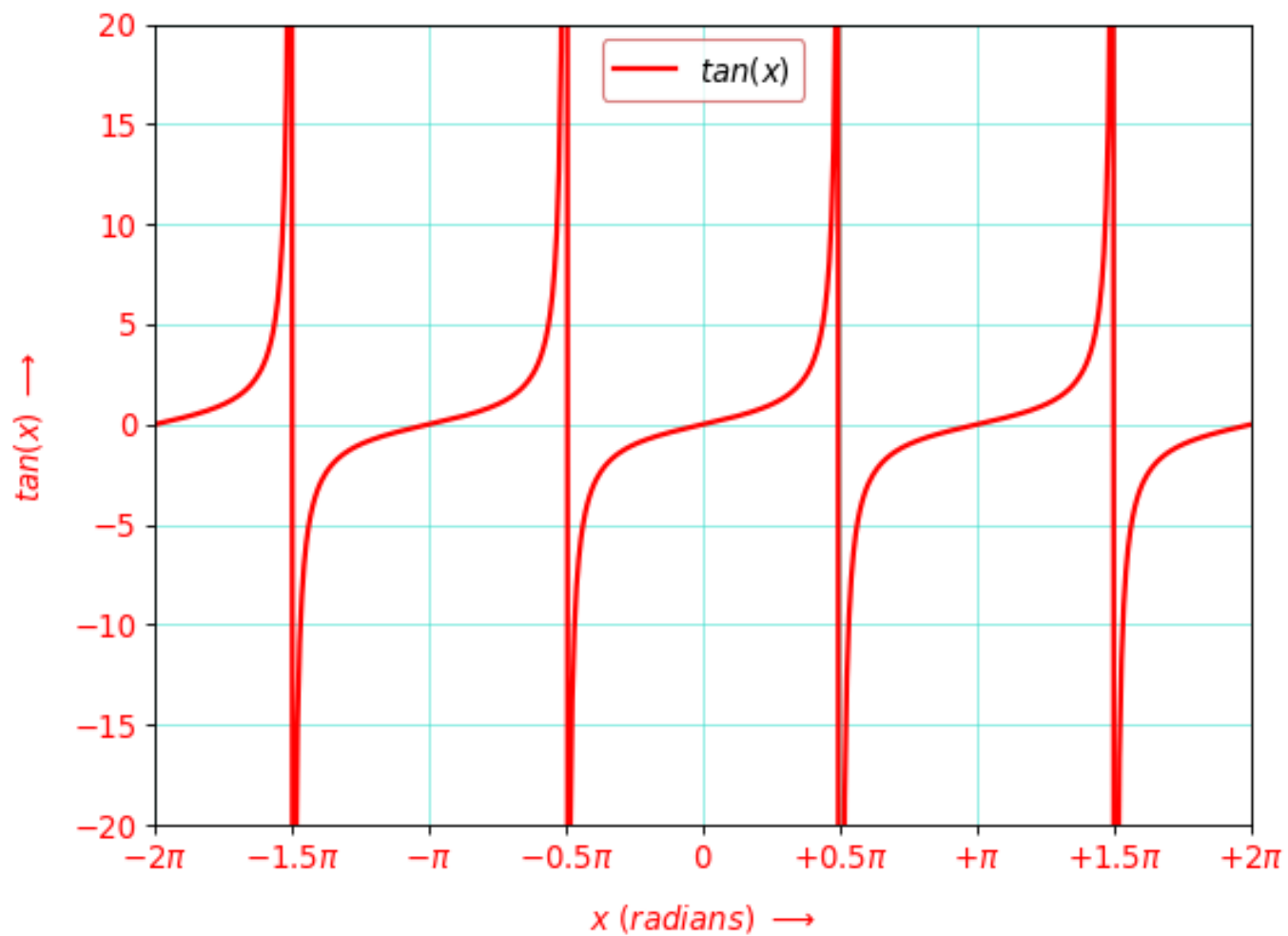
#savefig("tan(x).png", dpi=72)

# Display graph

plt.show()

```

Variation of the $\tan(x)$ in a Double Period



tg(x)-alternative.py

```
# Imports

from numpy import *
from matplotlib.pyplot import *

# Create a new figure of size 8x6 points, using 80 dots per inch
fig = figure(figsize=(8, 6), dpi = 80)
ax = fig.add_subplot(111)
```

```

# x - y data

x = linspace(-2*pi, 2*pi, 500,endpoint=True)

y = tan(x)

x = x / pi

# Plot tangent values using blue color with a continuous line of width 1 (pixels)

plot(x , y , color="red", linewidth=2.0, linestyle="-",
      label="$\tan(x)$")

# Setting tick positions

ax.xaxis.set_ticks_position('bottom')

ax.yaxis.set_ticks_position('left')

# Setting axes limits

ax.set_xlim(-2 ,2)

ax.set_ylim(-30, 30)

# Setting tickers and ticklabels

#xticker = arange(-2, 2, 0.25)

xticker =linspace(-2,2,9)

xlabels = [ r"${-2\pi}$",r"${\frac{-3\pi}{2}}$",
            r"${-\pi}$",r"${\frac{-\pi}{2}}$",r'$0$',
            r"${\frac{\pi}{2}}$",r"${\pi}$",
            r"${\frac{3\pi}{2}}$", r"${2\pi}$"]

yticker = arange(-1 , 1 , 0.25)

ylabels = [r"$-1$" , r"$-0.25$" , r"$-0.5$",
            r"$-0.75$" , r"$0$" , r"$0.25$",
            r"$0.50$" , r"$0.75$" , r"$1$",]

ax.set_xticklabels(xlabels, size=14,color='red')

ax.set_yticklabels(ylabels, size=12,color='green')

xlabel(r'$x \ (\text{radians})$' + r'
$\xrightarrow{\hspace{1cm}}$',fontsize=12,color="red",labelpad=10)

```

```

ylabel( r'$\tan(x)$'+ r' $\rightarrow$', fontsize=12, labelpad=10, color="red")

# Legends

legend = ax.legend(loc='upper center', shadow=True,
                  bbox_to_anchor=[0.25,0.95],
                  ncol=1, title="Legend :", fancybox=True)

legend.get_title().set_color("red")

legend.get_title().set_fontsize(9)

frame = legend.get_frame()

frame.set_facecolor('white')

frame.set_edgecolor('blue')

numline=0

colors = ("black", "red", "blue", "green", "maroon",
          "indianred", "darkturquoise" )

for label in legend.get_texts():

    label.set_fontsize(9)

    label.set_alpha(1)

    numline = numline+1

    label.set_color(colors[numline])

    if numline >= 6:

        numline = 0

# Set the legend line width

for label in legend.get_lines():

    label.set_linewidth(1.5)

# Set graph title

title("$ Variation \ of \ the \ tan(x) \ in \ [-2\pi,2\pi]$",
      fontsize=20, color='red')

```

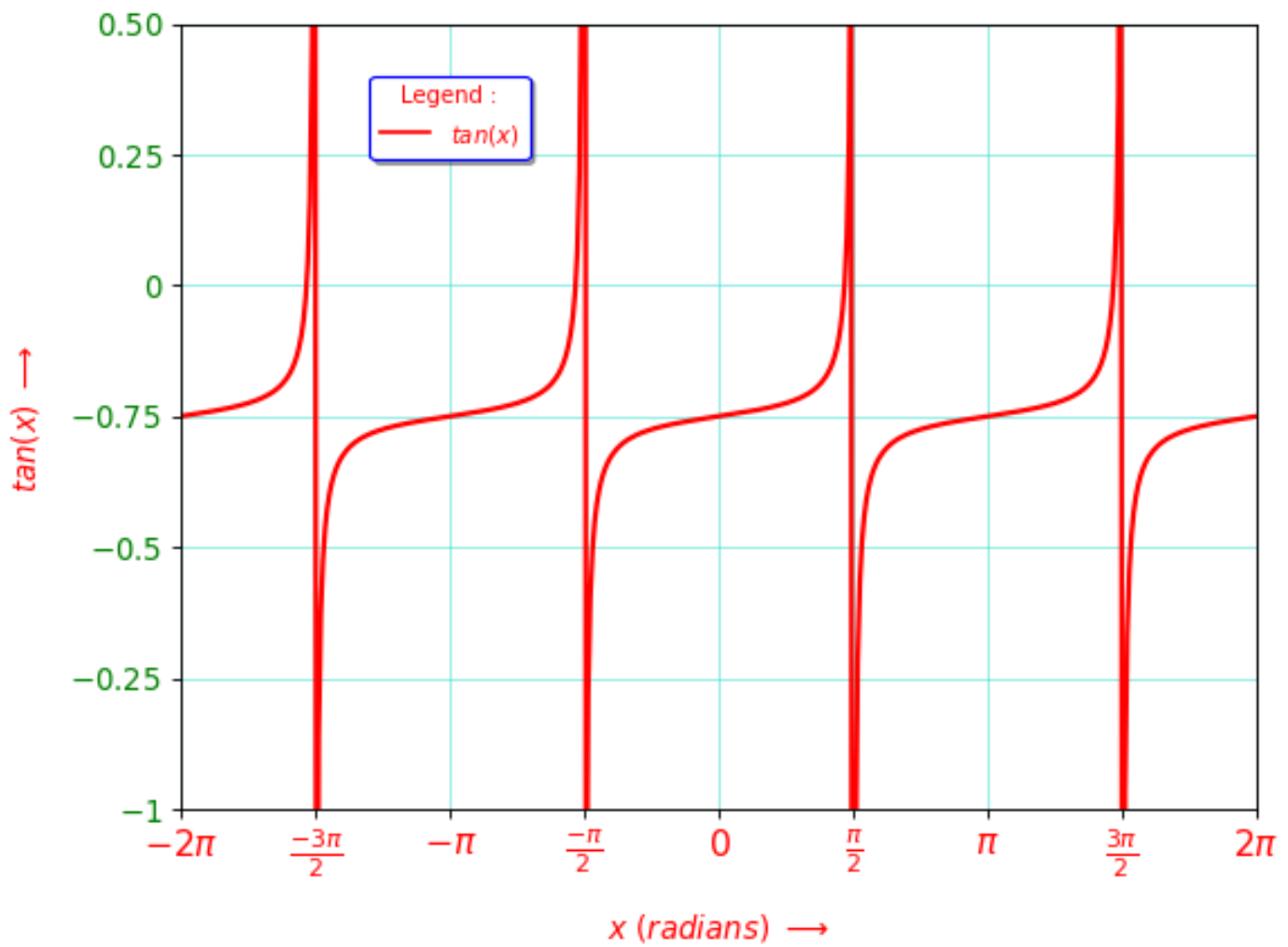
```
# Set the graph grid
grid(color='turquoise', alpha=0.6,)

# Make a room for x and y axes labels
subplots_adjust(left=0.15,bottom=0.15)

# Save figure using 72 dots per inch
#savefig("tan(x)-alternative.png", dpi=72)

# Display graph
show()
```

Variation of the $\tan(x)$ in $[-2\pi, 2\pi]$



Now Applications, from simple to highly decorated

`simplesincos.py`

```
from numpy import *
from matplotlib.pyplot import *
```

```
x=linspace(-3*pi, 3*pi, 1000, endpoint=True)
```

```
y=sin(x)
```

```
Q=cos(x)
```

```

x=x/pi

fig, ax = subplots()

ax.plot(x,y, label=r'$sin(x)$',color="red")
ax.plot(x,Q , label=r'$cos(x)$',color="blue")

legend=legend(fancybox=True, loc='upper center',
              shadow=True, facecolor="lightblue",edgecolor='blue',
              framealpha=0.3, frameon=True,
              bbox_to_anchor=[0.1,0.95],
              title="Legend")

legend.get_title().set_color("red")

legend.get_title().set_fontsize(9)

legend.get_title().set_color("red")

legend.get_title().set_fontsize(9)
# The frame is matplotlib.patches,
# rectangle instance surrounding the legend.

frame = legend.get_frame()
frame.set_facecolor('white')
frame.set_edgecolor('blue')

# The frame is matplotlib.patches,
# rectangle instance surrounding the legend.

frame = legend.get_frame()
frame.set_facecolor('white')
frame.set_edgecolor('blue')

# Set the font size and font color
#Note that : It is very difficult to obtain
#colored legend texts.
#This demo is the arrangement of the unique demo
#found only in Matplotlib user guide legend demo

numline=0

colors = ("black","red" , "blue" , "green" , "maroon" ,
          "indianred" , "darkturquoise" )

for label in legend.get_texts():

    label.set_fontsize(9)

    label.set_alpha(1)

```

```

numline = numline+1

label.set_color(colors[numline])

if numline >= 6:
    numline = 0

xlabel(r'$\underline{\text{radians}} \ \ \rightarrow$',color='blue')

xticks([-3,-2,-1,0,1,2,3],
        ['$-3\pi$', '$-2\pi$', '$-\pi$',
         '$0$', '$\pi$', '$2\pi$', '$3\pi$'],
        color='red',fontsize=9)

yticks([-1.00,-0.75,-0.50,-0.25,0,0.25,0.50,0.75,1.00],
        ['$-1.00$', '$-0.75$', '$-0.50$', '$-0.25$', '$0$',
         '$0.25$', '$0.50$', '$0.75$', '$1.00$'],
        color='green',fontsize=9)

ylabel(r'$\sin(x)/\cos(x) \ \ \rightarrow$',color='blue')

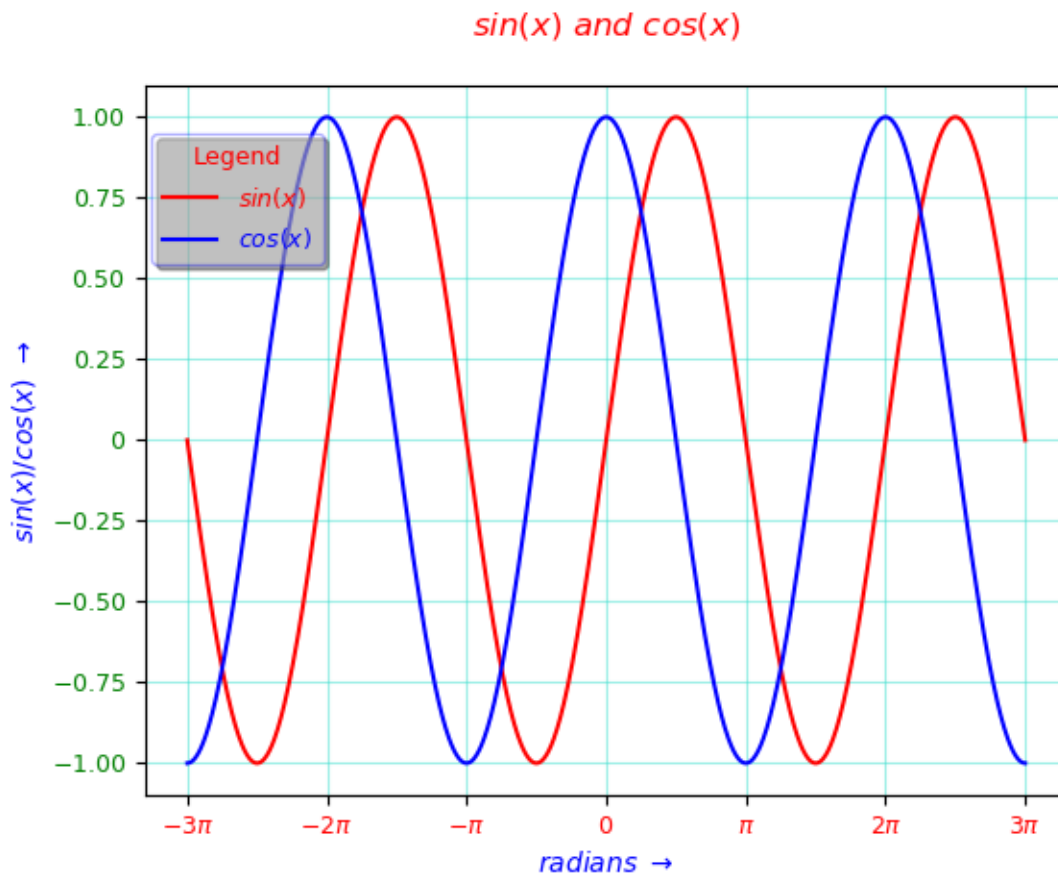
title(r'$\sin(x) \ \ \text{and} \ \ \cos(x)$' +'\n', color='red',fontsize=12)

grid(color="turquoise", alpha=0.5)

subplots_adjust(left=0.15)

show()

```



xsquare.py

```
import numpy as np
from matplotlib.pyplot import *

# Data
x = np.linspace(-3 , 3 , endpoint = True)

a = 1

b = 0

y = a * x**2 + b

# Create plots with default ticks.

#figure size and dpi

fig,ax = subplots()
```

```

ax.figure = (8,12);
ax.dpi=80

title(r"$y = x^2 \ , \ y = 3x^2-3 \ and \ y = 5x^2+10$"+"\n",
      color='red',fontsize=12)
ax.plot(x, y, color = 'red' ,linestyle = '-',
        label="{y = x^2}$")
ax.plot(x,3*x**2-3, color = "blue" , linestyle="--" ,
        label="{3x^2-3}$" )
ax.plot(x,5*x**2+10, color = "maroon" , linestyle="-. " ,
        label="{5x^2+10}$" )

# legend

legend = ax.legend(loc='upper center', shadow=True,
                  bbox_to_anchor=[0.35,0.95],
                  ncol=1, title="Legend", fancybox=True)
legend.get_title().set_color("red")
legend.get_title().set_fontsize(9)

# The frame is the rectangle instance surrounding the legend.

frame = legend.get_frame()
frame.set_facecolor('white')
frame.set_edgecolor('blue')

numline=0

colors = ("black","red" , "blue" , "green" , "maroon" ,
          "indianred" , "darkturquoise" )

for label in legend.get_texts():

    label.set_fontsize(9)

    label.set_alpha(1)

    numline = numline+1

    label.set_color(colors[numline])

    if numline >= 6:

        numline = 0

# Set the legend line width

for label in legend.get_lines():
    label.set_linewidth(1.5)

```

```

# Axes limits set after the current data points

if x.min() == 0 :
    xmin = 0.1
else :
    xmin = x.min()

if y.min() == 0 :
    ymin = 0.1
else :
    ymin = y.min()

if x.max() == 0 :
    xmax = 0.1
else :
    xmax = x.max()

if y.max() == 0 :
    ymax = 0.1
else :
    ymax = y.max()

xlim=(xmin*1.1,xmax*1.1)
ylim=(ymin*1.1,ymax*1.1)

xlabel(r"$x \rightarrow$",color="red",fontsize=14)
ylabel(r"$y \rightarrow$",color="green",fontsize=14)

# Simple annotation
annotate(r'${Vertex}$',
        color='red', xy=(0,10) , xycoords='data',
        xytext=(-1,20) , textcoords='data',
        arrowprops=dict(facecolor='red', arrowstyle="->",
        edgecolor='red') , fontsize=11)

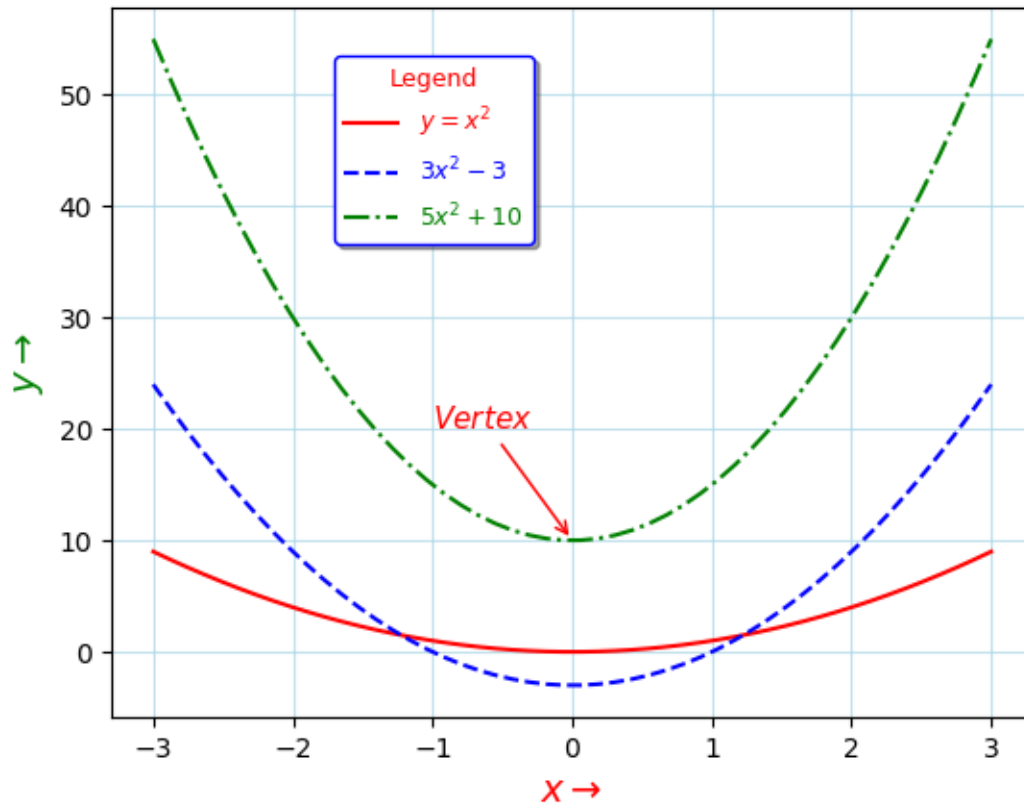
# Make a room for y axis label
subplots_adjust(left=0.15)

# Setting graph grids
grid(color="lightblue", alpha=0.75)

# Display graph
show()

```

$$y = x^2, y = 3x^2 - 3 \text{ and } y = 5x^2 + 10$$



Note that : That in $ax^2 + b$, a parameter is responsible to the widening

of the second degree graph and the b parameter is responsible to the

gap of the graph vertex from the origin.

How do you make publication grade graphs

First we will try the simplest default generated plot for our function.

Especially we will give our domain limits and inspect the produced ranges.

```
# tryout third_degree.py
```

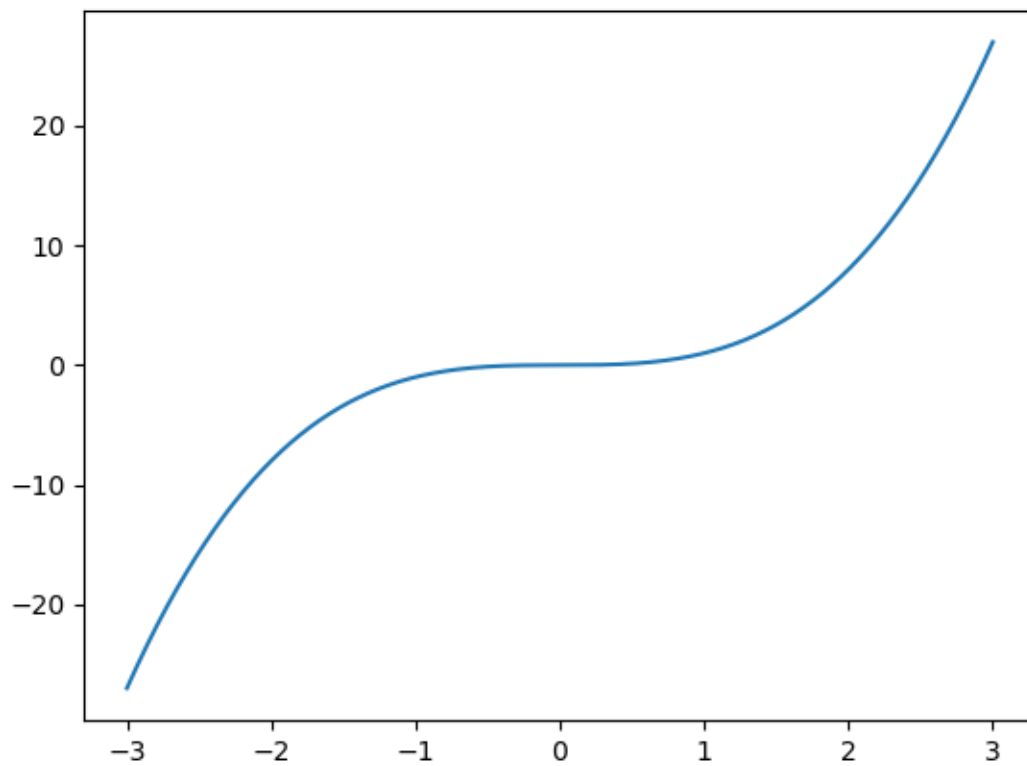
```
from numpy import *  
from matplotlib.pyplot import *
```

```
x = linspace(-3, 3, 1000, endpoint=True)
```

```
y = x**3
```

```
plot(x,y)
```

```
show()
```



Now, we'll choose from the well worked graph templates,
modify it for our needs and and voila.

third_degree.py

```
import numpy as np
from matplotlib.pyplot import *

# Data
x = np.linspace(-3 , 3 , endpoint = True)

a = 1

b = 0

y = x**3

# Create plots with user defined ticks.

#figure size and dpi

fig,ax = subplots()

ax.figure = (8,12);
ax.dpi=80

title(r"$y = x^3$", color='red', fontsize=12)

ax.plot(x, y, color = 'red' ,linestyle = '-',
        label="{y = x^3}")

# legend

legend = ax.legend(loc='upper center', shadow=True,
                  bbox_to_anchor=[0.35,0.95],
                  ncol=1, title="Legend", fancybox=True)
legend.get_title().set_color("red")
legend.get_title().set_fontsize(9)

# The frame is the rectangle instance surrounding the legend.

frame = legend.get_frame()
frame.set_facecolor('white')
frame.set_edgecolor('blue')

numline=0
```

```

colors = ("black","red" , "blue" , "green" , "maroon" ,
          "indianred" , "darkturquoise" )

for label in legend.get_texts():

    label.set_fontsize(9)

    label.set_alpha(1)

    numline = numline+1

    label.set_color(colors[numline])


    if numline >= 6:

        numline = 0


# Set the legend line width

for label in legend.get_lines():
    label.set_linewidth(1.5)

# Axes limits set after the current data points

if x.min() == 0 :
    xmin = 0.1
else :
    xmin = x.min()

if y.min() == 0 :
    ymin = 0.1
else :
    ymin = y.min()


if x.max() == 0 :
    xmax = 0.1
else :
    xmax = x.max()

if y.max() == 0 :
    ymax = 0.1
else :
    ymax = y.max()

xlim=(xmin*1.1,xmax*1.1)
ylim=(ymin*1.1,ymax*1.1)

xlabel(r"$x \rightarrow$",color="red",fontsize=14)

```

```
ylabel(r"$y \rightarrow$",color="green",fontsize=14)
```

```
xticks([-3,-2,-1,0,1,2,3],
        ['$-3$', '$-2$', '$-1$',
         '$0$', '$1$', '$2$', '$3$'],
        color='red',fontsize=9)
```

```
yticks([-20,-10,0,10,20],
        ['$-20$', '$-10$', '$0$', '$10$', '$20$'],
        color='green',fontsize=9)
```

```
# Make a room for y axis label
```

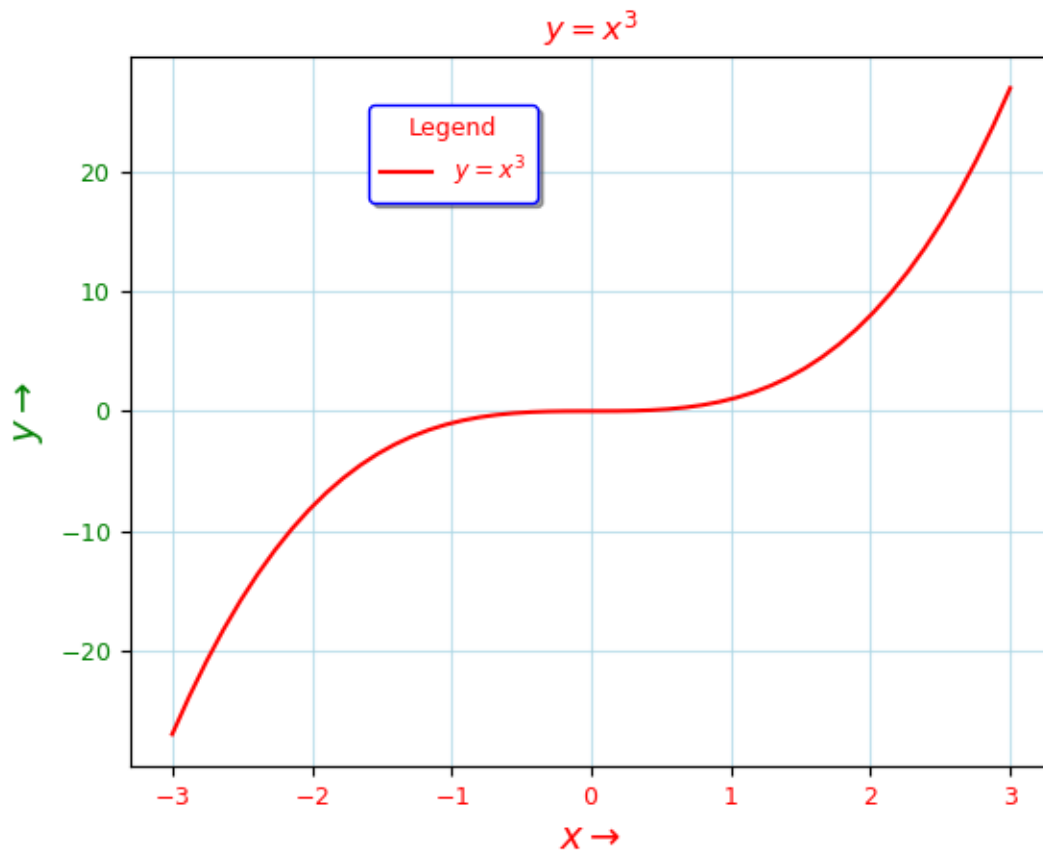
```
subplots_adjust(left=0.15)
```

```
# Setting graph grids
```

```
grid(color="lightblue", alpha=0.75)
```

```
# Display graph
```

```
show()
```



log_exp.py

```

from numpy import *
from matplotlib.pyplot import *

# Data
x = np.linspace(0 , 2 , 1000,endpoint = True)

y = log(x)
y1=log(exp(x))
y2=exp(x)

# Create plots with user defined ticks.

#figure size and dpi

fig,ax = subplots()

ax.figure = (8,12);

```

```

ax.dpi=80

title(r"$Inverse \ Functions \ : \ y = \log(x) \ and \ y = \exp(x)$" + "\n",
color='red', fontsize=12)

ax.plot(x, y, color='red', linestyle='-',
        label="$y = \log(x)$")
ax.plot(x, y1, color='blue', linestyle='-.',
        label="$y = \log(\exp(x))=x$")
ax.plot(x, y2, color='green', linestyle='-',
        label="$y = \exp(x)$")
# legend

legend = ax.legend(loc='upper center', shadow=True,
                  bbox_to_anchor=[0.25,1],
                  ncol=1, title="Legend", fancybox=True)
legend.get_title().set_color("red")
legend.get_title().set_fontsize(9)

# The frame is the rectangle instance surrounding the legend.

frame = legend.get_frame()
frame.set_facecolor('white')
frame.set_edgecolor('blue')

numline=0

colors = ("black", "red", "blue", "green", "maroon",
          "indianred", "darkturquoise")

for label in legend.get_texts():
    label.set_fontsize(9)
    label.set_alpha(1)
    numline = numline+1
    label.set_color(colors[numline])

    if numline >= 6:
        numline = 0

# Set the legend line width

for label in legend.get_lines():
    label.set_linewidth(1.5)

# Axes limits set after the current data points

```

```

if x.min() == 0 :
    xmin = 0.1
else :
    xmin = x.min()

if y.min() == 0 :
    ymin = 0.1
else :
    ymin = y.min()

if x.max() == 0 :
    xmax = 0.1
else :
    xmax = x.max()

if y.max() == 0 :
    ymax = 0.1
else :
    ymax = y.max()

xlim=(xmin*1.1,xmax*1.1)
ylim=(ymin*1.1,ymax*1.1)

xlabel(r"$x \rightarrow$",color="red",fontsize=14)
ylabel(r"$y \rightarrow$",color="green",fontsize=14)

xticks([0, 0.5 , 1 , 1.5 , 2],
        ['$0$', '$0.5$', '$1$',
         '$1.5$', '$2$'],
        color='red',fontsize=9)

yticks([-6,-4,-2,0,2,4,6],
        ['$-6$', '$-4$', '$-2$', '$0$', '$2$', '$4$', '$6$'],
        color='green',fontsize=9)

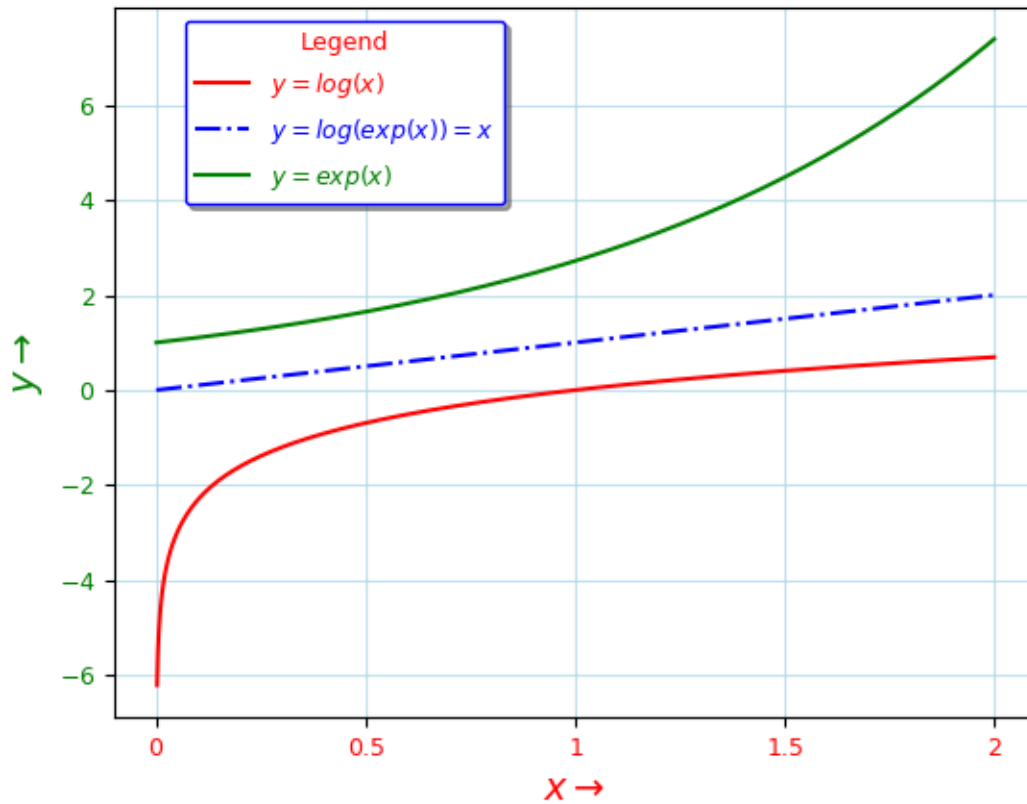
# Make a room for y axis label
subplots_adjust(left=0.15)

# Setting graph grids
grid(color="lightblue", alpha=0.75)

# Display graph
show()

```

Inverse Functions : $y = \log(x)$ and $y = \exp(x)$



sincos-bde-4.py

```
from numpy import *
from matplotlib.pyplot import *

#http://matplotlib.sourceforge.net/examples/pylab_examples/spine_placement_demo.html
#http://mple.m-artwork.eu/home/posts/sinsinesinus

fig = figure(figsize=(6, 6))
ax = fig.add_subplot(111)

x = arange(-2*np.pi, 2*np.pi, 0.01)
y = cos(x)

ax.xaxis.set_ticks_position('bottom')
ax.yaxis.set_ticks_position('left')

ax.plot(x, y, linewidth="3", alpha=0.3, label = r'$ y = \cos(x) $', color="red")
```

```

ax.plot([-2*pi,2*pi], [0, 0], ls="-", color="green", linewidth="3",alpha=0.5)
ax.plot([0, 0], [-1.5, 1.5], ls="-", color="green", linewidth="3",alpha=0.5)
ax.plot([0,2*pi],[-1.25,-1.25] , 'ro')
ax.plot([0, pi/2], [-1.25, -1.25], ls="-", color="green", linewidth="3",alpha=0.5)
ax.plot([1.25, 2*pi], [-1.25, -1.25], ls="-", color="green", linewidth="3",alpha=0.5)

ax.set_xlim(-2*pi ,2*pi)
ax.set_ylim(-1.5, 1.5)

xticker = arange(-2.0*pi, 2.5*pi, pi/2)
xlabels = [ r"${-2\pi}$",r"$\frac{-3\pi}{2}$", r"${-\pi}$",r"$\frac{-\pi}{2}$",r"$0$",r"$\frac{\pi}{2}$",r"${\pi}$",r"$\frac{3\pi}{2}$", r"${2\pi}$"]

yticker = arange(-1, 1.5, 0.5)
ylabels = ["-1","-0.5","0","0.5","1"]

ax.set_yticks(yticker)

ax.set_xticks(xticker)
ax.set_xticklabels(xlabels, size=14,color='red')
ax.set_yticklabels(ylabels, size=12,color='green')

ax.text(0.6, 1.1, r"$One \ Period \ of \ y=\cos(x)$", color='red',fontsize=12)
ax.text(2.2, -1.2, r"$One \ Period$", color='red',fontsize=12)

legend = ax.legend(loc='upper center', shadow=True,
                  bbox_to_anchor=[0.15,0.95],
                  ncol=1, title="Legend :", fancybox=True)
legend.get_title().set_color("red")
legend.get_title().set_fontsize(9)

frame = legend.get_frame()
frame.set_facecolor('white')
frame.set_edgecolor('blue')

numline=0

colors = ("black","red" , "blue" , "green" , "maroon" ,
          "indianred" , "darkturquoise" )

for label in legend.get_texts():

    label.set_fontsize(9)

    label.set_alpha(1)

    numline = numline+1

```

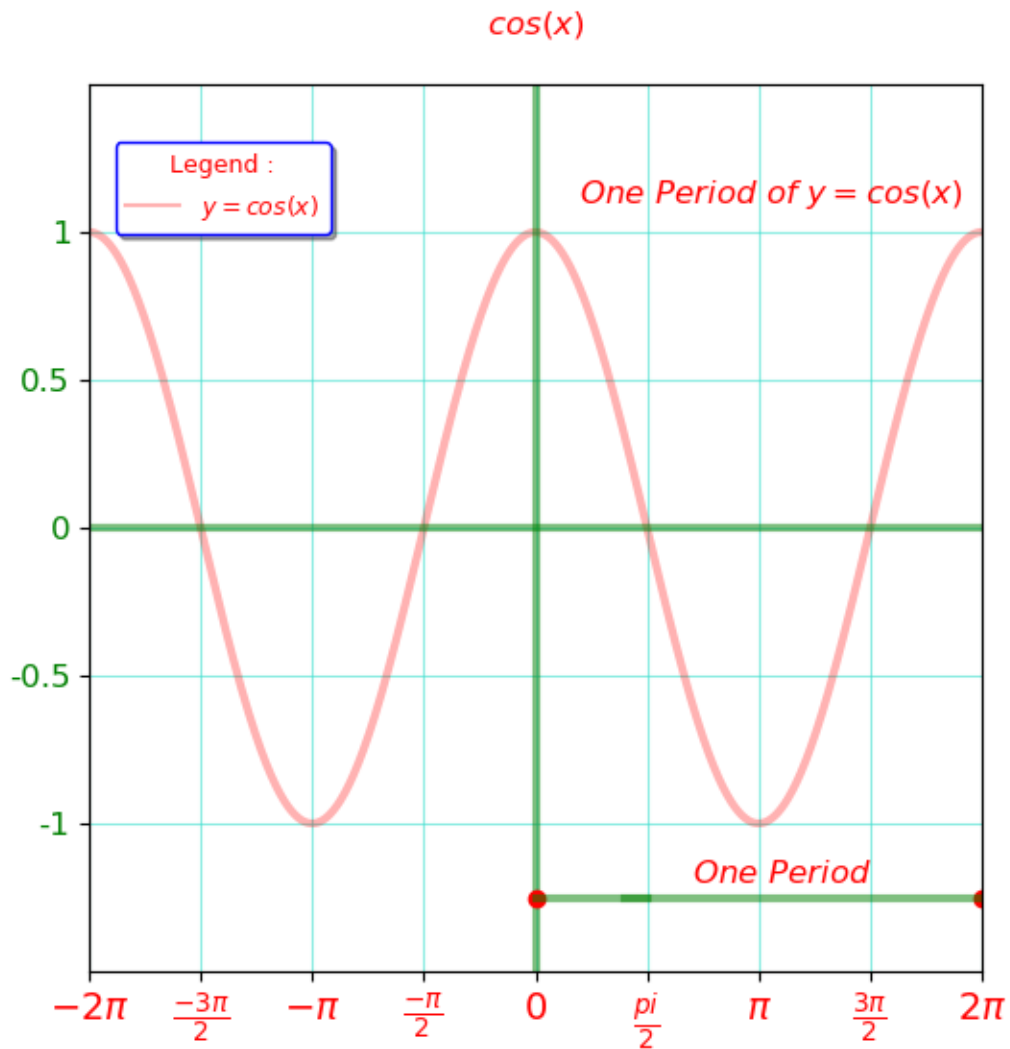
```
label.set_color(colors[numline])

if numline >= 6:
    numline = 0

#Set the legend line width
for label in legend.get_lines():
    label.set_linewidth(1.5)

title(r'$\cos(x)$'+'\n',color='red',fontsize=12)
grid(color='turquoise', alpha=0.6,)

show()
```

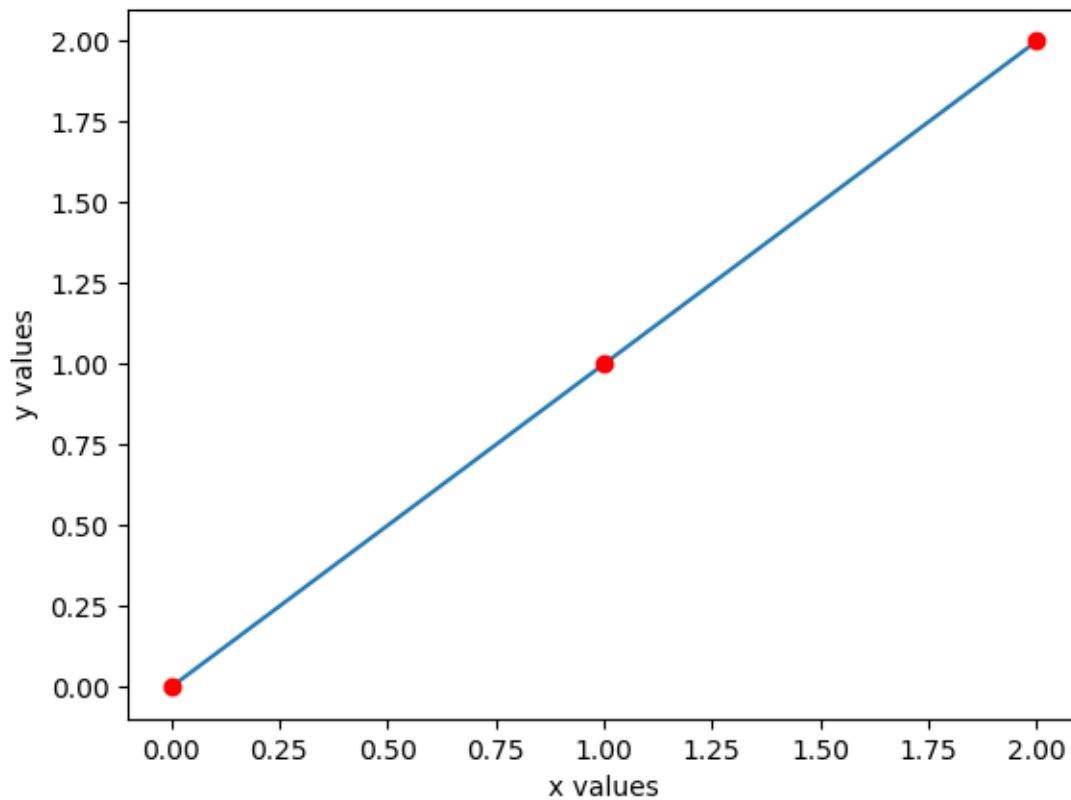


Some scatter graphs

line2.py

a single line graph with explicit (x,y) data and the points together

```
from matplotlib.pyplot import *  
  
# Data  
X = [0,1,2]  
Y = [0,1,2]  
Q = [0,1,2]  
  
# Axis labels  
ylabel('y values')  
xlabel('x values')  
  
# Axis plots  
plot(X,Y)  
plot(X,Q,'ro')  
show()
```



a single line graph with explicit (x,y) data and the points together

line2-light.py

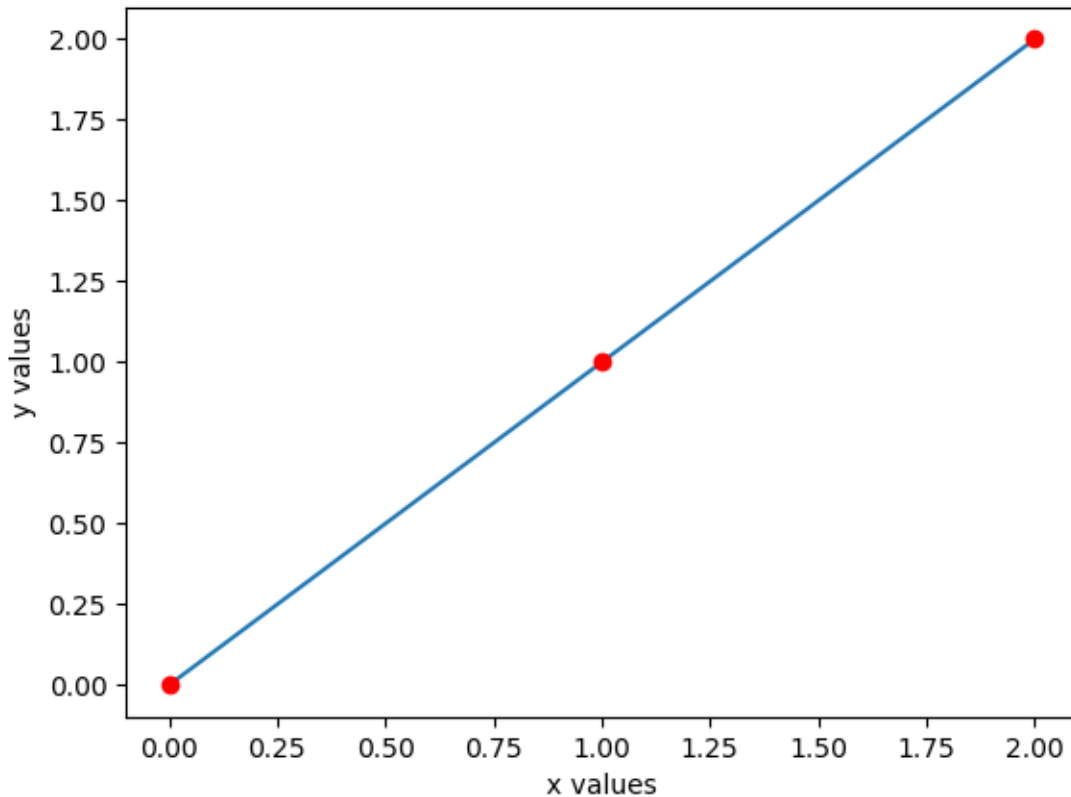
```
import matplotlib.pyplot as plt

# Data
X = [0,1,2]
Y = [0,1,2]
Q = [0,1,2]

# Axis labels
plt.ylabel('y values')
plt.xlabel('x values')

# plots
plt.plot(X,Y)
plt.plot(X,Q,'ro') # ro means redden dot

# display
plt.show()
```



```
# we will continue with the less verbose forms for the  
programs
```

```
continue with the less verbose forms for the programs
```

```
# line styles maybe specified in varieties of ways
```

```
# supported line styles and colors are described in the
```

```
# official matplotlib.pyplot documents in:
```

```
#
```

```
https://matplotlib.org/api/\_as\_gen/matplotlib.pyplot.plot.ht  
ml
```

line-2-blue-hexagon.py

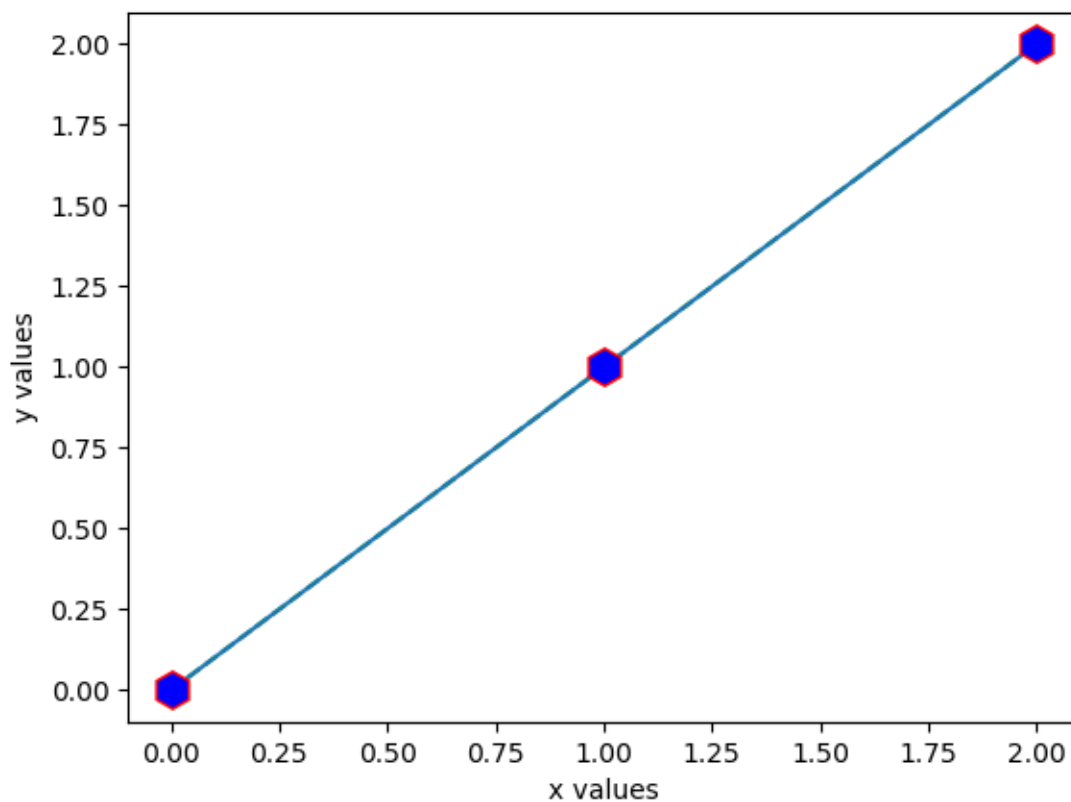
```
from matplotlib.pyplot import *

# Data
X = [0,1,2]
Y = [0,1,2]
Q = [0,1,2]

# Axis labels
ylabel('y values')
xlabel('x values')

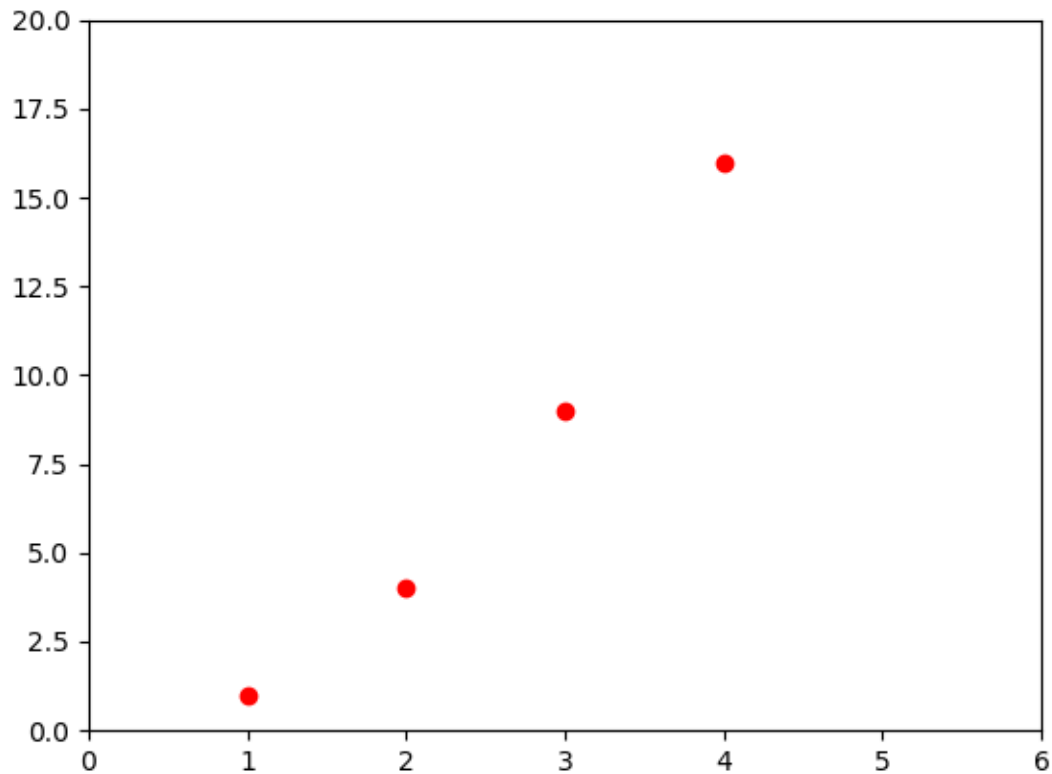
# plots
plot(X,Y,color='green', linestyle='solid')
plot(X,Q, marker='h',
      markerfacecolor='blue', markeredgecolor='red',markersize=14)

# display
show()
```



`#page2reddot.py`

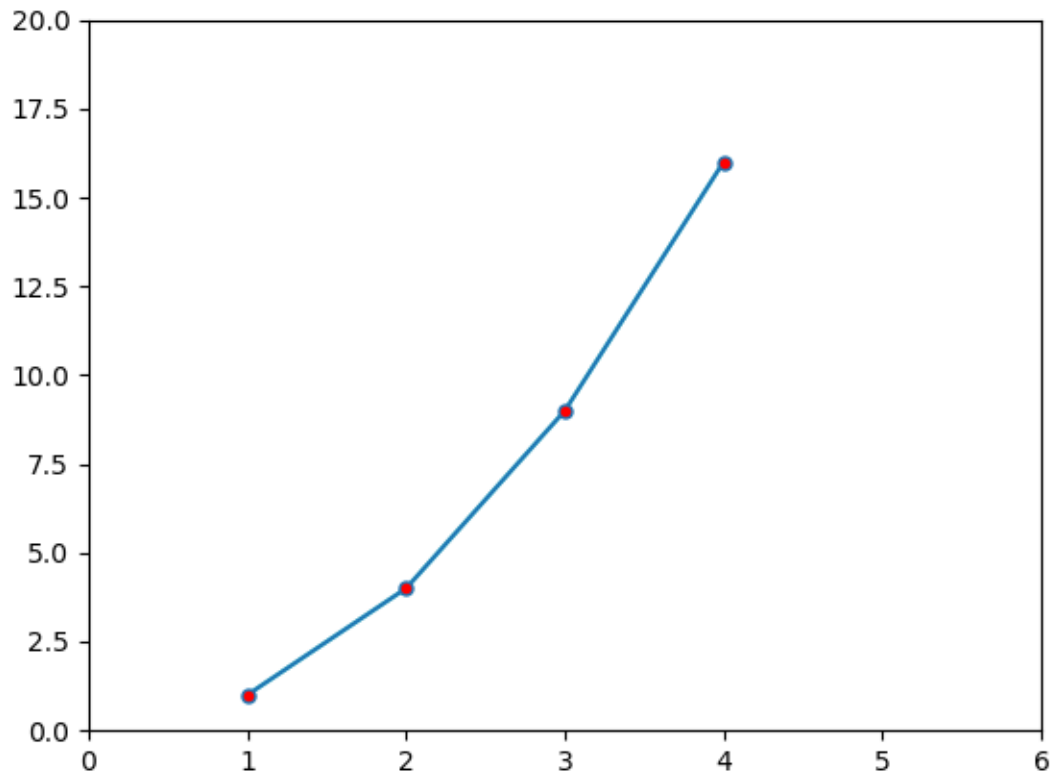
```
from matplotlib.pyplot import *  
  
plot([1,2,3,4],[1,4,9,16], 'ro')  
axis([0,6,0,20])  
  
show()
```



as simple as that, but we have all the possibilities to enhance the tracks

page2reddot2.py

```
from matplotlib.pyplot import *
plot([1,2,3,4],[1,4,9,16], linestyle='solid', color='turquoise')
plot([1,2,3,4],[1,4,9,16],marker='o' , markerfacecolor='red',markersize=5)
axis([0,6,0,20])
show()
```



We will need numpy for mathematical expressions

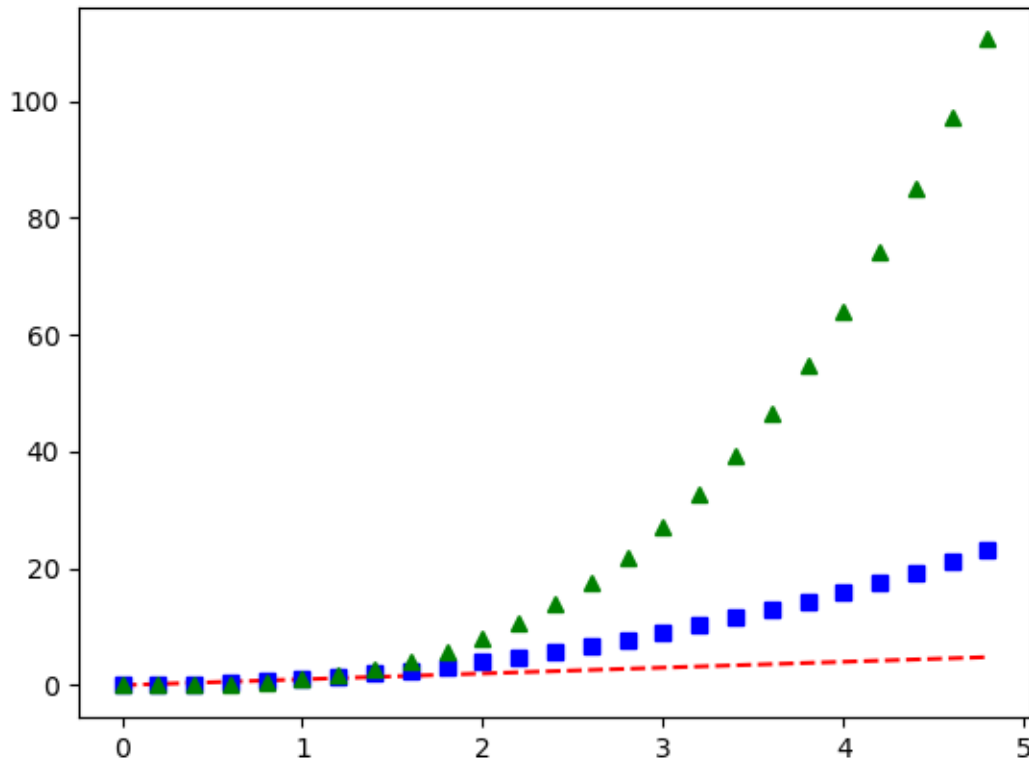
page3reddashes.py

```
from numpy import *
from matplotlib.pyplot import *

t=arange(0,5.0,0.2)

plot(t,t, 'r--', t,t**2, 'bs', t,t**3, 'g^')

show()
```



Quite simple yet separate configurable plots.

`t = arange (0 , 5.0 , 0.2)` means, `tmin = 0` , `tmax = 5`

and the whole range is traversed by the discrete points where the

distance from the previous point is 0.2

These are contiguous not continuous data, digital and not analogue.

We can produce more detailed codes for the same plot

`scatter-bde-1.py`

```
from numpy import *
from matplotlib.pyplot import *

t=arange(0,5.0,0.2)

plot(t,t, color ="red" , linestyle ="dashed")

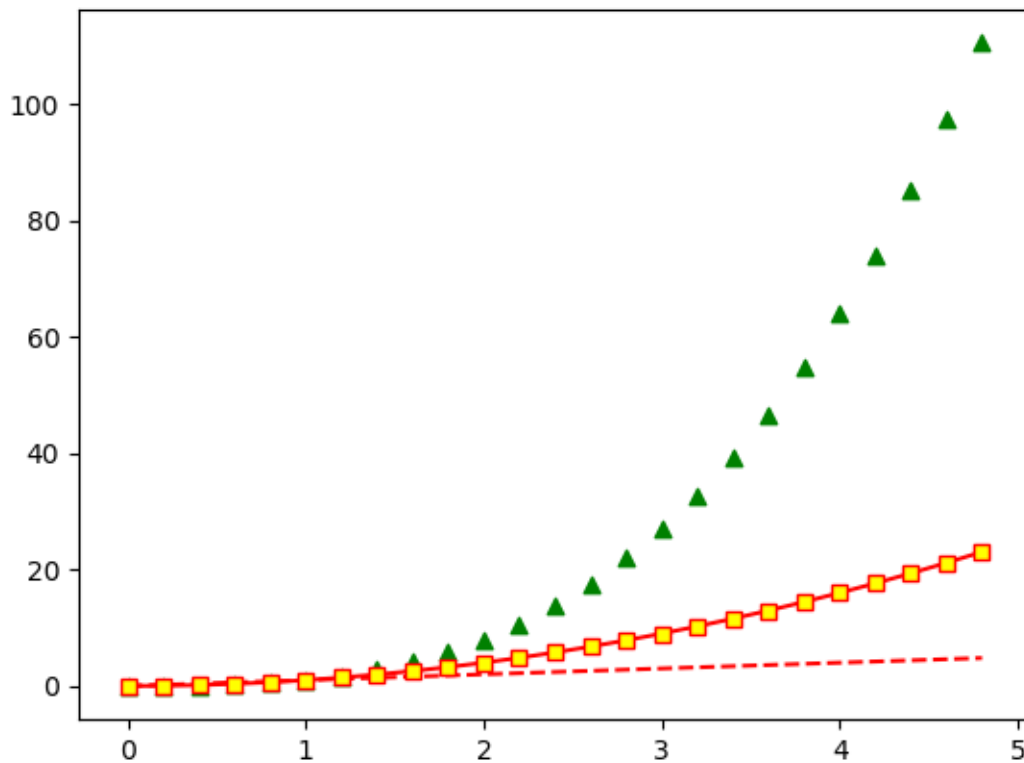
tsquare = t**2

plot(t,tsquare, marker='s' , markerfacecolor ="yellow" , color ="red")

tcube = t**3

scatter(t,tcube, marker = '^', color ="green" )

show()
```



Red dots are provided as the default scatter form

scatter-bde-2.py

```

from matplotlib.pyplot import *
from dask.array.creation import linspace

# Plotting area
fig,ax = subplots()

# Figure size and dpi
ax.figure = (8,12)
ax.dpi = 80

# Data
x = (1,2,3)
y = (1,2,3)
a = (1,3)
b = (1,3)

# plot
plot(a , b , linestyle='--', color = 'turquoise')

# scatter
scatter(x,y, color='red' , label = r"$Reflexive \ Relation$")

# Setting legend position
legend=legend(loc=[0.55,0.25])

# Setting x and y axes labels
xlabel("x --->" , color = 'red')
ylabel("y --->" , color = 'green')

# Setting graph title
title("Reflexive Relation Between Elements \u2192 a Set Consisting of Three Elements ",
      color='red')

# Setting point legends

```

```

for label in legend.get_texts():

    label.set_fontsize(9)

    label.set_alpha(1)

    label.set_color('red')

# Setting the domain and range limits

xmin=min(x); xmax=max(x);
ymin=min(y); ymax=max(y);

xlim=(xmin*1.1,xmax*1.1)
ylim=(ymin*1.1,ymax*1.1)

xticks([1 , 1.5 , 2 , 2.5 , 3],

        color='red',fontsize=9)
yticks([1 , 1.5 , 2 , 2.5 , 3] , color='green',fontsize=9 )

# Make a room for x and y axes labels

subplots_adjust(left=0.15,bottom=0.15)

# Simple annotation

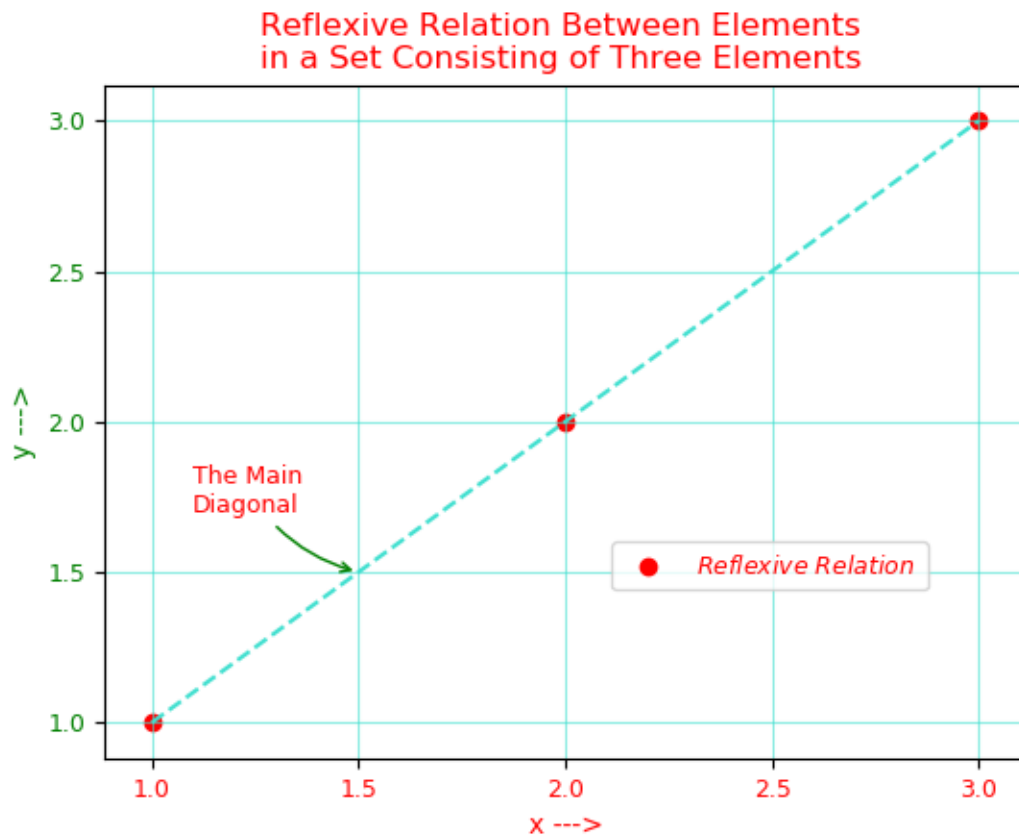
annotate('The Main\nDiagonal',
         color='red',xy=(1.5,1.5) , xytext=(1.1,1.7) ,
         arrowprops=dict(facecolor='red',arrowstyle="->",
         connectionstyle="arc3,rad=.2",edgecolor='green'),fontsize=9)

# Setting graph grids

grid(color='turquoise', alpha=0.6,)

show()

```



scatter-bde-3.py

```
from matplotlib.pyplot import *
from dask.array.creation import linspace

# Plotting area

fig,ax = subplots()

# Figure size and dpi

ax.figure = (8,12)

ax.dpi =80

# Data

x = (1,2,3)

y = (1,2,3)
```

```

a = (1,3)

b= (1,3)

# plot
plot(a , b , linestyle='--', color = 'turquoise')

# scatter

scatter(x,y, marker='o' , facecolor ="red",
        s=20, edgecolor = 'red',
        label =r"$Reflexive \ Relation$")

# Setting legend position

legend=legend(loc=[0.55,0.25])

# Setting x and y axes labels

xlabel(r"$x \ \underline{\longrightarrow}$" , color = 'red')

ylabel(r"$y \ \underline{\longrightarrow}$" , color = 'green')

# Setting graph title

title(r"$Reflexive \ Relation \ Between \ Elements$" + " \n"+
      "$in \ a \ Set \ Consisting \ of \ Three \ Elements$",
      color='red')

# Setting point legends

for label in legend.get_texts():

    label.set_fontsize(9)

    label.set_alpha(1)

    label.set_color('red')

# Setting the domain and range limits

xmin=min(x); xmax=max(x);
ymin=min(y); ymax=max(y);

xlim=(xmin*1.1,xmax*1.1)
ylim=(ymin*1.1,ymax*1.1)

xticks([1 , 1.5 , 2 , 2.5 , 3],
        [r'$1$', r'$1.5$', r'$2$', r'$2.5$', r'$3$'],
        color='red',fontsize=9)

yticks([1 , 1.5 , 2 , 2.5 , 3] ,

```

```

[r'$1$', r'$1.5$', r'$2$', r'$2.5$', r'$3$'],
color='green',fontsize=9 )

# Make a room for x and y axes labels

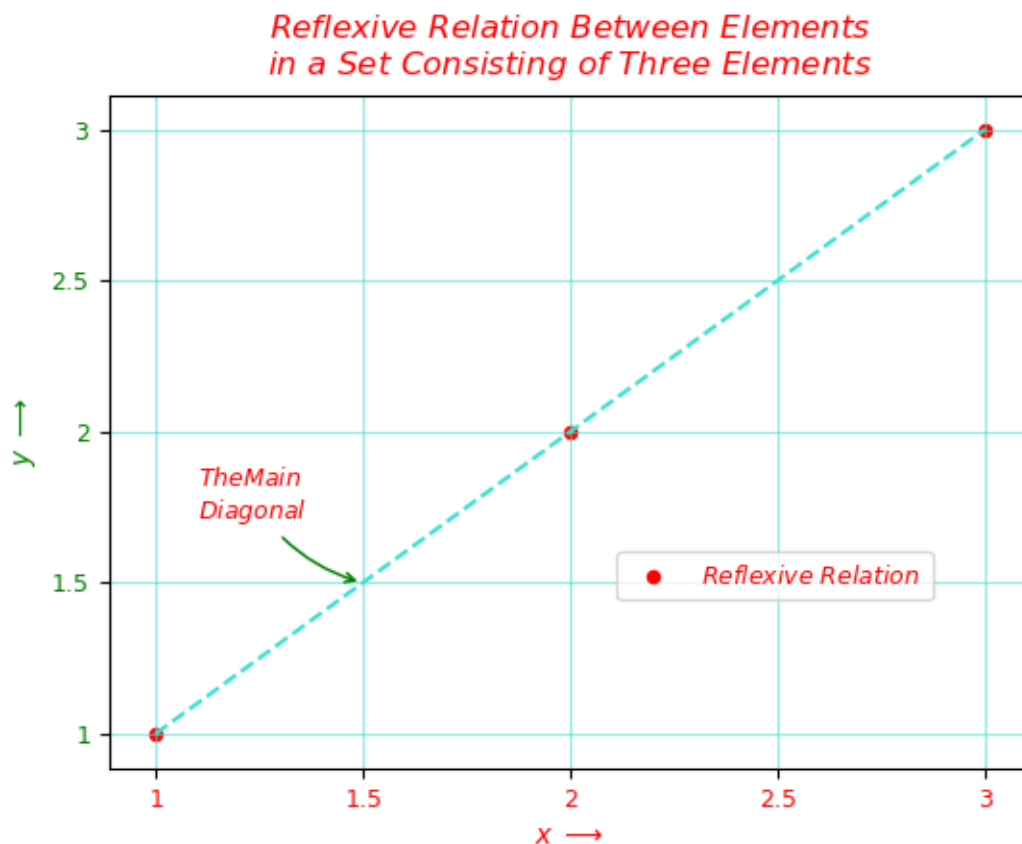
subplots_adjust(left=0.15,bottom=0.15)

# Simple annotation
annotate(r'$The Main$'+'\n'+ r'$Diagonal$', color='red',xy=(1.5,1.5) ,
xycoords = "data", xytext=(1.1,1.7) , textcoords='data',
arrowprops=dict(facecolor='red',arrowstyle="->",
connectionstyle="arc3,rad=.2",edgecolor='green'),fontsize=9)
# Setting graph grids

grid(color='turquoise', alpha=0.6,)

show()

```



markers

[matplotlib.markers](https://matplotlib.org/3.1.1/api/markers_api.html)

This module contains functions to handle markers. Used by both the marker functionality of [plot](#) and [scatter](#).

All possible markers are defined here:

marker	description
"."	point
","	pixel
"o"	circle
"v"	triangle_down
"^"	triangle_up
"<"	triangle_left
">"	triangle_right
"1"	tri_down
"2"	tri_up
"3"	tri_left
"4"	tri_right
"8"	octagon
"s"	square
"p"	pentagon
"P"	plus (filled)
"*"	star
"h"	hexagon1
"H"	hexagon2
"+"	plus
"x"	x
"X"	x (filled)
"D"	diamond
"d"	thin_diamond
" "	vline
"_"	hline
TICKLEFT	tickleft
TICKRIGHT	tickright
TICKUP	tickup
TICKDOWN	tickdown
CARETLEFT	caretleft (centered at tip)

marker	description
CARETRIGHT	caretright (centered at tip)
CARETUP	caretup (centered at tip)
CARETDOWN	caretdown (centered at tip)
CARETLEFTBASE	caretleft (centered at base)
CARETRIGHTBASE	caretright (centered at base)
CARETUPBASE	caretup (centered at base)
"None", " " or ""	nothing
'\$...\$'	render the string using <code>mathtext</code> .
verts	a list of (x, y) pairs used for Path vertices. The center of the marker is located at (0,0) and the size is normalized.
path	a Path instance.
(numsides, style, angle)	The marker can also be a tuple (numsides, style, angle), which will create a custom, regular symbol. numsides: the number of sides style: the style of the regular symbol: 0 a regular polygon 1 a star-like symbol 2 an asterisk 3 a circle (numsides and angle is ignored) angle: the angle of rotation of the symbol

For backward compatibility, the form (verts, 0) is also accepted, but it is equivalent to just verts for giving a raw set of vertices that define the shape.

[None](#) is the default which means ‘nothing’, however this table is referred to from other docs for the valid inputs from marker inputs and in those cases [None](#) still means ‘default’.

Classes

MarkerStyle ([marker, fillstyle])	Parameters:
---	--------------------

